



UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

Olivaris - Plataforma de gestión colaborativa para el olivar

Realizado por
Jorge Cano Melero



Para la obtención del título de
Grado en Ingeniería Informática

Dirigido por
Juan Luis Jiménez Laredo

En el departamento de
Dpto. de Ingeniería de Computadores, Automática y Robótica

Convocatoria de junio, curso 2025/26

Agradecimientos

Quiero agradecer este trabajo a mis padres, por ser los pilares fundamentales de mi vida, los que me han educado y han dado forma a mi personalidad.

Gracias por ayudarme a tomar una decisión tan importante para mí: cambiar el rumbo de mi vida y luchar por lo que uno quiere, independientemente de la edad, afrontando la situación como si de un reto se tratara. Gracias a ellos puedo estar escribiendo estas palabras y estar presentando este Trabajo de Fin de Grado.

No puedo estar más orgulloso de todo lo que habéis hecho por mí y siempre estaré a vuestro lado.

Dar las gracias a otra persona tan importante como lo es mi hermano. Tu personalidad tan fuerte y trabajadora son un impulso para mí y un reflejo donde mirarme. A pesar de ser el pequeño de la familia, demuestras una madurez increíble y siempre estas ayudando a los demás, demostrando ese amor y compromiso que tienes.

Me siento muy orgulloso de a persona en la que te estás convirtiendo y de lo que estás logrando, y que feliz me hace estar presente contigo para poder disfrutarlos juntos.

Y como no iba a poder agradecer desde lo más profundo de mi corazón este proyecto a mi pareja de vida, Carla. No sabría qué agradecerle en tan pocas líneas después de todo lo que has hecho por mí y de todo el apoyo que me has dado desde hace más de 9 años.

Gracias por ser enseñarme a comprender lo que significa la palabra amor, tratándolo como un sentimiento tan especial y que va más allá. Gracias por enseñarme a vivir más, por mirar siempre el lado positivo de las cosas y por agradecer cualquier gesto que nos da la vida.

Y por último, dar las gracias se queda corto al tutor de este Trabajo de Fin de Grado, Juan Luis Jiménez Laredo. Gracias por estar siempre disponible cuando lo he necesitado, gracias por ayudarme en aquellos momentos del proyecto donde estaba perdido y no sabía cómo avanzar, gracias por tu positividad y tu empatía; sin ti este trabajo realizado no tendría tanto significado para mí. Gracias por hacer el proceso enriquecedor, motivarme y confiar en mí.

A todas estas personas, gracias de nuevo por toda la ayuda que me habéis dado; habéis conseguido que disfrute del proceso y que el camino sea mucho más bonito. El orgullo que siento por todos vosotros se queda corto.

Olivaris - Plataforma de gestión colaborativa para el olivar

Jorge Cano Melero

Palabras clave: Cuaderno de Explotación Comercial, Gestión agrícola, Backend, API REST, JWT, SIGPAC, Parcelas, Fitosanitarios.

Resumen:

El presente Trabajo de Fin de Grado aborda el diseño e implementación de Olivaris, una plataforma digital orientada a la gestión colaborativa de explotaciones del olivar. El proyecto surge en un contexto de creciente digitalización del sector agrícola y de evolución normativa en torno al Sistema de Información de Explotaciones Agrarias, el Registro Autonómico de Explotaciones Agrícolas y el Cuaderno Digital de Explotación Agrícola. En este marco, Olivaris se plantea como una herramienta para centralizar la gestión de parcelas, recintos, usuarios, entidades habilitadas y actividades agrícolas, especialmente aquellas relacionadas con el uso de productos fitosanitarios.

La solución desarrollada se basa en una arquitectura cliente-servidor, con un backend que expone una API REST, persistencia en base de datos, autenticación mediante JWT y control de acceso por roles. El sistema permite registrar y visualizar, parcelas, gestionar actividades asociadas a explotaciones y modelar la relación entre agricultores y entidades habilitadas mediante permisos de acceso y gestión.

Aunque la plataforma no realiza una integración efectiva con los servicios oficiales de SIEX/REA, su diseño se ha elaborado teniendo en cuenta la estructura conceptual y los requisitos de información asociados a este marco, de forma que facilite una posible integración futura. Como resultado, se obtiene un prototipo funcional y extensible que contribuye a la digitalización de la gestión del olivar y sirve como base para futuras mejoras, como el funcionamiento *offline*, la generación avanzada de informes o el acceso multiplataforma.

Olivaris - Plataforma de gestión colaborativa para el olivar

Jorge Cano Melero

Keywords: Commercial Farm Logbook, Agricultural Management, Backend, REST API, JWT, SIGPAC, Parcels, Phytosanitary Treatments.

Abstract:

This Final Degree Project addresses the design and implementation of Olivaris, a digital platform aimed at collaborative management of olive grove farms. The project arises in a context of increasing digitalization of the agricultural sector and regulatory evolution around the System of Information of Agricultural Farms, the Regional Registry of Agricultural Farms, and the Digital Farm Operation Logbook. Within this framework, Olivaris is presented as a tool to centralize the management of parcels, plots, users, authorized entities and agricultural activities, especially those related to the use of phytosanitary products.

The developed solution is based on a client-server architecture, with a backend that exposes a REST API, database persistence, authentication through JWT and role-based access control. The system allows registering and viewing parcels, managing activities associated with farms and modeling the relationship between farmers and authorized entities through access and management permissions.

Although the platform does not perform an effective integration with the official SIEX/REA services, its design has been developed taking into account the conceptual structure and information requirements associated with this framework, in order to facilitate a possible future integration. As a result, a functional and extensible prototype is obtained that contributes to the digitalization of olive grove management and serves as a basis for future improvements, such as offline functionality, advanced report generation or cross-platform access.

Yo, **Jorge Cano Melero**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77021264Z, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Jorge Cano Melero

Granada, 18 de junio de 2026

D. **Juan Luis Jiménez Laredo**, Profesor del Área de Arquitectura y Tecnología de Computadores del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Olivaris - Plataforma de gestión colaborativa para el olivar***, ha sido realizado bajo su supervisión por **Jorge Cano Melero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada, 18 de junio de 2026.

El director: Juan Luis Jiménez Laredo

Índice general

1	Introducción	1
1.1.	Estructura de la memoria	2
2	Descripción del problema y alcance del proyecto	4
2.1.	Contexto y motivación	4
2.2.	Descripción del problema	4
2.3.	Solución propuesta	5
2.4.	Objetivos y alcance	5
2.4.1.	Objetivo general	5
2.4.2.	Objetivos específicos	6
2.4.3.	Alcance y limitaciones	6
2.4.4.	Justificación de las decisiones	7
3	Estado del arte	9
3.1.	Contexto del olivar y necesidad de digitalización	9
3.2.	Marco normativo: SIEX, REA y CUE	10
3.3.	Soluciones software existentes para la gestión agrícola	10
3.4.	Comparativa de soluciones y posicionamiento de Olivaris	13
3.4.1.	Análisis del puesto que ocupa Olivaris	13
3.5.	Tecnologías para el desarrollo de plataformas web y móviles	14
3.5.1.	Arquitecturas Software	14
3.5.2.	Comunicación y Servicios Web	20
3.5.3.	Ecosistema del Desarrollo Web	21
3.5.4.	Sistemas de almacenamiento	24
3.5.5.	Despliegue y estrategias	25
3.5.6.	Autenticación y autorización	27
3.5.7.	Elección de arquitectura y tecnologías	28
3.6.	Conclusiones del estado del arte	31
4	Metodología	32
4.1.	Enfoque metodológico	32
4.1.1.	Miembros del equipo Scrum	32
4.1.2.	Artefactos de Scrum	33
4.1.3.	Eventos de Scrum	33
4.1.4.	¿Por qué usar la metodología Scrum?	34
4.1.5.	Adaptación de la metodología Scrum al proyecto	35
4.1.6.	Sprints realizados	36
4.2.	Organización del código y repositorios	38
4.3.	Presupuesto del proyecto	38

5	Análisis y requisitos	41
5.1.	Recopilación de información	41
5.2.	CUE comercial	43
5.3.	Personas	45
5.4.	Escenarios	47
5.5.	Historias de Usuario	49
5.6.	Criterios de aceptación	53
5.7.	Requisitos funcionales	56
5.7.1.	Autenticación y autorización	56
5.7.2.	Entidades habilitadas	57
5.7.3.	Parcelas y recintos	58
5.7.4.	Actividades	58
5.7.5.	Usuarios	60
5.8.	Requisitos no funcionales	60
5.8.1.	Usabilidad	60
5.8.2.	Seguridad	61
5.8.3.	Mantenibilidad	61
5.8.4.	Disponibilidad y Rendimiento	61
6	Diseño y arquitectura	62
6.1.	Arquitectura de alto nivel	62
6.1.1.	Arquitectura monolítica	62
6.1.2.	Componentes principales	63
6.1.3.	Comunicación entre los componentes	64
6.2.	Modelos de datos	65
6.2.1.	Esquema de Base de Datos	65
6.2.2.	Diagrama de clases	71
6.3.	Diseño de la interfaz	72
6.3.1.	Decisiones de diseño	72
6.4.	Diseño de la API	78
6.4.1.	Tecnología utilizada y diseño de endpoints	79
6.4.2.	Autenticación y autorización	89
6.4.3.	API del Visor SIGPAC	90
7	Implementación	95
7.1.	Sprint 1	95
7.1.1.	AuthService	96
7.1.2.	JwtService	98
7.1.3.	EmailService	99
7.1.4.	UserService	100
7.1.5.	Relación entre los componentes	100
7.2.	Sprint 2	107
7.2.1.	EntityService	109
7.2.2.	Migraciones de Bases de datos	112
7.2.3.	Implementación de Integración Continua	113
7.2.4.	Tests en Spring Boot	114
7.3.	Sprint 3	117

7.3.1. Backend	120
7.3.2. Frontend	129
7.4. Sprint 4	139
7.4.1. Despliegue con Docker	139
8 Evaluación y experimentación	142
8.1. Métricas de evaluación	142
8.2. Locust	142
8.2.1. Resumen de pruebas realizadas	143
8.3. Ejecución y análisis de resultados	143
8.3.1. Comparación de tiempos en el servidor on premise	146
8.4. Conclusiones y limitaciones de la evaluación	147
9 Conclusiones y trabajo futuro	149
9.1. Limitaciones del sistema	150
9.2. Trabajo futuro	150
9.3. Conclusión final	151
Bibliografía	152
Anexo 1: Declaración de uso responsable de herramientas de IA	156
Anexo 2: Glosario	158

Índice de figuras

4.1. Ejemplo de una Historia de Usuario	35
4.2. Ejemplo del tablero para el sprint 1	36
4.3. Roadmap del proyecto visto desde GitHub Projects	38
5.1. Persona 1: Agricultor	46
5.2. Persona 2: Administradora de la Cooperativa	47
6.1. Diagrama de comunicación entre componentes	64
6.2. Diagrama E/R de la base de datos	66
6.3. Diagrama de clases	71
6.4. Logo de la aplicación Olivaris	72
6.5. Paleta de colores	73
7.1. Esquema de la base de datos del Sprint 1	95
7.2. Esquema de comunicación entre los componentes	107
7.3. Esquema de la base de datos del Sprint 2	108
7.4. Esquema de la base de datos del Sprint 3	118
7.5. Componente del mapa visto desde la aplicación	133
7.6. Pantalla de parcelas	134
7.7. Pantalla de actividades	135
7.8. Pantalla de entidades	136
7.9. Pantalla de usuarios del sistema	137
7.10. Pantalla del perfil de usuario	138
8.1. Inicio de Locust	144
8.2. Estadísticas para 100 usuarios en el login desde el CSIRC	144
8.3. Estadísticas para 100 usuarios en el login desde localhost	145
8.4. Estadísticas para 100 usuarios en el login desde on premise	145
8.5. Estadísticas para 1000 usuarios en el login desde on premise	146
8.6. Estadísticas para 1000 usuarios en el login desde on premise	146
8.7. Estadísticas para 1000 usuarios en la obtención de parcelas	147

Índice de tablas

3.1. Comparativa de Olivaris frente a otras soluciones de gestión agrícola.	13
5.1. Historia de usuario HU-1	49
5.2. Historia de usuario HU-2	49
5.3. Historia de usuario HU-3	50
5.4. Historia de usuario HU-4	50
5.5. Historia de usuario HU-5	50
5.6. Historia de usuario HU-6	50
5.7. Historia de usuario HU-7	50
5.8. Historia de usuario HU-8	51
5.9. Historia de usuario HU-9	51
5.10. Historia de usuario HU-10	51
5.11. Historia de usuario HU-11	51
5.12. Historia de usuario HU-12	51
5.13. Historia de usuario HU-13	52
5.14. Historia de usuario HU-14	52
5.15. Historia de usuario HU-15	52
5.16. Historia de usuario HU-16	52
5.17. Historia de usuario HU-17	52
5.18. Historia de usuario HU-18	53
8.1. Resumen de pruebas de carga con Locust	143

1. Introducción

En los últimos años, la gestión de las explotaciones agrícolas ha experimentado un proceso creciente de digitalización, impulsado por la necesidad de mejorar la organización interna de las explotaciones y por la evolución del marco normativo asociado a la Política Agraria Común, la trazabilidad de las actuaciones agrarias y el uso sostenible de insumos como fertilizantes y productos fitosanitarios. Como consecuencia, agricultores y entidades de apoyo deben gestionar una cantidad cada vez mayor de información sobre parcelas, recintos, cultivos, tratamientos, productos aplicados, fechas de actuación y documentación asociada.

Tradicionalmente, parte de esta información se ha registrado mediante cuadernos en papel, hojas de cálculo o aplicaciones no conectadas entre sí. Aunque estos mecanismos pueden resultar adecuados en explotaciones de pequeña escala, presentan limitaciones importantes cuando se requiere consultar información histórica, generar informes, compartir datos con una entidad colaboradora o preparar la información para su comunicación a sistemas administrativos. Estas limitaciones son especialmente relevantes en sectores como el olivar, donde la gestión de parcelas y actividades agrícolas tiene una gran importancia económica, territorial y medioambiental.

A nivel normativo, el Real Decreto 1054/2022 establece el Sistema de Información de Explotaciones Agrícolas, Ganaderas y de la Producción Agraria, conocido como SIEX. Este sistema se concibe como una infraestructura de información formada por bases de datos y registros administrativos interconectados, cuyo objetivo es facilitar la gestión, el seguimiento y el control de la información relativa a las explotaciones agrarias. Dentro de este marco se sitúan el Registro Autonómico de Explotaciones Agrícolas, o REA, que recoge la información general de las explotaciones y sus unidades de producción, y el Cuaderno Digital de Explotación Agrícola, o CUE, destinado al registro de determinadas actividades realizadas sobre dichas explotaciones.

Aunque la utilización del CUE digital ha sido objeto de ajustes normativos y no debe entenderse como una obligación universal e inmediata para todos los agricultores, la tendencia regulatoria apunta hacia una mayor interoperabilidad y digitalización de la información agrícola. Además, determinadas obligaciones vinculadas a productos fitosanitarios, fertilización o intervenciones relacionadas con la PAC hacen especialmente relevante disponer de herramientas capaces de registrar información de forma ordenada, trazable y compatible con los modelos definidos por la administración.

Este Trabajo de Fin de Grado aborda esta problemática mediante el diseño e implementación de Olivaris, una plataforma digital orientada a la gestión colaborativa de explotaciones del olivar. La aplicación centraliza el registro de parcelas y recintos, la gestión de usuarios y entidades habilitadas, la definición de permisos de acceso y el registro de actividades agrícolas asociadas a las

explotaciones. Aunque la integración efectiva con los servicios oficiales de SIEX/REA no forma parte del alcance final implementado, el diseño del sistema se ha realizado teniendo en cuenta la estructura conceptual de dicho marco, con el objetivo de facilitar una posible integración futura.

La solución desarrollada se basa en una arquitectura cliente-servidor, con un backend encargado de exponer una API REST, gestionar la persistencia de datos, aplicar mecanismos de autenticación y autorización, y mantener la lógica de negocio principal. Sobre esta base se construye una aplicación cliente orientada a facilitar el uso por parte del agricultor y de las entidades colaboradoras. De este modo, Olivaris no pretende sustituir a los sistemas oficiales, sino proporcionar una base tecnológica funcional, extensible y alineada con las necesidades actuales de digitalización de la gestión agrícola en el olivar.

1.1. Estructura de la memoria

La memoria va a estar dividida en los siguientes capítulos:

- **Capítulo 1: Introducción**

En este capítulo se presenta una introducción general al producto desarrollado en el Trabajo de Fin de Grado. Además, se describen los distintos capítulos en los que se estructura la memoria.

- **Capítulo 2: Objetivos y Alcance**

En este capítulo se detallan el objetivo general, los objetivos específicos y el alcance del proyecto.

- **Capítulo 3: Estado del arte**

En este capítulo se realizará un análisis del contexto técnico y social del problema, así como un análisis de productos software relacionados, herramientas existentes para su desarrollo e implementación así como librerías.

- **Capítulo 4: Metodología**

En este capítulo se explica cómo se ha trabajado durante el desarrollo del proyecto, abarcando tanto las metodologías de desarrollo como posibles intervenciones con usuarios externos (entrevistas, tests de usabilidad, etc).

- **Capítulo 5: Análisis y requisitos**

En este capítulo se va a realizar un análisis del problema general y se van a plasmar los requisitos. Para su realización se realizarán perfiles de usuario, historias de usuario y se expondrán los requisitos funcionales y no funcionales del sistema.

- **Capítulo 6: Diseño del sistema y arquitectura**

El contenido de este capítulo contendrá una descripción de cómo ha sido diseñada la solución sin entrar en detalles del código. Para su realización, se va a dividir en distintas secciones: arquitectura de alto nivel, modelo de datos, diseño de API y diseño de servicios.

- **Capítulo 7: Implementación**

Este capítulo explicará cómo se ha materializado el diseño del sistema en código. Estará dividido en las siguientes secciones: descripción de los módulos, tecnologías utilizadas, decisiones técnicas y problemas encontrados.

- **Capítulo 8: Evaluación y experimentación**

En este capítulo se va a comprobar cómo de bien funciona la implementación del sistema. Para ello, se podrán realizar pruebas de rendimiento, carga y disponibilidad del sistema.

- **Capítulo 9: Conclusiones y trabajos futuros**

En este punto se procederá a comentar qué se ha conseguido finalmente, qué limitaciones se han encontrado durante el desarrollo del proyecto y qué caminos quedan abiertos para ampliar o mejorar lo que ya se ha conseguido de cara al futuro.

- **Bibliografía:**

Referencias a los documentos de los que se ha obtenido información que ha sido utilizada en el presente documento.

- **Anexo 1: Declaración de usos responsable de herramientas de IA**

En este capítulo se realiza una declaración sobre cómo ha sido utilizada la Inteligencia Artificial para el desarrollo del proyecto.

- **Anexo 2: Glosario**

El glosario va a recoger todos aquellos términos técnicos y de relevancia que se hayan usado en el presente documento, con la finalidad de aclarar aquellos conceptos que puedan ser confusos o no conocidos.

2. Descripción del problema y alcance del proyecto

2.1. Contexto y motivación

La gestión agrícola se encuentra inmersa en un proceso progresivo de digitalización que afecta tanto a la organización interna de las explotaciones como a la forma en que estas recopilan, consultan y comunican su información. En este contexto, la necesidad de registrar de manera ordenada parcelas, recintos y actividades no responde únicamente a una cuestión de eficiencia operativa, sino también a la exigencia de disponer de datos trazables, actualizados y fácilmente reutilizables a lo largo de todo el ciclo de explotación.

En la práctica, este tipo de información sigue gestionándose con frecuencia mediante cuadernos en papel, hojas de cálculo o herramientas dispersas que no están conectadas entre sí. Aunque estos métodos pueden ser suficientes para explotaciones pequeñas o para un uso puntual, su falta de centralización dificulta la consulta histórica, incrementa la probabilidad de errores y complica la coordinación entre el agricultor y posibles entidades colaboradoras. Esta limitación resulta especialmente relevante en el caso del olivar, un cultivo de gran peso económico y territorial en el que la planificación de labores, el seguimiento de tratamientos y la gestión precisa de la superficie cultivada tienen un valor estratégico.

Además, este problema se enmarca en un contexto normativo cada vez más exigente, relacionado con el Sistema de Información de Explotaciones Agrícolas, Ganaderas y de la Producción Agraria (SIEEX), el Registro Autonómico de Explotaciones Agrícolas (REA) y el Cuaderno Digital de Explotación Agrícola (CUE). La evolución de este marco refuerza la necesidad de contar con soluciones capaces de estructurar la información agraria de forma coherente con los requisitos administrativos, facilitando tanto el cumplimiento de las obligaciones como la preparación de futuras integraciones con sistemas oficiales.

2.2. Descripción del problema

Como se ha comentado anteriormente, la evolución tecnológica junto con las nuevas normativas está dificultando el trabajo tanto de productores como de cooperativas. Por un lado, los agricultores necesitan llevar un control estricto de una gran cantidad de datos (como la delimitación de parcelas o los tratamientos aplicados). Al combinar esta exigencia con métodos tradicionales, como las hojas de Excel o las anotaciones en papel, es habitual que se pierda información o se cometan errores al registrar datos de manera incorrecta o mezclada. Como

consecuencia, cuando la entidad habilitada necesita recabar estos datos, debe invertir demasiado tiempo en descifrar información desordenada, incompleta o en formatos incompatibles, lo que ralentiza el proceso y lo vuelve mucho más complejo de lo que debería ser.

Además de lo anterior, los agricultores se enfrentan a un obstáculo puramente burocrático: la obligación de implantar el Cuaderno Digital de Explotación. Esto fuerza un cambio radical en su método de trabajo, ya que deben sustituir sus herramientas tradicionales por una plataforma que agilice el registro de sus actividades cotidianas y que, al mismo tiempo, sea capaz de conectarse directamente con los servicios ofrecidos por el Registro Autonómico de Explotaciones Agrícolas (REA) para poder generar ese Cuaderno Digital.

2.3. Solución propuesta

Para dar respuesta a los problemas analizados, este proyecto plantea el desarrollo de Olivaris, una plataforma digital diseñada específicamente para centralizar, simplificar y asegurar la gestión de la información en el sector agrícola, con un enfoque especial en el cultivo del olivar. El propósito de esta herramienta es transformar el modelo tradicional de trabajo basado en el papel y las hojas de cálculo dispersas, ofreciendo en su lugar un ecosistema único que facilite el día a día del campo y que esté actualizado de acuerdo a lo que exige la normativa.

De forma general, Olivaris propone una plataforma digital para:

- **Gestionar parcelas y recintos de forma unificada**
- **Registrar las actividades agrícolas diarias**
- **Gestionar usuarios y entidades habilitadas**
- **Definir roles y permisos**
- **Mantener un diseño compatible con el marco SIEX/REA/CUE**

2.4. Objetivos y alcance

Una vez definida la solución conceptual que plantea Olivaris para resolver los problemas del sector, es necesario concretar los objetivos técnicos y funcionales que guiarán el desarrollo de la plataforma, asegurando que cada necesidad detectada se traduzca en una característica real del sistema.

2.4.1. Objetivo general

El objetivo principal de este proyecto consiste en diseñar e implementar una aplicación móvil que funcione como un **Cuaderno de Explotación Comercial**

digital. Esta herramienta centralizará el registro y la gestión de actividades agrícolas para agricultores y entidades habilitadas, poniendo especial énfasis en el control de los tratamientos fitosanitarios debido a su obligatoriedad normativa.

2.4.2. Objetivos específicos

Para alcanzar el objetivo general, se establecen los siguientes objetivos específicos, los cuales delimitan las funcionalidades que serán implementadas en el sistema:

- **Gestión y visualización de parcelas:** desarrollar un módulo encargado de registrar, almacenar y visualizar las parcelas y recintos de las explotaciones con sus respectivas geometrías. Para ello, el alcance incluye la integración con la API pública de SIGPAC para automatizar la carga de datos y el despliegue de un mapa interactivo en la interfaz que facilite la localización visual de los terrenos.
- **Registro de actividades:** implementar un sistema dinámico para la creación, modificación y eliminación de tareas agrícolas (tanto planificadas como completadas). Esta funcionalidad contará con validaciones automáticas de datos (fechas coherentes, campos obligatorios) y un formulario especial para tratamientos fitosanitarios
- **Autenticación:** configurar un sistema de seguridad robusto basado en el estándar JSON Web Token (JWT) para gestionar el acceso a la plataforma. Esto permitirá proteger la privacidad de la información según el rol asignado.
- **Autorización y gestión de roles:** desarrollar la lógica de negocio que permita la coexistencia de tres perfiles bien definidos (administrador, entidad habilitada y agricultor). La funcionalidad clave de este módulo será la delegación segura, permitiendo que las entidades habilitadas gestionen las actividades de los agricultores (bajo previa autorización).
- **Módulo de cumplimiento y exportación normativa:** capacidad de poder generar y exportar el Cuaderno Digital de Explotación (CUE) con los campos y formatos oficiales exigidos por el marco normativo actual.

2.4.3. Alcance y limitaciones

El alcance de este proyecto define las fronteras funcionales de la plataforma Olivaris, delimitando las capacidades que serán desarrolladas y aquellas que, por criterios de viabilidad o restricciones externas, quedan fuera del proyecto.

Alcance del sistema desarrollado

El desarrollo se centrará en construir un producto mínimo viable (MVP) completamente operativo que cubra el núcleo central de la gestión agrícola. El

alcance técnico incluye:

- **Arquitectura móvil completa:** se implementará una aplicación móvil funcional (frontend y backend) que centraliza el flujo de trabajo comercial del cuaderno digital.
- **Estructura de datos normalizada:** se diseñará y desplegará una base de datos relacional modelada bajo los estándares de información del CUE y el SIEX, asegurando la integridad de los registros agrícolas y fitosanitarios.
- **Capa de integración visual:** la aplicación se conectará con los servicios oficiales de SIGPAC y mostrará en la interfaz de usuario las geometrías través de mapas.
- **Capa de seguridad:** se utilizarán mecanismos que permitan controlar el acceso a los datos entre agricultores y gestión de las acciones delegadas a las entidades habilitadas.

Limitaciones

Por motivos de viabilidad temporal y de recursos, quedan excluidos del desarrollo los siguientes bloques:

- **Sincronización directa con el SIEX/REA:** no se incluirá la conexión en tiempo real ni el envío directo de datos a las APIs de los servicios oficiales de la administración pública por falta de recursos que permitan la conexión con dichos servicios.
- **Módulos de gestión empresarial avanzada:** no se contemplarán herramientas de contabilidad analítica, control de costes financieros, facturación ni gestión de nóminas de trabajadores.
- **Persistencia en modo desconectado (*offline*):** el sistema requiere una conexión activa a internet para operar, quedando fuera del alcance la sincronización compleja de datos en zonas sin cobertura de red.
- **Ecosistema multiplataforma:** el desarrollo se limita a un único cliente accesible a través de un dispositivo móvil, excluyendo la creación de aplicaciones web.

2.4.4. Justificación de las decisiones

Las decisiones de diseño y la acotación del alcance responden a una estrategia de desarrollo orientada a la viabilidad y eficiencia del proyecto. Por un lado, las limitaciones temporales propias de un Trabajo de Fin de Grado obligan a priorizar las funcionalidades *core* de la aplicación (el registro de actividades y la lógica de permisos) para garantizar la entrega de un prototipo sólido, estable y testeado.

Por otro lado, la exclusión de la integración directa con la API oficial del SIEX/REA responde a una barrera burocrática y falta de recursos para este

proyecto. El acceso a los entornos de pruebas y producción de la administración requiere obligatoriamente la posesión de un certificado digital de entidad habilitada o la constitución de una empresa del sector tecnológico. Al no disponer de este marco legal e institucional, se ha optado por la solución más viable: modelar localmente la base de datos de Olivaris de la forma más parecida a los requisitos de la normativa vigente. De este modo, el sistema queda preparado para que la exportación de sus datos sea compatible con las futuras integraciones oficiales de la administración.

3. Estado del arte

En esta sección se va a proceder a realizar un estudio del estado del arte en el ámbito de la planificación de recursos empresariales (Enterprise Resource Planning o ERP) para el sector de la oliva. Se analizarán cuáles son sus funcionalidades y limitaciones, así como las tecnologías utilizadas para su desarrollo.

Posteriormente, se expondrán qué tipos de arquitecturas y tecnologías pueden ser utilizadas para desarrollar el presente proyecto.

3.1. Contexto del olivar y necesidad de digitalización

El sector del olivar en España tiene una importancia estratégica fundamental, siendo el líder mundial con más del 40 % de la producción total [1]. Esta posición no solo implica una gran relevancia económica, sino también una responsabilidad muy alta en cuanto a la calidad y la trazabilidad del producto. Sin embargo, la gestión diaria en el campo todavía presenta un contraste importante: mientras que las almazaras se han modernizado, el registro de lo que ocurre en las parcelas sigue dependiendo en gran medida de métodos tradicionales.

Actualmente, gran parte de la información sobre tratamientos o recolección se anota en cuadernos de papel o archivos de Excel. Aunque esto puede funcionar en casos muy pequeños, genera problemas graves cuando la explotación crece o cuando hay que coordinarse con otros. El uso de estos métodos manuales provoca con frecuencia pérdida de información y errores al registrar datos de forma desordenada.

Además, cuando un agricultor necesita que su cooperativa o entidad habilitada gestione sus datos, estos agentes invierten mucho tiempo interpretando los datos, anotaciones incompletas o formatos que no son compatibles entre sí. El software genérico tampoco es una solución ideal, ya que no suele contemplar aspectos específicos de nuestro sector como la gestión por campañas agrícolas.

A esta necesidad de eficiencia operativa se le suma ahora una exigencia legal. Con la entrada del Real Decreto 1054/2022, se ha creado el Sistema de Información de Explotaciones Agrarias (SIEX) y el Cuaderno Digital de Explotación (CUE). Esta normativa obliga a los agricultores a dar un salto digital, ya que el uso del cuaderno de campo en papel dejará de ser válido, siendo obligatorio el formato digital para tratamientos fitosanitarios a partir de enero de 2027.

Esta situación genera una barrera burocrática para muchos productores que no cuentan con herramientas sencillas para adaptarse. Por tanto, la digitalización ya no es solo una opción para mejorar la rentabilidad, sino una obligación legal.

3.2. Marco normativo: SIEX, REA y CUE

El desarrollo de Olivaris no solo responde a una necesidad técnica, sino que se enmarca en una transformación legal del sector agrario en España, impulsada principalmente por el **Real Decreto 1054/2022** [2]. Este decreto busca digitalizar la relación entre los agricultores y la administración a través de tres pilares fundamentales [3]:

- **SIEX (Sistema de Información de Explotaciones Agrarias):** se define como una infraestructura de bases de datos y registros administrativos interconectados. Su objetivo es centralizar toda la información de las explotaciones para facilitar su gestión y seguimiento por parte de las autoridades.
- **REA (Registro Autonómico de Explotaciones Agrícolas):** es el registro donde se recoge la información general de cada explotación. Funciona como la base de datos oficial de la que se extraen los datos de parcelas y cultivos que luego deben ser gestionados.
- **CUE (Cuaderno Digital de Explotación Agrícola):** es la herramienta electrónica destinada a que los agricultores registren actividades específicas como los tratamientos fitosanitarios, riegos o fertilización. Su uso será obligatorio para el registro de fitosanitarios a partir de enero de 2027.

Dada la complejidad de estos registros digitales, la normativa contempla la figura de las entidades habilitadas (como cooperativas o empresas de servicios). Estas entidades pueden actuar en nombre de los agricultores mediante una delegación de permisos. Esto permite que personal técnico de la cooperativa gestione directamente el REA y el CUE del titular de la explotación, garantizando que la información se registre correctamente y a tiempo.

Este escenario normativo es lo que justifica la creación de una plataforma como Olivaris. Olivaris nace para simplificar este proceso burocrático, ofreciendo un entorno intuitivo donde agricultores y cooperativas puedan colaborar de forma sencilla.

Es importante aclarar que, aunque Olivaris no está integrada de forma oficial con los servicios del SIEX o la REA en esta fase del proyecto, su base de datos y lógica de negocio se han diseñado siguiendo la estructura conceptual y los requisitos de información de este marco legal. De este modo, el sistema queda preparado para facilitar una posible integración futura con las APIs oficiales de la administración.

3.3. Soluciones software existentes para la gestión agrícola

Actualmente, podemos encontrar diversos softwares ERP que están directamente relacionados con la producción de oliva, otros que no se relacionan

directamente pero se ubican dentro del sector agrícola y otros que son genéricos pero se pueden adaptar al sector. Vamos a proceder a comentar algunos de ellos:

■ **Agroptima - Relación directa (<https://www.agroptima.com>)**

Agroptima es un Software de gestión agrícola cuyo objetivo es conseguir la simpleza a la hora de gestionar los cultivos y explotaciones por parte de agricultores y empresas agrícolas (según declara la propia empresa).

En términos generales destaca por proporcionar un cuaderno de campo al agricultor, permitirle llevar un control de los gastos y costes agrícolas (como los trabajadores) y saber en todo momento en qué estado se encuentra su finca.

Entrando en más detalle, entre sus funcionalidades se encuentran:

- Localización de campos y cultivos, consulta de superficies y códigos SIGPAC.
- Anotación de toda la información que el agricultor considere relevante desde el móvil y sin cobertura. Entre esta información se destaca: trabajadores, maquinaria, horas trabajadas y productos.
- Toda actividad realizada y anotada quedará en el sistema y servirá para mejorar la toma de decisiones y aumentar la rentabilidad de la gestión de la finca.
- Las fincas o cultivos se podrán importar en la aplicación con un archivo Excel, permitiendo visualizar las parcelas coloreadas según el tipo de cultivo y su superficie total.
- Gestión de roles, permitiendo que el usuario pueda gestionar aquellas actividades relacionadas con el.

Agroptima también destaca por ofrecer a sus clientes la posibilidad de generar un Cuaderno de Campo.

■ **AgriCloud - Relación directa (<https://www.agricloud.es>)**

AgriCloud es un Software de gestión agrícola diseñado para llevar un control de las fincas, trabajos, personal y facturación, pensada para agricultores, cooperativas, empresas de servicios agrícolas y trabajadores por campaña. AgriCloud destaca por ofrecer funcionalidades como las siguientes:

- Un cuaderno de campo digital según la normativa (muy importante como hemos mencionado anteriormente).
- Control horario y partes de trabajo desde el móvil.
- Facturación electrónica conectada con Hacienda.
- Costes ingresos e informes automáticos en un solo lugar.
- Acceso desde cualquier dispositivo, tanto para móvil como tablet u ordenador.

- Generación de un cuaderno de campo digital, facilitando la gestión y el seguimiento de las actividades agrícolas de forma eficiente y precisa.
- **Agroex (<https://agroex.es>)**
Agroex es un Software de gestión agrícola industrial y ganadería, enfocado en empresas grandes. Cuentan con un software adaptado a las necesidades específicas del sector, permitiendo optimizar recursos y ahorrar costes (tanto en campaña agrícola específica como a lo largo del año), gracias a la administración y gestión tanto de los cultivos como del personal. Además, cuenta con integraciones como AEMET o MAGRAMA, permitiendo el acceso a datos en tiempo real que permite ayudar en la toma de decisiones.

Agroex ofrece soluciones para distintos sectores, desde un enfoque en la explotación agrícola y consultoría agraria a otros como bodegas, viñedos y viveros.
 - **Campogest (<https://www.campogest.com>)**
Campogest es un Software diseñado por la empresa Hispatec y destinado para los Técnicos Agrícolas, permitiéndoles realizar una gestión de sus actividades a través del móvil o tablet.

Esta aplicación destaca por ofrecer las siguientes funcionalidades:
 - Gestión de fincas, cultivos, previsión meteorológica, tratamientos fitosanitarios y recomendaciones personalizadas a través del móvil.
 - Generación de un cuaderno de campo teniendo en cuenta las recomendaciones en materia de Normativa Legal y Protocolos de Calidad.
 - Posibilidad de generar la finca o parcela a través de la cartografía SIGPAC, pudiendo asignar un propietario.

- **Visual NACERT (<https://visualnacert.com>)**
Visual Nacert es una empresa que diseña soluciones digitales de gestión para empresas del sector agroalimentario, agricultores e industria. Cuentan con una aplicación agrícola, a la que se puede acceder a través de móvil o tablet, con o sin cobertura.

Desde esta aplicación, permiten al agricultor/cliente realizar las funciones:

- Gestión de costes insumos, mano de obra y maquinaria, generando gráficos a partir de estos datos e informes por parcela.
- Gestión del Cuaderno de Campo Digital SIEX (de acuerdo a la normativa).
- Generación de informes de sostenibilidad.

3.4. Comparativa de soluciones y posicionamiento de Olivaris

Una vez analizados los sistemas anteriores, es necesario contrastar sus capacidades con la propuesta técnica de Olivaris. Mientras que la mayoría de herramientas comerciales buscan cubrir todo el espectro agrícola de forma general, Olivaris busca resolver el problema específico de la comunicación y gestión compartida entre el agricultor y las entidades habilitadas de acuerdo al marco normativo.

Para comparar la propuesta de Olivaris con respecto a otras plataformas se presenta la siguiente tabla:

Criterios de comparación	Agroptima	AgriCloud	Agroex	Campogest	Visual Nacert	Olivaris
Gestión de parcelas	Sí	Sí	Sí	Sí	Sí	Sí
Uso de SIGPAC / Geoespacial	Sí	Sí	Sí	Sí	Sí	Sí
Registro actividades agrícolas	Sí	Sí	Sí	Sí	Sí	Sí
Gestión fitosanitarios	Sí	Sí	Sí	Sí	Sí	Sí
Acceso móvil	App	Web/App	Parcial	App	App	App
Acceso web	Sí	Sí	Sí	No	Sí	No
Funcionamiento offline	Sí	No	No	Sí	Sí	No
Generación de informes	Sí	Sí	Sí	Sí	Sí	Inicial
Gestión colaborativa	Media	Media	Industrial	Técnica	Media	Alta
Roles y permisos	Básicos	Sí	Complejos	Sí	Sí	Avanzados
Orientación específica olivar	No	No	No	No	No	Sí
Marco SIEX/REA/CUE	Parcial	Sí	Parcial	Sí	Sí	Diseño

Tabla 3.1: Comparativa de Olivaris frente a otras soluciones de gestión agrícola.

3.4.1. Análisis del puesto que ocupa Olivaris

Tras realizar la comparativa, se observa que Olivaris ocupa un nicho estratégico que las soluciones actuales no cubren de forma equilibrada. El sistema a desarrollar destaca por las siguientes características:

- **Especialización funcional:** a diferencia de Agroptima o AgriCloud, que son herramientas multicultivo, Olivaris está diseñada pensando en los ciclos y necesidades del olivar, simplificando la interfaz y priorizando la usabilidad.
- **Modelo de delegación de permisos:** analizando otras plataformas, la gran diferencia de Olivaris radica en su sistema de roles y permisos. Mientras otras aplicaciones se centran en el uso individual, Olivaris modela la relación entre agricultor y entidad habilitada. Esto permite que una cooperativa gestione las actividades fitosanitarias en nombre de sus socios con una alta trazabilidad y siguiendo la normativa SIEX.
- **Simplicidad:** frente a ERPs complejos como Agroex, orientados a la gran empresa y con altos costes, Olivaris ofrece un Producto Mínimo Viable (MVP) enfocado en la agilidad móvil, rápida adaptación y sencillez.

- **Preparación para la integración administrativa:** aunque herramientas como Visual Nacert ya están integradas oficialmente, Olivaris se presenta como una alternativa de código abierto y extensible cuyo modelo de datos es altamente compatible con la estructura de la administración, facilitando que pequeños desarrolladores o cooperativas locales puedan tener su propia tecnología sin depender de grandes licencias comerciales.

En conclusión, Olivaris pretende ser una herramienta de código abierto, ágil y colaborativa para el día a día del agricultor y su asesor técnico, con una base para una futura integración con los servicios del REA y CUE digital.

3.5. Tecnologías para el desarrollo de plataformas web y móviles

Históricamente, el desarrollo de servicios web se fundamentó en la Arquitectura Orientada a Servicios (SOA). Como indicaban Ortiz y Riestra (2009) [4], el reto principal era lograr la interoperabilidad entre sistemas heterogéneos mediante protocolos robustos como SOAP y el uso de XML para el intercambio de datos. En aquel momento, el foco estaba en permitir que terminales con capacidades limitadas pudieran acceder a lógica de negocio compleja residente en servidores.

No obstante, en la última década este paradigma ha evolucionado drásticamente. La rigidez de los protocolos basados en XML ha dado paso a la arquitectura REST (Representational State Transfer) y al formato JSON, que se han consolidado como el estándar de facto por su ligereza y facilidad de consumo, especialmente en dispositivos móviles.

En la actualidad, han emergido nuevos paradigmas que complementan y, en muchos casos, sustituyen a las arquitecturas monolíticas tradicionales. Destacan especialmente los microservicios y las arquitecturas orientadas a eventos (explicados a continuación), soluciones que han surgido como respuesta a la necesidad de construir sistemas con una mayor escalabilidad, flexibilidad y resiliencia ante grandes volúmenes de datos.

A continuación, se desglosarán los patrones de arquitectura software modernos, formatos de intercambio de datos y servicios web, ecosistemas de desarrollo web, desarrollo de aplicaciones móviles y por último, infraestructura y seguridad. Este análisis permitirá examinar de manera detallada cómo han evolucionado los servicios web hasta alcanzar los estándares de eficiencia y robustez contemporáneos.

3.5.1. Arquitecturas Software

La arquitectura de software ha evolucionado progresivamente hasta convertirse en un ecosistema adaptable. El crecimiento exponencial en el desarrollo de servicios

web y la necesidad de ofrecer soporte a aplicaciones móviles con alta disponibilidad han impulsado la aparición de nuevos enfoques y modelos arquitectónicos. En este contexto, la transición de sistemas centralizados (arquitecturas monolíticas) hacia sistemas distribuidos ha surgido de manera natural, motivada por la necesidad de desplegar funcionalidades de forma independiente y escalable.

De acuerdo con Richards y Ford [5], las arquitecturas de software se pueden clasificar principalmente en dos categorías: monolíticas y distribuidas.

A continuación, se describen los estilos arquitectónicos más relevantes siguiendo la clasificación propuesta por los autores mencionados.

Arquitectura Monolítica

Es un tipo de arquitectura de software caracterizada principalmente por contener el código en una sola unidad de despliegue. Dentro de este tipo de arquitectura se pueden encontrar los siguientes tipos:

■ **Arquitectura en capas**

Se utiliza como punto de partida ya que es el patrón más común y tradicional en el desarrollo de software. Este tipo de arquitectura se organiza en capas horizontales, donde cada una tiene un rol específico dentro de la aplicación (por ejemplo, presentación y lógica de negocio). Aunque no hay un número fijo de capas, el estándar suele ser de cuatro:

- **Capa de Presentación:** representa la interfaz de usuario con la que se interacciona y es la encargada de manejar las peticiones que son realizadas al servidor.
- **Capa de Negocio:** contiene las reglas y la lógica del dominio de la aplicación.
- **Capa de Persistencia:** es la encargada de manejar los datos del sistema (interacción con la base de datos).
- **Capa de Base de Datos:** corresponde al motor de almacenamiento de datos físico, es decir, la propia base de datos.

Una de las características más relevantes de este tipo de arquitectura es que las capas están aisladas. Esto significa que un cambio en una capa no debería afectar a las demás, siempre que la interfaz (el contrato) entre ellas se mantenga. En este punto, se puede diferenciar entre capas de tipo “cerrada”, que sería aquella que solo se puede comunicar con una inmediatamente inferior (tipo de capa por defecto), o de tipo “abierta”, la cual permite que capas superiores puedan saltársela para acceder directamente a capas inferiores.

Generalmente, este estilo se implementa como una única unidad de despliegue (monolito), lo que simplifica la gestión inicial pero dificulta la escalabilidad individual de componentes.

Valorando las características arquitectónicas se destaca:

- Costo y simplicidad: es una arquitectura muy fácil de entender y barata de implementar inicialmente.
- Facilidad de desarrollo: al ser un estándar tan conocido, los equipos pueden empezar a trabajar rápidamente.
- Escalabilidad y Elasticidad: esta características es, quizás, su mayor debilidad y esto ocurre porque al ser monolítica, no va a permitir un escalado de las partes específicas del sistema de forma independiente.
- Tolerancia a fallos: esta es otra debilidad que presenta, ya que si se produce un fallo en una de las capas (por ejemplo, base de datos), el fallo va a afectar a toda la aplicación.

Como conclusión, se puede destacar que esta arquitectura es ideal para aplicaciones pequeñas y sencillas donde el presupuesto es limitado y la complejidad no justifica sistemas distribuidos más costosos.

■ **Arquitectura Pipeline**

Este estilo de arquitectura se basa en el principio de computación de una secuencia de transformaciones. Su estructura es lineal y se compone de dos elementos principales: *pipes* (tuberías) y *filters* (filtros). Por un lado, los *pipes* representan los canales de comunicación entre las etapas de procesamiento, suelen ser unidireccionales y su función es puramente el transporte de datos de un filtro a otro; mientras que por el otro lado, los *filters* son componentes independientes que realizan una tarea específica de procesamiento o transformación de los datos que reciben del *pipe*. Además, éstos son autónomos y no deben conocer la existencia de otros filtros en la cadena.

En cuanto a sus características, esta arquitectura destaca por un alto desacoplamiento, ya que al ser los filtros independientes, es fácil intercambiar, añadir o modificar etapas en el pipeline sin afectar al resto del sistema. Además, este estilo es “*stateless*” entre filtros, es decir, cada filtro procesa lo que recibe y lo pasa al siguiente sin mantener un estado global complejo.

Valorando sus características arquitectónicas más relevantes, se clasifican en:

- Costo y simplicidad: es un estilo relativamente económico y fácil de implementar para los casos de uso adecuados.
- Facilidad de desarrollo: la separación clara de responsabilidades en filtros independientes facilita mucho el trabajo de codificación.
- Escalabilidad: al ser generalmente una arquitectura monolítica, es difícil de escalar de forma elástica.
- Tolerancia a fallos: si un filtro o una tubería falla, todo el proceso suele detenerse, lo que lo hace poco resiliente.

Como conclusión, esta arquitectura es ideal para aplicaciones que requieren un procesamiento de datos secuencial y discreto, como herramientas de

procesamiento de texto, compiladores o sistemas de extracción, transformación y carga (ETL) sencillos.

- **Arquitectura Microkernel**

También conocida como *plug-ins*, es un estilo de arquitectura diseñado para productos software que se pueden instalar (como navegadores o IDEs). Su estructura se divide en dos componentes principales: *Core System* (sistema central), el cual contiene toda la lógica mínima necesaria para que la aplicación funcione, encargándose de la ejecución básica y la gestión de los *plug-ins*; y *Plug-in Components*, que son módulos independientes entre sí con lógica adicional, características especiales o adaptadores personalizados para extender el sistema central.

En cuanto a su funcionamiento, el sistema central necesita saber qué *plug-ins* están disponibles. Para ello, utiliza un registro o catálogo que contiene información sobre el *plug-in*, cómo acceder a él y qué protocolos utiliza. Por parte del *plug-in*, para que este funcione debe cumplir con un contrato (interfaz) estándar definido por el sistema central.

Como conclusión, es una arquitectura ideal para aplicaciones que necesitan evolucionar constantemente añadiendo nuevas funciones o lógica según el cliente o el caso de uso, sin tener que reescribir el núcleo del sistema.

Arquitectura Distribuida

En esta arquitectura, a diferencia de la monolítica, los componentes van a estar separados y se conectarán a través de protocolos de acceso remoto. Entre sus tipos, se pueden destacar los siguientes:

- **Arquitectura basada en servicios**

Este estilo se considera un híbrido que combina aspectos de la arquitectura monolítica y los microservicios, siendo una opción extremadamente popular para aplicaciones empresariales.

A diferencia de los microservicios, este estilo se basa en la creación de servicios de dominio de granularidad gruesa, refiriéndose a servicios más grandes que los microservicios y menos detallados, que encapsulan una funcionalidad de negocio más amplia y compleja. En una aplicación típica, suele haber entre 4 y 12 servicios, siendo cada servicio una unidad de despliegue independiente que contiene toda la lógica necesaria para un dominio de negocio específico. Sin embargo, la base de datos suele ser centralizada y compartida con todos (aunque si es necesario se puede “partir”), al igual que la interfaz de usuario, que hay una y es la que se comunica con los servicios.

Este estilo es calificado en el libro como uno de los más equilibrados, destacando:

- Agilidad y despleabilidad: al tener servicios independientes, es fácil realizar cambios y desplegarlos rápidamente.

- Facilidad de prueba: los servicios de dominio pueden probarse de forma aislada, lo que mejora la calidad del software.
- Costo y simplicidad: es mucho menos complejo y costoso de implementar que una arquitectura de microservicios pura.
- Escalabilidad y elasticidad: aunque es mejor que una arquitectura de capas, la base de datos compartida suele actuar como un cuello de botella que limita el escalado infinito.
- Tolerancia a fallos: el fallo de un servicio no detiene todo el sistema, pero un fallo en la base de datos compartida puede ser crítico.

Como conclusión, este estilo es usado para aplicaciones que necesitan un buen equilibrio entre agilidad, costo y escalabilidad, especialmente cuando el dominio del negocio tiene un acoplamiento semántico que dificultaría mucho el uso de microservicios.

■ **Arquitectura dirigida por eventos**

A diferencia del modelo tradicional basado en peticiones (*request-based*), donde un componente solicita datos de forma determinista, este modelo reacciona a situaciones que ocurren en el sistema (eventos). Se compone de componentes de procesamiento de eventos desacoplados que operan de manera asíncrona.

Se van a distinguir dos formas fundamentales de implementar este estilo:

- Topología de Broker (intermediario):
Esta topología está caracterizada por no tener un orquestador central, siendo el flujo de mensajes distribuido gracias a un *broker* ligero. Funciona mediante una cadena de eventos: un componente procesa un evento iniciador, publica lo que hizo como un ".evento de procesamiento", y otros componentes interesados reaccionan a él.
- Topología de Mediador:
Esta topología se utiliza cuando se requiere un control sobre el flujo de trabajo. Para ello, se cuenta con un "Mediador de Eventos" que será el encargado, conociendo toda la lógica de negocio, de coordinar los pasos necesarios para procesar un evento inicial.

Este estilo está caracterizado por ser asíncrono, permitiendo de esta manera que el sistema responda al usuario casi instantáneamente mientras el procesamiento pesado ocurre en segundo plano. Esto evita bloqueos pero requiere del uso de técnicas (como colas de errores) que permitan gestionar fallos sin detener el sistema y que garanticen que los mensajes no se pierdan.

Como conclusión, su uso es recomendado en aplicaciones que necesitan reaccionar en tiempo real a grandes flujos de información, como en sistemas de procesamiento de datos de telemetría.

■ **Arquitectura de microservicios**

Richards y Ford la definen como la "estrella brillante" del ecosistema actual, diseñada específicamente para abordar los problemas de escalabilidad y

agilidad de los sistemas monolíticos.

Esta arquitectura se basa en el concepto de desacoplamiento extremo, tanto a nivel técnico como de dominio. A diferencia de otros estilos, su objetivo principal no es la reutilización (que a menudo se evita para no crear acoplamiento), sino la separación de intereses para permitir cambios rápidos. Aquí aparece una entidad clave que es el “microservicio”, siendo una unidad física separada que se puede desplegar, escalar y actualizar sin afectar a los demás, y cuya funcionalidad será única.

Para garantizar el desacoplamiento de este estilo, cada microservicio posee sus propios datos. De esta manera, ningún otro servicio puede acceder directamente a la base de datos de otro; la comunicación debe ser a través de interfaces (APIs).

Entre sus componentes destacan:

- *API Gateway*: actúa como punto de entrada para los clientes, manejando tareas como métricas, seguridad y enrutamiento.
- *Sidecars*: componentes adjuntos a los servicios que manejan tareas transversales (comunicación, monitoreo) sin afectar a la lógica de negocio.
- *Service Mesh*: infraestructura para gestionar la comunicación entre todos los microservicios de forma consistente.

Aunque parezcan todos puntos a favor en esta arquitectura, es uno de los estilos más difíciles de implementar debido a la alta necesidad de gestión de los datos entre múltiples servicios y la complejidad de verificar que todo el ecosistema funcione correctamente. Además, requiere de conocimientos en áreas como Devops, automatización y monitoreo.

Según el libro, su calificación es la siguiente:

- Escalabilidad y elasticidad: es el mejor estilo para sistemas que necesitan crecer de forma masiva y dinámica.
- Agilidad y despleabilidad: Permite ciclos de entrega continuos.
- Tolerancia a fallos: el aislamiento asegura que el fallo de un servicio no derribe todo el sistema (siempre que se implementen patrones de resiliencia).
- Costo y simplicidad: es el estilo más caro y complejo de desarrollar, mantener y operar. No se recomienda para aplicaciones sencillas.

Como solución, la arquitectura de microservicios es muy útil para resolver problemas de gran escala, pero conlleva un “impuesto de complejidad” que solo debe pagarse si las necesidades de negocio lo justifican.

3.5.2. Comunicación y Servicios Web

La comunicación entre componentes es el elemento que define la eficiencia de una arquitectura distribuida. Mientras que en el pasado la interoperabilidad se buscaba mediante protocolos estrictos basados en estándares corporativos, el auge de la web móvil ha impuesto la necesidad de protocolos de baja latencia y formatos de datos ligeros. Según Richards y Ford [5], el éxito de una arquitectura distribuida depende de la gestión de los contratos de comunicación y el ancho de banda disponible.

Esta interacción es posible gracias a las Interfaces de Programación de Aplicaciones (API). Entre sus tipos principales destacan:

- **REST (*Representational State Transfer*)**

Consolidado como el estándar de facto para servicios web. REST utiliza los métodos del protocolo HTTP para definir acciones sobre los recursos del sistema a través de *endpoints* (URLs).

Los principales verbos HTTP y su propósito son: [6]

- *GET*: solicita una representación de un recurso específico. Las peticiones que usan el método GET sólo deben recuperar datos.
- *PUT*: reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición.
- *POST*: se utiliza para enviar una entidad a un recurso en específico, causando a menudo un cambio en el estado o efectos secundarios en el servidor.
- *DELETE*: borra un recurso en específico.
- *HEAD*: pide una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta.
- *PATCH*: aplica modificaciones parciales a un recurso.
- *OPTIONS*: utilizado para describir las opciones de comunicación para el recurso de destino.
- *TRACE*: realiza una prueba de bucle de retorno de mensaje a lo largo de la ruta al recurso de destino.
- *CONNECT*: establece un túnel hacia el servidor identificado por el recurso.

Para confirmar el resultado de estas acciones, el servidor devuelve códigos de estado HTTP, tales como: 200 OK (éxito), 400 Bad Request (error del cliente) o 500 Internal Server Error (error del servidor).

- **GraphQL [7]**

GraphQL utiliza una sintaxis intuitiva que permite a los clientes enviar una única consulta GraphQL a una API y recibir exactamente los datos necesarios (en lugar de acceder a endpoints complejos con muchos parámetros). Esta

obtención de datos más eficiente puede mejorar el rendimiento del sistema y la facilidad de uso para los desarrolladores.

Es especialmente útil para crear APIs en entornos complejos con requisitos de interfaz que cambian rápidamente. Ni las API de REST ni las de GraphQL son inherentemente superiores; son herramientas diferentes que se adaptan a diferentes tareas.

- **gRPC** [8]

Es un marco de comunicación de alto rendimiento desarrollado por Google. Utiliza como sistema de comunicación el protocolo RPC (*Remote Procedure Call*) en lugar de texto, lo que lo hace significativamente más rápido que REST para la comunicación interna entre microservicios.

- **SOAP** [9]

SOAP (anteriormente conocido como Simple Object Access Protocol) es un protocolo ligero para el intercambio de información en entornos descentralizados y distribuidos. Es un protocolo basado en XML que define tres partes en todos los mensajes: *sobre*, que define una infraestructura para describir qué hay en un mensaje y cómo procesarlo; *reglas de codificación*, que expresa instancias de tipos de datos definidos por la aplicación, y *estilos de comunicación*, que pueden seguir el formato de llamada a procedimiento remoto (RPC) o el formato orientado a mensajes (Documento).

En cuanto al formato de intercambio de datos, Se ha producido una transición del uso de XML (extenso y costoso de procesar) al dominio absoluto de JSON (JavaScript Object Notation). JSON es un formato de texto ligero, fácil de leer para humanos y sumamente rápido de procesar por los motores modernos.

Finalmente, la comunicación entre sistemas va más allá del modelo tradicional de petición-respuesta de HTTP, existiendo otros protocolos para servicios que requieren inmediatez como **WebSockets**, proporcionando un canal de comunicación bidireccional constante entre el cliente y el servidor, y **Server-Sent Events**, permitiendo que el servidor envíe actualizaciones de forma automática al cliente sin que este tenga que solicitarlas repetidamente.

3.5.3. Ecosistema del Desarrollo Web

En este apartado se describen las principales tecnologías consideradas para el desarrollo del proyecto Olivaris. Estas se clasifican en dos grandes áreas: el Frontend (lado del cliente) y el Backend (lado del servidor).

Tecnologías para el frontend

El frontend comprende todo aquello con lo que el usuario interactúa directamente. En la última década, este campo ha evolucionado desde documentos estáticos hacia [Single Page Application](#) y [Progressive Web Apps](#) de gran complejidad. El objetivo actual es gestionar estados complejos, asegurar la accesibilidad y

optimizar el rendimiento bajo el paradigma de *Client-Side Rendering* (CSR) o enfoques híbridos.

Aunque hoy en día se utilizan muchas herramientas, la base de cualquier interfaz web descansa sobre tres pilares:

- **HTML5 (*HyperText Markup Language*)** [10]
Es el componente más básico de la Web. Define la estructura y el significado del contenido mediante etiquetas (marcas), permitiendo que el navegador interprete textos, imágenes y multimedia.
- **CSS3 (*Cascading Style Sheets*)** [11]
Lenguaje de estilos encargado de la presentación visual. Define cómo se renderizan los elementos en pantalla (diseño, colores, tipografías y adaptabilidad).
- **JavaScript** [12]
Si bien es más conocido como un lenguaje de *scripting* para páginas web, y es usado en muchos entornos fuera del navegador, JavaScript es un lenguaje de programación basada en prototipos, multiparadigma, de un solo hilo, dinámico, con soporte para programación orientada a objetos, imperativa y declarativa. Gracias a JavaScript, se ha conseguido que la web se convierta en una pieza interactiva y dinámica para el usuario.

Mención especial merece **TypeScript**, como un superset de JavaScript que añade tipado estático. Su uso mejora la detección de errores en tiempo de desarrollo y facilita el mantenimiento de bases de código extensas.

El desarrollo ha evolucionado del uso de "Vanilla JS" (JS puro) hacia marcos de trabajo (*frameworks*) que aumentan la eficiencia:

- **React** [13]
Librería basada en componentes reutilizables. Destaca por su flexibilidad y su capacidad para manejar interfaces de usuario altamente dinámicas mediante un DOM virtual.
- **Angular** [14]
Angular es un framework integral que utiliza TypeScript de forma nativa. Su arquitectura modular y su sistema de inyección de dependencias lo hacen ideal para aplicaciones empresariales escalables.

Para el ecosistema móvil, se han evaluado diversas estrategias que permiten alcanzar a usuarios de Android e iOS:

- **Flutter** [15]
Flutter es un framework de código abierto de Google para crear aplicaciones multiplataforma compiladas de forma nativa a partir de una única base de código, ofreciendo un alto rendimiento y una interfaz consistente.
- **React Native** [16]
React Native es una librería de JavaScript que permite construir aplicaciones nativas para Android e iOS. Proporciona componentes nativos que se asignan directamente a los bloques de construcción de la interfaz.

- **Kotlin** [17]

Kotlin es un lenguaje de programación de tipo estático utilizado para desarrollar aplicaciones móviles en Android, destacando por su seguridad y concisión.

- **Ionic** [18]

Ionic es un SDK (kit de desarrollo de software) de código abierto que permite crear aplicaciones multiplataforma utilizando una única base de código. La aplicación se construye una sola vez con tecnologías web (como HTML, CSS y JavaScript), y se despliega en cualquier dispositivo.

Tecnologías para el backend

El backend representa la capa de lógica de procesamiento y acceso a datos que reside en el servidor. El enfoque actual prioriza la escalabilidad, la capacidad de respuesta ante grandes volúmenes de datos y la exposición de servicios mediante APIs (*Application Programming Interfaces*) robustas, generalmente bajo el estándar REST o GraphQL.

Dentro de los lenguajes más utilizados para el desarrollo del backend, se encuentran:

- **Java**

Java se mantiene como un pilar en sistemas empresariales críticos debido a su robustez, tipado fuerte y excelente gestión de memoria a través de la JVM (*Java Virtual Machine*).

Uno de los frameworks de Java más utilizados es Spring Boot [19], siendo el framework de código abierto líder, que facilita el uso de marcos de trabajo basados en Java para crear microservicios y aplicaciones web autónomas.

En cuanto a sus ventajas, proporciona configuración automática de librerías, servidores embebidos (como Tomcat) y un ecosistema maduro para la creación de entornos de pruebas. Además, Aumenta drásticamente la productividad al eliminar la configuración manual repetitiva.

- **Python**

Es el lenguaje con mayor crecimiento actual debido a su sintaxis limpia y su dominio en Inteligencia Artificial y Ciencia de Datos, lo que facilita la integración de modelos de *Machine Learning* en el backend.

Python también puede ser útil para el desarrollo web, ofreciendo una amplia gama de frameworks como Django, Flask o FastApi.

En cuanto a Django [20], es un framework para desarrollo de aplicaciones con todo incluido "diseñado para el desarrollo rápido. Utiliza una arquitectura basada en módulos reutilizables, lo que reduce significativamente el tiempo de salida al mercado (*Time-to-Market*). Es la base de plataformas de gran escala como Instagram.

Por otro lado, FastApi [21], siendo una opción moderna y de alto rendimiento basada en el tipado de Python. Destaca por la generación automática de documentación técnica (OpenAPI/Swagger) y su soporte nativo para programación asíncrona, lo que lo hace ideal para APIs de baja latencia.

- **Node.js** [22]

Node.js no es un lenguaje en sí, sino un entorno de ejecución que permite ejecutar JavaScript en el servidor. Su arquitectura está basada en un modelo de E/S (entrada/salida) sin bloqueo y orientado a eventos. Es extremadamente eficiente para aplicaciones en tiempo real y servicios que manejan una gran cantidad de conexiones simultáneas, ofreciendo una alta escalabilidad horizontal.

Para su desarrollo web, cuenta con frameworks como **Express**, siendo un framework minimalista y flexible que ofrece libertad total al desarrollador, **NestJs**, que destaca por ser progresivo y modular (inspirado en Angular) que utiliza TypeScript por defecto, ideal para construir aplicaciones backend escalables y fáciles de mantener.

3.5.4. Sistemas de almacenamiento

El almacenamiento de datos ha evolucionado desde sistemas de archivos planos hacia Sistemas de Gestión de Bases de Datos (SGBD). En el desarrollo moderno, la elección de una base de datos no depende solo del volumen de información, sino de su estructura, la velocidad de consulta y el nivel de consistencia requerido. El estado del arte actual se divide principalmente en los modelos Relacionales (SQL) y No Relacionales (NoSQL).

Bases de Datos Relacionales (SQL) [23]

Este modelo organiza los datos en tablas con relaciones predefinidas. Cada tabla representa una entidad, estructurando la información en filas (registros) y columnas (atributos). Su principal fortaleza es la integridad referencial y el uso del lenguaje estandarizado SQL (*Structured Query Language*) para la manipulación de datos.

Estas bases de datos destacan por tener un esquema rígido (es necesario definir la estructura antes de insertar datos) y cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

Entre las más destacadas se encuentran: MySQL / MariaDB, PostgreSQL y Microsoft SQL Server.

Base de Datos NoSQL [24]

Diseñadas para modelos de datos específicos, las bases de datos NoSQL ofrecen esquemas flexibles que permiten una alta escalabilidad horizontal. Son ideales para aplicaciones modernas que requieren un desarrollo rápido e iterativo.

Se clasifican según su forma de almacenar la información:

- Orientadas a documentos: almacenan datos en formatos similares a JSON (como MongoDB).
- Clave - Valor: optimizadas para lecturas muy rápidas mediante una clave única (Redis).
- Orientadas a grafos: enfocadas en la relación entre nodos (como Neo4j).

Están caracterizadas por ofrecer escalabilidad horizontal mediante clústeres distribuidos y manejo eficiente de grandes volúmenes de datos no estructurados.

Bases de Datos en la Nube [25]

Representan la evolución del almacenamiento hacia el modelo *as-a-service*. Aunque pueden ser relacionales o NoSQL, se diferencian de las locales por aprovechar los beneficios de la computación en la nube:

- Escalabilidad y Agilidad: permiten aumentar los recursos de forma elástica según la demanda.
- Reducción de costes: eliminan la necesidad de gestionar hardware físico y mantenimiento de infraestructura.
- Alta disponibilidad: ofrecen réplicas automáticas y copias de seguridad gestionadas por el proveedor (como AWS RDS o Azure SQL).

3.5.5. Despliegue y estrategias

El despliegue de software (*deployment*) es el proceso crítico que transforma el código fuente en una aplicación funcional y accesible para los usuarios finales. En la actualidad, se concibe como un ciclo continuo y automatizado que garantiza la disponibilidad y estabilidad del sistema. Este proceso se compone de las siguientes etapas fundamentales:

- **Integración Continua (CI):** es la fase donde el código se compila, valida y somete a pruebas automáticas cada vez que un desarrollador realiza un cambio, asegurando la integridad de la base de código.
- **Entrega/Despliegue continuo (CD):** una vez validado el código, se realiza el envío automático a los entornos de prueba o producción.
- **Aprovisionamiento de infraestructura:** consiste en la creación y configuración de los recursos (servidores, bases de datos, redes) donde residirá la aplicación, idealmente mediante Infraestructura como Código (IaC).
- **Monitorización y *feedback*:** una vez en producción, la aplicación es vigilada para detectar errores en tiempo real, permitiendo un *rollback* (vuelta atrás) inmediato si se detectan anomalías.

Un aspecto fundamental es decidir que estrategia de despliegue se va a llevar a cabo cuando se realiza una aplicación y cómo se libera a los usuarios. Los tipos de estrategia pueden variar según las necesidades de la infraestructura y según la estrategia que se quiera seguir.

Según la infraestructura, el despliegue se puede realizar en servidores físicos propios (*on-premise*), a través de proveedores en la nube (como *AWS*), o ejecutándose a través de eventos, sin gestión directa de servidores (*serverless*). Por otro lado, según la estrategia de liberación que se siga, se encuentran los siguientes tipos: [26]

- *Rolling upgrade*
Se realiza una actualización secuencial del servicio, actualizando uno a uno los componentes. Este tipo de despliegue tiende a ser lento tanto en su ejecución como en su capacidad de volver atrás (*rollback*) y generalmente se emplea en combinación con otras estrategias.
- *Blue/Green*
Estrategia muy común que implica el uso de dos versiones del servicio: la versión nueva (*blue*) y la versión estable existente (*green*). Los usuarios continúan utilizando la versión verde mientras se prueba y evalúa la versión azul. Cuando se considera lista, los usuarios cambian a la versión azul. Si se detecta algún problema, es posible regresar a la versión verde.
- *Red/Black*
Similar a la estrategia blue/green, con la diferencia de que se desvía una pequeña porción del tráfico desde la versión negra (estable) hacia la nueva versión. Luego de evaluar su rendimiento, se realiza un desvío aún más grande del tráfico y, finalmente, se redirecciona todo el tráfico hacia la nueva versión. Se mantiene la versión negra en línea para permitir un *rollback* rápido si es necesario.
- *Dark Launcher*
Se implementa una nueva versión en paralelo a la versión estable existente. La particularidad aquí es que se duplica una parte del tráfico para observar cómo se comporta la nueva versión bajo una carga duplicada. Luego, se realiza un redireccionamiento parcial del tráfico hacia la nueva versión.
- *Canary Release*
Esta estrategia implica un despliegue parcial basado en el tiempo y el rendimiento de la nueva versión, consiguiendo de esta manera una implementación gradual y controlada.

En cuanto a las tecnologías y herramientas que se pueden utilizar para realizar el despliegue y gestionar una aplicación, se pueden clasificar en:

- Herramientas de contenerización, siendo las más utilizadas son Docker y Kubernetes. Por un lado, Docker [27] empaqueta software en unidades estandarizadas llamadas contenedores que incluyen todo lo necesario para que el software se ejecute, incluidas bibliotecas y herramientas de sistema, pudiendo ejecutar aplicaciones en cualquier entorno. Por otro lado, Kubernetes [28] facilita la automatización y la configuración declarativa,

además de administrar contenedores, orquestando la infraestructura de cómputo, redes y almacenamiento para que las cargas de trabajo de los usuarios no tengan que hacerlo.

- Herramientas de automatización, como GitHub Actions o Jenkins que ejecutan los flujos de trabajo de forma automática.
- Herramientas que permiten la definición de la infraestructura como código (IaC), como Terraform, que permiten definir y gestionar la infraestructura mediante archivos de configuración, facilitando la replicabilidad.

3.5.6. Autenticación y autorización

La seguridad es un apartado crítico en el desarrollo de cualquier sistema; es una responsabilidad transversal que se apoya en protocolos estandarizados para garantizar la interoperabilidad.

La autenticación (*AuthN*) se define como el proceso técnico de verificar que un usuario es quien dice ser, constituyendo la primera barrera de defensa. A continuación, se detallan los mecanismos más utilizados:

- **Basada en sesiones (formularios + cookies) [29]**

Es el método tradicional donde el usuario envía sus credenciales y, tras la validación, el servidor crea una sesión y devuelve una cookie al navegador. En cada petición posterior, el cliente envía dicha cookie para que el servidor valide el ID de sesión.

Es un método que destaca su alta seguridad en aplicaciones web simples y capacidad de revocación instantánea del acceso; sin embargo, presenta dificultades para escalar (debido al estado en el servidor).

- **Basada en tokens (JWT) [30]**

Es el estándar predominante en aplicaciones modernas y arquitecturas desacopladas. Tras la validación, el servidor genera un JSON Web Token (JWT) firmado digitalmente. El cliente lo almacena y lo incluye en la cabecera de cada petición para acceder a recursos protegidos.

Entre sus ventajas, destaca por una escalabilidad total (stateless) y compatibilidad nativa con aplicaciones móviles; sin embargo, presenta un riesgo de seguridad si el token es interceptado, ya que el acceso es válido hasta su expiración (difícil de revocar antes de tiempo).

- **OAuth 2.1 y OpenID Connect [31]**

OAuth 2.1 es un framework que permite el acceso delegado (una aplicación accede a datos de otra sin compartir contraseñas), mientras que OpenID Connect (OIDC) es una capa de identidad sobre OAuth que habilita el *Single Sign-on*.

Este sistema de autenticación destaca por su gran escalabilidad e integración con múltiples proveedores (Google, GitHub, etc.); sin embargo, presenta una mayor complejidad técnica en su implementación inicial.

- **Multifactor [32]**

Se trata de un mecanismo que combina diversos factores de verificación: conocimiento (PIN/contraseña), posesión (dispositivo móvil) e inherencia (biometría).

Como ventaja destaca por mitigar riesgos de robo de credenciales o tokens; aunque un atacante obtenga el JWT, carecería del segundo factor; sin embargo, Puede degradar la experiencia de usuario (UX) al añadir fricción y lentitud al proceso de entrada.

3.5.7. Elección de arquitectura y tecnologías

Tras el análisis de las opciones disponibles en cuanto a arquitectura, las decisiones tomadas para la creación del sistema son las siguientes:

Arquitectura de alto nivel y comunicación

La elección de una arquitectura **cliente-servidor** es el punto de partida para garantizar el desacoplamiento entre la interfaz de usuario y la lógica de negocio. Este modelo permite que el cliente móvil se centre exclusivamente en la experiencia del agricultor a pie de campo, mientras que el servidor gestiona de forma centralizada la persistencia de datos y las reglas de validación. Además, esta estructura facilita un soporte multiplataforma, permitiendo que en el futuro se puedan añadir otros clientes, como una versión web, sin modificar la lógica interna del sistema.

Para la comunicación entre estos componentes, se ha implementado una **API REST**, consolidada como el estándar “de facto” por su simplicidad y facilidad de integración con dispositivos móviles. Al tratarse de una arquitectura *stateless*, el servidor no almacena información de la sesión entre peticiones, lo que mejora significativamente la escalabilidad y la independencia de cada consulta. El intercambio de información se realiza mediante **JSON**, un formato de texto ligero, rápido de procesar y fácil de leer tanto por humanos como por los motores de desarrollo modernos.

Elección de arquitectura monolítica

A pesar del auge de los microservicios, para este proyecto se ha seleccionado una **arquitectura monolítica**. Esta decisión responde principalmente al alto acoplamiento existente entre las entidades del dominio de Olivaris, como las actividades, parcelas y recintos. Centralizar la lógica en una única unidad de despliegue simplifica en gran medida la gestión de la coherencia de datos y las validaciones de negocio, evitando la complejidad técnica que supondría coordinar múltiples servicios distribuidos.

Además, a diferencia de una arquitectura distribuida, las transacciones realizadas con un monolito sobre la base de datos son más rápidas y el desarrollo

es más ágil, permitiendo evolucionar el sistema sin romper contratos entre servicios independientes y evitando gestionar fallos distribuidos, lo cual resultaría costoso y complejo para el alcance de este proyecto.

Autenticación, autorización y control de roles

La seguridad será gestionada en Olivaris mediante **JSON Web Token (JWT)**. Este mecanismo es ideal para aplicaciones móviles porque permite identificar al usuario de forma fehaciente en cada petición mediante un token firmado digitalmente, eliminando la necesidad de sesiones activas en el servidor.

Gracias a la información contenida en el JWT, el backend puede restringir el acceso a funcionalidades específicas según el perfil o rol que tiene el usuario. Este control es relevante porque permite validar legalmente la delegación de permisos y qué acciones puede realizar cada usuario sobre el sistema.

Persistencia y despliegue

Para la gestión de los datos, el servidor utiliza un **ORM (JPA e Hibernate)**, lo que permite mapear los objetos de Java directamente a una base de datos relacional de forma automática. Esta práctica optimiza el trabajo con la base de datos, reduce errores y agiliza el desarrollo al ocultar la complejidad de las operaciones SQL manuales.

Finalmente, el despliegue se ha realizado mediante tecnologías de contenerización como **Docker**. El uso de contenedores garantiza que la aplicación se ejecute de la misma forma en cualquier entorno, ya sea en local o en el servidor final, empaquetando el código junto con sus bibliotecas y dependencias. Además, se utiliza **Nginx** como proxy inverso para centralizar el tráfico web, manejar certificados SSL y mejorar la seguridad general del sistema sin exponer la aplicación directamente al exterior.

Lenguajes de programación

En cuanto a los lenguajes de programación para el desarrollo del sistema, se han seleccionado los siguientes, buscando un equilibrio entre rendimiento, seguridad y mantenibilidad.

- **Frontend: React Native**

El acceso a la plataforma será principalmente a través de dispositivos móviles, dada la naturaleza del trabajo agrícola. La movilidad y disponibilidad inmediata que ofrece un dispositivo móvil son claves para que los agricultores interactúen con el sistema a pie de campo.

Las razones de la elección han sido:

- Utiliza JavaScript/TypeScript, uno de los lenguajes con mayor comunidad y demanda laboral, superando actualmente el ecosistema de Flutter en oportunidades regionales.
- A diferencia de las soluciones híbridas, React Native utiliza componentes nativos reales de iOS y Android, respetando las guías de diseño y ofreciendo una experiencia de usuario fluida.
- El uso de Expo facilita la configuración del entorno (evitando dependencias complejas de Android Studio) y permite actualizaciones *Over-The-Air* (OTA), permitiendo corregir errores sin que el usuario tenga que descargar una nueva versión de las tiendas oficiales.
- El ecosistema de librerías es muy amplio. Si se necesita integrar un SDK de terceros (pagos, mapas, analíticas), es muy probable que ya exista un paquete oficial o muy probado para React Native.

■ **Backend: Java con SpringBoot**

Para la lógica de servidor se ha optado por el ecosistema de Java, priorizando la robustez y la seguridad del sistema. Las razones por las que se ha elegido son las siguientes:

- El tipado estático de Java permite detectar la mayoría de los errores en tiempo de compilación. Esto ofrece una seguridad que lenguajes dinámicos como Python o JavaScript no pueden garantizar en la misma medida.
- Java maneja hilos de ejecución de forma eficiente, siendo superior en tareas que requieren un uso intensivo de CPU o procesamiento paralelo.
- Spring Boot ofrece una estructura de proyecto coherente y estandarizada. La gestión de acceso a datos y seguridad se realiza bajo patrones conocidos, facilitando que cualquier desarrollador se integre rápidamente en cualquier proyecto.
- Cuenta con *Spring Security*, uno de los marcos de seguridad más avanzados y granulares del mercado.
- El sistema está preparado para manejar transacciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) complejas. Además, la facilidad de Java para generar reportes en PDF y su compatibilidad con sensores IoT aseguran que el proyecto pueda crecer sin limitaciones técnicas.

Teniendo en cuenta lo anterior, aunque inicialmente la carga de procesamiento no sea elevada, Spring Boot permite gestionar transacciones ACID de forma fiable. Además, en este proyecto una única acción desencadenará varios procesos internos, por lo que resulta una opción adecuada.

Por otro lado, Java dispone de bibliotecas muy completas para la generación de PDF y, si en el futuro se incorporan nuevas funcionalidades, ofrece una buena capacidad de escalado, por ejemplo mediante la integración con sensores IoT.

También he elegido esta tecnología porque Java y Spring Boot siguen teniendo una alta demanda en el mercado laboral, por lo que considero que este proyecto es una buena oportunidad para comenzar a profundizar en su aprendizaje.

■ **Base de datos: PostgreSQL**

Se ha optado por un modelo relacional (SQL) debido a la complejidad de las asociaciones y dependencias jerárquicas entre las entidades del dominio de Olivaris.

El uso de PostgreSQL en lugar de MySQL o MariaDB se debe a las siguientes razones:

- Existen relaciones jerárquicas, por lo que un modelo relacional es el más eficiente para garantizar la integridad de los datos cuando existen múltiples tablas interconectadas.
- PostgreSQL destaca sobre otros motores SQL por su soporte avanzado de tipos no estructurados ([JSONB](#)), permitiendo lo mejor de los dos mundos: la rigidez del SQL y la flexibilidad de NoSQL en campos específicos.
- Su estricto cumplimiento de los principios ACID garantiza que la información nunca se corrompa, incluso ante fallos del sistema.

3.6. Conclusiones del estado del arte

El análisis realizado muestra que existen soluciones consolidadas para la gestión agrícola y oleícola, muchas de ellas orientadas al cuaderno de campo, la gestión de parcelas, la generación de informes o el control económico de explotaciones. Sin embargo, también se observa la oportunidad de desarrollar una plataforma centrada en la gestión colaborativa del olivar, con un modelo de permisos entre agricultores y entidades habilitadas y con un diseño alineado con el marco SIEX/REA/CUE. En este contexto, Olivaris se plantea como un prototipo funcional y extensible que permite explorar estas necesidades desde una arquitectura propia basada en servicios web, gestión geoespacial y control de acceso por roles.

4. Metodología

En este capítulo se expone cómo se ha desarrollado el proyecto en términos de organización y planificación de sus distintas fases.

4.1. Enfoque metodológico

Para la organización del proyecto se ha utilizado la metodología ágil **Scrum** [33]. Este tipo de metodología resulta útil porque permite realizar entregas del proyecto de forma iterativa y adaptarse a los cambios de manera rápida y eficiente.

A continuación se explican, en distintos apartados, las piezas clave que componen el marco Scrum.

4.1.1. Miembros del equipo Scrum

Un equipo Scrum está formado por varios miembros, y cada uno desempeña un rol con unas responsabilidades concretas. Aunque normalmente la cantidad de miembros es reducida, puede haber situaciones en las que este número crezca considerablemente si así lo requiere el proyecto. Por lo general, los equipos Scrum deben componerse de tres roles: el **propietario del producto** (o *Product Owner*), el *Scrum Master* y el **equipo de desarrollo**.

En el proyecto, estos roles han sido asignados de la siguiente manera:

- **Propietario del producto**

Este rol ha sido asignado tanto al tutor del TFG, el Dr. Juan Luis Jiménez Laredo, como al autor del presente proyecto. Se ha decidido así puesto que, entre ambos, se ha ido dando forma a la propuesta: qué necesidades debía cubrir el sistema, los requisitos funcionales y las historias de usuario. Asimismo, se encargaron de la priorización en cada iteración y de determinar cuándo se consideraba cerrada, comprobando que el desarrollo funcionase de forma incremental.

- **Scrum Master**

Este rol ha sido asignado al autor del proyecto, quien se ha encargado de asegurar el cumplimiento de las prácticas Scrum mediante la orientación, la eliminación de impedimentos, la optimización del rendimiento y el control de las entregas durante cada iteración.

- **Equipo de Desarrollo**

Este rol ha sido asignado al autor del proyecto, ya que ha sido el responsable de la ejecución técnica y del desarrollo del software.

4.1.2. Artefactos de Scrum

Los artefactos, dentro de la metodología Scrum, son la información que el equipo utiliza para definir el trabajo que hay que realizar para crear un producto. Hay tres tipos de artefactos: *backlog* del producto, *backlog* del sprint y meta o incremento del sprint.

- **Pila (*backlog*) del producto**

En este artefacto se agrupan todas las actividades en forma de lista que un equipo tiene que completar para crear el producto. En este proyecto se ha utilizado la herramienta GitHub Projects para crear un tablero donde se recojan todas las actividades que se han planteado (historias de usuario, requisitos, mejoras y correcciones).

- **Backlog del Sprint**

Esta es la lista de elementos (historias de usuario y actividades) que el equipo de desarrollo ha seleccionado para realizar durante un sprint. Con la herramienta GitHub Projects, además de poder mantener una pila de producto con todas las actividades, se pueden agrupar por iteraciones o sprints y crear tableros que solo muestren las actividades relacionadas con ese sprint.

- **Meta del Sprint**

Este artefacto corresponde a la parte del producto que se obtiene al acabar una iteración. En este caso, cada vez que se completa una iteración, se obtiene una nueva funcionalidad que se añade al producto y que puede ser probada, con el fin de ir obteniendo algo “tangible” que el *Product Owner* y el cliente puedan ver.

4.1.3. Eventos de Scrum

Con eventos de Scrum se hace referencia a todas aquellas prácticas que se realizan dentro del equipo Scrum de manera periódica, englobando las ceremonias y reuniones que se llevan a cabo en distintos momentos durante el desarrollo del producto.

Entre los eventos de Scrum, se encuentran los siguientes:

- **Organización del Backlog**

Consiste en realizar una limpieza y actualización de la pila del producto, de manera que pueda estar lista para usarse por parte del equipo de desarrollo y del resto de miembros del equipo en cualquier momento. Esta labor es realizada por el *Product Owner*.

- **Planificación de Sprints**

La planificación del sprint consiste en una reunión que realiza el equipo de desarrollo para planificar cuál será el trabajo que se va a realizar durante el sprint. Esta reunión es dirigida por el *Scrum Master* y sirve para añadir las

historias de usuario o actividades importantes que se tengan que realizar, pasándolas del *Product Backlog* al sprint específico.

- **Ejecución de Sprint**

Esta fase corresponde a la ejecución de todo lo planteado en el sprint, desarrollando e implementando todas aquellas tareas que han sido fijadas.

- **Reunión rápida**

Este tipo de reunión se realiza al comienzo de la jornada de manera diaria. Son reuniones de corta duración donde se plantea lo que se va a realizar ese día y si hay algún obstáculo, de manera que todo el equipo trabaje en sintonía y se acerque cada vez más a la consecución del sprint.

- **Revisión del Sprint**

La revisión del sprint consiste en una reunión en la que el equipo de desarrollo muestra a las partes interesadas del producto los elementos del backlog que están finalizados a través de una “demo”, decidiendo el *Product Owner* si se publica o no el incremento.

- **Retrospectiva del Sprint**

Esta es la última reunión, donde el equipo se reúne para analizar qué cosas se han conseguido, cuáles han funcionado y dónde han surgido problemas, buscando de esta forma qué aspectos deben mejorarse de cara a las próximas iteraciones.

4.1.4. ¿Por qué usar la metodología Scrum?

La elección de la metodología Scrum se ha basado en sus valores y en las ventajas que ofrece, siendo una de las metodologías más utilizadas actualmente por los equipos de trabajo.

Para empezar, el hecho de trabajar en equipo hace que cada uno de los miembros tenga un papel importante que desarrollar para alcanzar la meta establecida, de modo que todos se sientan parte del proceso y aporten valor al producto. Además, la alta comunicación que existe en esta metodología y el hecho de que sea iterativa hacen que se busquen siempre la autocrítica en el equipo y su mejora de cara a las próximas iteraciones, permitiendo así que el producto vaya mejorando y adecuándose a lo que el cliente quiere.

Además de lo anterior, Scrum tiene una alta flexibilidad y capacidad de adaptación, ya que permite realizar cambios en la estructura cuando son necesarios, además de permitir entregar el producto software a las partes interesadas en periodos cortos de tiempo. Esto ayuda a comprobar si el camino hacia el objetivo es el adecuado o si hay que realizar modificaciones.

Con respecto al presente proyecto, hay que destacar que los roles, artefactos y reuniones han sido adaptados, ya que el equipo de desarrollo ha estado compuesto por un solo individuo y muchas decisiones se han tomado en conjunto con el tutor del TFG. Sin embargo, se han seguido todos los principios y la teoría Scrum de la mejor forma posible.

4.1.5. Adaptación de la metodología Scrum al proyecto

Una vez explicadas todas las partes que componen la metodología Scrum, se va a proceder a explicar cómo esas partes han sido adaptadas y tratadas en el presente proyecto.

En cuanto a los miembros del equipo Scrum, tanto el equipo de desarrollo como el Scrum Master han sido asumidos por el autor del TFG, mientras que el Product Owner o Propietario del Producto ha sido asumido por el tutor del TFG.

Como se ha comentado anteriormente, se ha utilizado la herramienta GitHub Projects [34] para elaborar el *Product Backlog*, donde se han añadido todas las historias de usuario así como actividades necesarias para poder desarrollar el software. A través de esta pila de producto se han ido organizando los diferentes sprints con las actividades necesarias para la realización de cada uno de ellos, ya que la herramienta Projects permite crear distintas vistas (cada una corresponde a un sprint) y asignar tareas del backlog a estas vistas.

Tanto el Product Backlog como la planificación de cada uno de los sprints han sido llevados a cabo por el autor del proyecto, añadiendo toda la información (historias de usuario y tareas) que ha considerado necesaria para desarrollar el software junto con las recomendaciones del tutor del TFG (Dr. Juan Luis Jiménez Laredo) cuando ha sido requerido. Asimismo, la ejecución de cada sprint ha sido realizada por el autor del proyecto. En las imágenes 4.1 y 4.2 se puede ver un ejemplo de cómo es una historia de usuario y de cómo sería un sprint:



Figura 4.1: Ejemplo de una Historia de Usuario

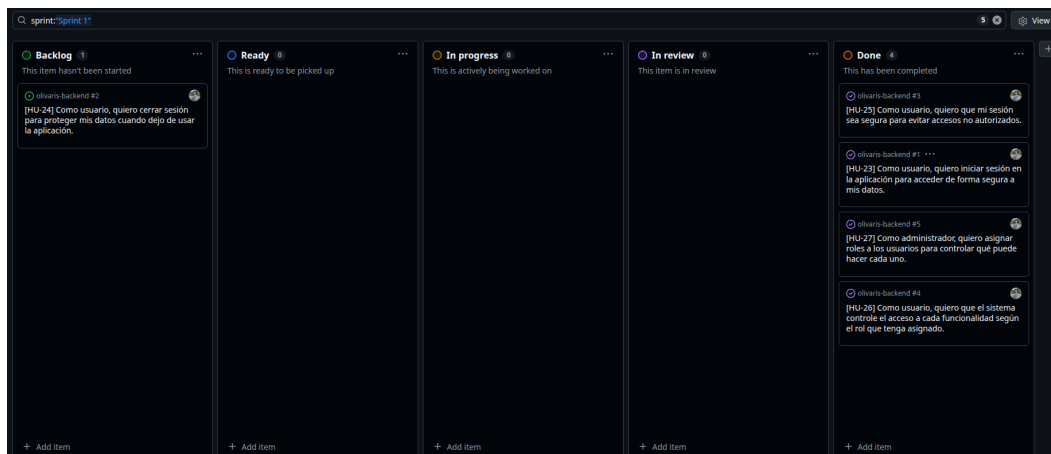


Figura 4.2: Ejemplo del tablero para el sprint 1

En cuanto a las reuniones realizadas durante todo el desarrollo del sistema, han sido tratadas de la siguiente forma:

- **Reunión diaria:** ha sido realizada por el autor del proyecto, revisando qué hizo el último día y planteando las siguientes tareas a desarrollar, teniendo en cuenta los plazos fijados y reorganizando tanto el sprint como el backlog en caso necesario.
- **Revisión del sprint:** esta reunión ha sido realizada entre el tutor del TFG y el autor del proyecto al finalizar cada sprint. En esta reunión se han explicado las partes del software que han sido desarrolladas, se ha mostrado el funcionamiento del sistema y se ha debatido sobre posibles correcciones a realizar.
- **Reunión de retrospectiva:** ha sido realizada junto con el tutor del TFG y ha servido para tratar qué aspectos de lo realizado durante el sprint son mejorables y qué se podría añadir o contemplar para futuros sprints. También ha servido para fijar las ideas de lo que iba a ser desarrollado en el próximo sprint y qué camino se iba a ir tomando.

4.1.6. Sprints realizados

Para alcanzar el objetivo fijado y desarrollar este software, se han realizado distintos sprints o iteraciones, cada uno centrado en una funcionalidad nueva que se iba integrando en el sistema. Los sprints han sido los siguientes:

- **Sprint 1:** en este sprint se comenzó implementando en el backend el sistema de autenticación y autorización de usuarios, así como el desarrollo de la base de datos inicial (a la que luego se fueron añadiendo tablas), englobando los servicios relacionados con el registro de usuarios, el inicio de sesión, la confirmación del registro a través de notificaciones enviadas por correo electrónico y la creación de JSON Web Tokens para garantizar la seguridad en los posteriores accesos al servidor.

- **Sprint 2:** este sprint ha sido una continuación del anterior, ya que se agregó a la base de datos y al sistema la interacción de los usuarios con una “entidad habilitada” (puede ser una cooperativa, empresa, etc.). Esta interacción se basa en los permisos que un usuario puede darle a una entidad para manipular sus datos, teniendo cada usuario que pertenece a la entidad unos roles determinados (administrador o agricultor).

Para desarrollar este sprint se ha tenido que investigar qué tipo de usuarios hay en un CUE comercial y qué permisos puede delegar un usuario sobre sus datos a una entidad [3]. Además, en este sprint se han introducido **tests unitarios y de integración**, por lo que se ha tenido que investigar cómo hacerlo en Spring Boot y también se ha creado un *workflow* en [GitHub Actions](#), para que cada vez que se suba un cambio a la rama de trabajo, se ejecuten los tests y se construya el programa.

- **Sprint 3:** en este sprint se implementó la lógica relacionada con la creación de parcelas, recintos y actividades fitosanitarias asociadas a cada recinto. También se implementó el frontend, así como las peticiones a la API desarrollada para obtener los datos que se muestran en el lado del cliente.

En este apartado fue necesario investigar cómo programar en React Native con JavaScript, cómo utilizar los componentes y cómo mostrar en el mapa las parcelas de un usuario.

- **Sprint 4:** este ha sido el último sprint. Se centró principalmente en desplegar el backend realizado en un servidor en la nube, de manera que pudiera estar continuamente ejecutándose. Para ello, se investigó cómo funciona Docker y qué archivos eran necesarios implementar en función del estado del proyecto.

Además de esto, se investigó cómo generar una APK del frontend para que pudiera ser descargada desde cualquier dispositivo móvil.

Finalmente, el sprint también sirvió para añadir mejoras en cuanto a la usabilidad de la aplicación y otras funcionalidades que se consideraron teniendo en cuenta el tiempo disponible.

Al finalizar cada sprint, se ha realizado un Sprint Review junto con el tutor del presente TFG para debatir si se ha cumplido lo establecido, qué cosas son necesarias modificar y por donde hay que dirigir el siguiente sprint.

Finalmente, se puede ver un *roadmap* del proyecto, según se han ido realizando actividades desde GitHub Projects:

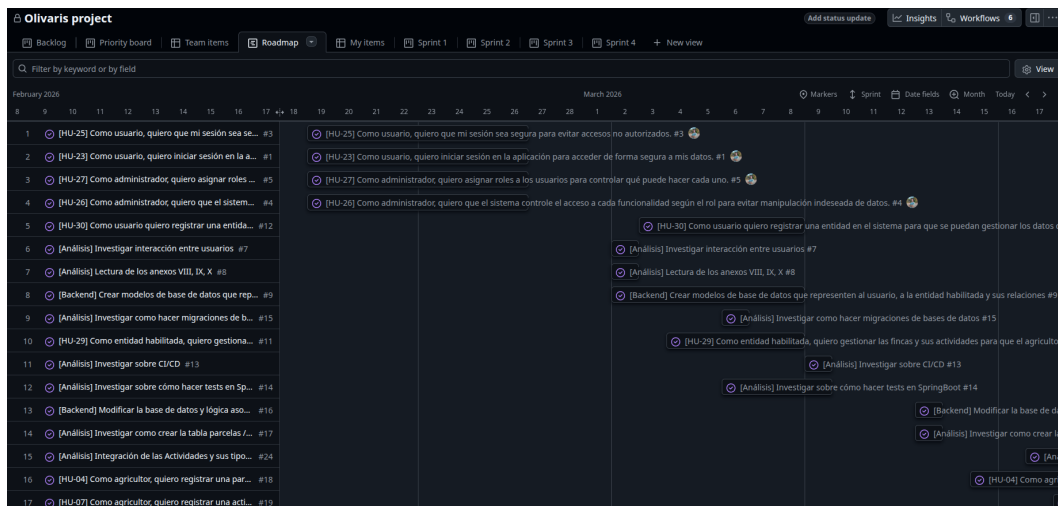


Figura 4.3: Roadmap del proyecto visto desde GitHub Projects

4.2. Organización del código y repositorios

La gestión del código fuente del proyecto se ha organizado a través de la organización de GitHub [OlivarisUGR](#), que agrupa tres repositorios públicos y separa claramente las responsabilidades de cada parte del sistema.

- **olivaris-backend**: contiene el código del servidor y toda la lógica de negocio implementada en Java. En este repositorio se encuentran los servicios de autenticación, acceso a datos, reglas de dominio y despliegue del backend.
- **olivaris-frontend**: alberga el desarrollo de la interfaz de usuario de la aplicación, realizada con TypeScript. Aquí se centraliza la parte visual y la comunicación con la API del backend.
- **olivaris-docs**: reúne la documentación del proyecto en formato $\text{\LaTeX}$, incluyendo la memoria, anexos y el presente TFG.

Esta separación ha permitido mantener un repositorio por cada bloque funcional del proyecto, facilitando el mantenimiento, la revisión de cambios y la evolución independiente de cada componente. Además, la organización en GitHub ha servido como punto central para coordinar el trabajo, mientras que la planificación y el seguimiento de tareas se han apoyado en GitHub Projects.

4.3. Presupuesto del proyecto

En esta sección del capítulo se va a detallar la cuantía económica necesaria para la ejecución del proyecto, incluyendo todos los costes y gastos, con el objetivo de ofrecer una visión de los recursos financieros requeridos, facilitar la toma de decisiones y servir de referencia para llevar un control de lo invertido.

Para elaborar la presupuesto, se va a dividir en distintas secciones:

Coste de personal

El desarrollo del proyecto ha requerido una dedicación aproximada equivalente a cuatro meses de trabajo a media jornada, incluyendo las tareas de análisis, diseño, implementación, pruebas, documentación e investigación de las tecnologías utilizadas.

Para estimar el coste de personal, se ha tomado como referencia el salario bruto anual de un perfil de *desarrollador junior full stack* relacionado con la ingeniería del software, utilizando como base los datos salariales publicados por Glassdoor [35]. El rango salarial consultado se sitúa entre 17.000 € y 22.000 € brutos anuales.

Con el objetivo de obtener una estimación acorde al nivel técnico del proyecto, se ha considerado el valor superior del rango (22.000 € brutos anuales) además de un extra por realizar labores que se escapan de la competencia de un desarrollador full stack junior, como es la extracción de requisitos, la definición de la arquitectura y de la base de datos. Teniendo esto en cuenta, el salario asciende a **26.000 €** brutos anuales, que equivalen aproximadamente a un coste de **13'54 €/hora**.

Realizando la siguiente operación: $13'54 \text{ €/h} \times 320 \text{ h}$ (4 meses a media jornada), el coste total del desarrollador es de **4.332'8 €** (aproximadamente).

Costes de infraestructura y Operación del servidor

El sistema ha sido desplegado en un servidor de **Hosting** facilitado por el **CSIRC**. Estos servicios están destinados a la comunidad universitaria así como para los grupos de investigación. El servidor solicitado cuenta con las siguientes características:

- 2vCPU (2 núcleos de procesador virtuales). Son utilizados en servidores en la nube y máquinas virtuales, y representan una parte asignada de un procesador físico.
- 4GB de memoria RAM.
- 75GB de almacenamiento.

El servidor con estas características tiene un precio de **48 €** trimestrales. Como el despliegue se hizo durante la fase final del proyecto, con los 3 meses alquilados es suficiente por ahora.

Gastos de suministros

Estos gastos incluyen todos aquellos costes derivados del desarrollo del proyecto, tales como electricidad y el acceso a Internet.

- **Internet:** conexión a Internet durante los 4 meses de desarrollo, con un precio de 27 € al mes, lo que hacen un total de **108 €**.
- **Electricidad:** el pago de electricidad cada mes es de 42.5 €, por lo que hace un total (con respecto a los 4 meses) de **170 €**.

El **coste total** de los suministros, al sumar ambas cantidades, es de **278 €**.

Total presupuestado

Para calcular el total presupuestado, se va a sumar la cantidad total de cada una de las secciones:

4.332'8 € (coste personal) + 48 € (servidor CSIRC) + 278 € (gasto de suministros)
= **4.658'8 €**

5. Análisis y requisitos

Antes de empezar con el desarrollo e implementación del sistema es necesario realizar una fase previa. En esta fase se van a extraer los requisitos que va a tener el sistema junto con las funcionalidades que tendrá. A continuación se va a explicar el proceso que se ha seguido.

5.1. Recopilación de información

Para recopilar información se ha recurrido a la realización de entrevistas, análisis de documentos y consultas a través de correo electrónico con el personal adecuado.

■ Entrevistas

Se han realizado varias entrevistas con la finalidad de ajustar el sistema lo máximo posible a lo que se necesita o sería interesante que hubiese en el sector agrícola.

• **José Cobos Repiso - Agricultor de Cooperativa San Lorenzo de Zagra**

Con esta reunión se pudo ver cómo es el día a día de un agricultor que tiene varias fincas y cómo las gestiona. José mostró las distintas tablas que tiene creadas en Excel a través de las cuáles puede llevar un control exacto de qué actividad ha realizado en cada campaña y sobre qué finca en particular. Fueron varios los puntos planteados durante la reunión sobre qué podría tener la aplicación Olivaris. Entre ellos se destacaron:

- Funcionalidad de introducir un cuaderno de campo, siendo un punto muy interesante a tratar ya que el próximo enero de 2027 va a ser obligatorio. Sobre esta funcionalidad se ha realizado un análisis más profundo que será comentado a continuación.
- Funcionalidad sobre control de costes y gastos, además de los pagos realizados a los empleados contratados.
- Funcionalidad sobre gestionar parcelas y las actividades realizadas en cada una de ellas, de manera que se tenga una vista de todas las parcelas que son de su propiedad y se puedan gestionar las actividades de cada una de ellas.

Gracias a esta reunión se pudo obtener una aproximación de qué tipo de tablas se tendrían en la base de datos, cómo se podrían relacionar entre ellas y qué funcionalidades tendría la aplicación.

• **Juan Luis Jiménez Laredo - Profesor titular de la ETSIIT y Director del presente TFG**

Las reuniones realizadas con Juan Luis han servido para llevar una

revisión continua del desarrollo del proyecto, con la finalidad de saber qué requisitos funcionales tendrá el sistema y qué tareas hay que priorizar. Además, contando también con experiencia en el sector de la oliva, ha podido dar una visión del sistema más precisa, siendo esto de gran ayuda para definir cuáles son las funcionalidades más relevantes de la aplicación.

Con la información extraída de la reunión realizada con José sobre el cuaderno de campo y con las conversaciones mantenidas con Juan Luis, se exploró la opción de introducir la funcionalidad del cuaderno de campo digital.

El **cuaderno de campo** [36] es una especie de cuaderno de bitácora que registra el uso de productos químicos (fitosanitarios y fertilizantes) sobre una explotación con la finalidad de vigilar la posible contaminación que pueda surgir derivada del abuso de estos productos. Este concepto está muy relacionado con el **Cuaderno Digital de Explotación Agrícola (CUE)**.

Según la Conserjería de Agricultura, Pesca, Agua y Desarrollo Rural [37], el Cuaderno Digital de Explotación Agrícola (o CUE digital), es la herramienta que se va a utilizar en España para anotar las actividades que se realizan en una explotación agrícola, recogiendo las actividades fitosanitarias, de fertilización o riegos aplicados. Para poder cumplimentar este CUE digital, es necesario que la explotación este previamente inscrita en REAFA (Registro Autonómico de Explotaciones Agrarias en Andalucía), ya que esta será la fuente que proporcione la información de las parcelas y cultivos existentes.

Este CUE digital, a diferencia del CUE de papel, obliga a que sea tratado desde medios electrónicos y que la información que lo cumplimente sea con los datos que la Administración tenga disponibles, procedentes del Registro Autonómico de Explotaciones (REA).

El aspecto que hace interesante la implementación de esta funcionalidad es que a partir del 1 de enero de 2027 será obligatorio el uso del CUE digital para consignar la información de los tratamientos fitosanitarios.

A partir de este punto, se ha realizado una búsqueda de cómo acceder a la API que proporciona la Junta de Andalucía para obtener los datos de una explotación y poder exportarlos al CUE digital. Para ello, primero es necesario dar de alta la “empresa” que va a desarrollar este proyecto en un portal junto con su certificado digital, estando aquí el primer problema: actualmente no hay constituida una empresa de desarrollo de software particular, por lo que no hay certificado digital que se pueda utilizar para darse de alta y poder acceder a la API.

Las soluciones buscadas han conllevado el envío de correos electrónicos a entidades que puedan suplir este problema y hacer de “empresa de desarrollo”. Los correos enviados se van a comentar a continuación.

■ Correos electrónicos

Se han enviado correos electrónicos a las siguientes entidades, con la finalidad de buscar ayuda para dar de alta una empresa en el sistema y poder obtener el certificado digital que permita acceder a la API.

- **Nomia Energy:** empresa de Granada con la que se ha contactado para darla de alta en el registro como empresa desarrolladora del Cuaderno Digital de Explotación comercial. Con su certificado digital se podría acceder a la API pero finalmente no se llegó a un acuerdo y esta opción se descartó.
- **Trabajador de la Junta de Andalucía:** se mantuvieron conversaciones con Juan Carlos Martín Fuentes, trabajador de la Junta de Andalucía, para recoger qué requisitos son necesarios para registrarse como empresa de desarrollo y sobre cómo tiene que ser el certificado digital, ya que se contempló la opción de trabajar con un certificado “autofirmado” en el entorno de preproducción (por no tener una empresa que proporcione el certificado), pero finalmente esta opción se descartó porque no admiten certificados autofirmados, debido a la poca fiabilidad que tienen.
- **Jefe de Seguridad de los Servicios Informáticos de la UGR:** se contactó con el Jefe de Seguridad para comentarle qué situación había, qué se quería realizar y cómo es el tipo de certificado digital que se necesita para poder acceder a esta API. Se comentó las partes que deben incluir el certificado: parte pública en formato .PEM, [TLS Web Client Authentication](#) (clientAuth) y el subdominio de la entidad o el CIF/NIF (el de la UGR en este caso), con la finalidad de saber si la UGR puede gestionar un certificado con estas características y de esta forma poder acceder al sistema.

La funcionalidad del CUE digital es muy interesante pero ha tenido un problema: el proceso para extraer la información es largo y lento (muchos correos electrónicos y tiempos de espera entre ellos), así como problemas derivados del uso de un certificado digital que no se tiene, por lo que esta funcionalidad ha pasado a un segundo plano. El proyecto se ha centrado en desarrollar el sistema con una base de datos y lógica de negocio propia que “supla” la falta de acceso a los endpoints de la API que se encargarían de realizar estas acciones.

5.2. CUE comercial

Durante esta fase de análisis, viendo la necesidad que va a haber a partir de enero del próximo año 2027 en facilitar un CUE comercial (aplicación software) a los titulares de una explotación, se ha orientado el desarrollo del sistema hacia este ámbito. Esta decisión ha causado que el sistema evolucione de un ERP clásico que se iba a desarrollar en un principio, a acotar el sistema a un diario de actividades

que permita registrar su información en un CUE digital utilizando los datos suministrados por el usuario y por la REA (en caso de poder acceder a la API).

A partir de este punto, el sistema a desarrollar debe poder acceder a la API o interfaz que suministran el **SIEX** (Sistema de Información de Explotaciones Agrarias) para consumir sus endpoints, permitiendo obtener datos de la REA y exportar datos al CUE digital. Sin embargo, este proceso empezó siendo lento y tedioso, ya que requería un certificado que como se ha comentado antes, ha sido difícil de conseguir, por lo que se tomó la decisión de elaborar un sistema con una base de datos propia, que tenga las entidades necesarias para almacenar la información que podrían rellenar los datos del CUE digital. De esta manera, el sistema también estaría adaptado a una posible integración futura de la API, ya que sería añadir el cliente de esta y que los datos sean enviados directamente a los servidores del REA y CUE que maneja el ministerio en vez de almacenarlos en las tablas.

Para obtener información sobre el CUE, se han accedido a los siguientes anexos principalmente:

- **Anexo IX - Modelo de autorización de acceso a la información del REA y CUE:** este anexo se ha utilizado para saber qué permisos son los que cede un usuario que pertenece a una entidad habilitada sobre dicha entidad. Además de este, también se ha utilizado el anexo **Anexo VIII - Modelo de Autorización de Representación** para investigar sobre los permisos.
- **Anexo VI - Interfaz Único Común:** este anexo se ha utilizado para comprender cómo funciona la interfaz de acceso o API. Como al final no se pudo acceder a ella, la consulta a este anexo fue al inicio del proyecto y sin entrar en detalle.
- **Anexo X - Registro de Autorizaciones para el uso de Aplicaciones Comerciales:** este anexo ha servido para comprender qué tipo de usuarios van a interactuar con la aplicación y entre ellos, pudiendo distinguir entre tres: empresa desarrollador del software, entidad habilitada y titular de la explotación.
- **Anexo III - Registros de los tratamientos fitosanitarios y de fertilización:** este anexo se ha utilizado para investigar sobre qué datos de una explotación así como de una actividad fitosanitaria son utilizados para el cuaderno de explotación.

Otras fuentes que se visitaron para obtener más información fue el **Cuaderno de Explotación de la Junta de Andalucía**. En esta página se ha consultado información sobre el CUE Comercial (aplicación desarrollada), como por ejemplo: la fecha en la que será obligatorio el uso del CUE Digital para los agricultores en las actividades de tipo fitosanitario, o el proceso que tiene que seguir una empresa para darse de alta como CUE Comercial y poder acceder a la interfaz IUWS.

5.3. Personas

Una parte muy importante durante el análisis de un sistema son los usuarios finales que lo utilizan. Por ello, se han creado personas ficticias que puedan representar al usuario que utiliza la aplicación, consiguiendo saber de esta manera cuáles son sus inquietudes y qué necesitan para poder realizar sus tareas correctamente.

En las imágenes [5.1](#) y [5.2](#) se muestran los tipos de usuarios que se han elaborado según las necesidades de la aplicación.

PLANTILLA DE PERSONAJE		
Nombre	Andrés Ramírez González	
Edad	43	
Sexo	Masculino	
Educación	Técnico agrícola	
Discapacidad	Ninguna	
Tipo de usuario	Agricultor	
Contexto de uso		
Cuándo	Durante su jornada laboral	
Dónde	En su finca	
Tipo de ordenador	No usa ordenador. Utiliza el móvil ya que es más cómodo de desplazar y más rápido de interactuar	
Misión		
Objetivo	Poder ver la parcela en la que se encuentra, crear una actividad y asignarla a esa parcela	
Expectativas	Encontrar rápidamente la parcela y que la creación de la actividad sea intuitiva	
Motivación		
Urgencia	Diariamente	
Deseo	Controlar todas las actividades realizadas en sus fincas	
Actitud hacia la tecnología		
Cómico con la gestión en papel pero abierto a dar el paso hacia el uso de las tecnologías actuales		

Figura 5.1: Persona 1: Agricultor

PLANTILLA DE PERSONAJE		
Nombre	María Díaz Mendoza	
Edad	35	
Sexo	Femenino	
Educación	Administración de Empresas	
Discapacidad	Ninguna	
Tipo de usuario	Administradora de Cooperativa	
Contexto de uso		
Cuándo	Durante su jornada laboral	
Dónde	En la oficina	
Tipo de ordenador	Utiliza una tablet ya que la gestión es más rápida	
Misión		
Objetivo	Gestionar las actividades de la finca de un agricultor de la cooperativa que ha dado su autorización para hacerlo	
Expectativas	Encontrar rápidamente al agricultor y crear la actividad	
Motivación		
Urgencia	Diariamente	
Deseo	Gestionar las actividades de los agricultores pertenecientes a la cooperativa	
Actitud hacia la tecnología		
Familiarizada y con rápida adaptación al cambio		

Figura 5.2: Persona 2: Administradora de la Cooperativa

5.4. Escenarios

Una vez que han sido planteadas las personas, se pasa a describir los escenarios. Los escenarios son una descripción de como el usuario utiliza el sistema para realizar una determinada tarea y lograr su objetivo. A través de esta técnica se consigue ver de una forma más clara y cercana cómo debe de funcionar el sistema,

pudiendo ayudar a encontrar y definir requisitos funcionales.

Los escenarios planteados para las personas creadas en el apartado anterior son los siguientes:

■ **Primer escenario: Andrés Ramírez González (Agricultor)**

Andrés se encuentra realizando su jornada laboral en una de sus fincas y utiliza su móvil como herramienta principal ya que necesita rapidez y se está moviendo constantemente.

Andrés abre la aplicación y esta muestra un mapa con la geolocalización actual y las parcelas de su finca bien delimitadas. Procede a pulsar en una parcela y el sistema muestra información básica sobre la finca y un botón que le permite crear una actividad nueva asociada a esa parcela. Andrés selecciona esta opción y el sistema le envía a una página que muestra un sencillo e intuitivo formulario que debe rellenar indicando el tipo de actividad que va a crear.

Una vez guardada la información en el sistema, se muestra un mensaje de confirmación y se asocia automáticamente a la parcela, pudiendo revisar Andrés el historial de actividades de esa finca en cualquier momento y controlando todo el trabajo realizado en cualquier momento sin necesidad de papel.

- **Valor añadido:** identificación rápida de la parcela, creación de actividades de forma ágil y control centralizado de todas las tareas.

■ **Segundo escenario: María Díaz Mendoza (Administradora de Cooperativa)**

María se encuentra en la oficina durante su jornada laboral, utilizando una tablet para gestionar de forma rápida y eficiente tanto las fincas como actividades de los agricultores.

María accede a la aplicación con su perfil de administradora de cooperativa y el sistema le muestra una lista con todos los agricultores asociados que han autorizado la gestión de sus fincas, por lo que procede a seleccionar uno de ellos y el sistema le muestra una vista con todas las actividades asociadas a ese agricultor.

María quiere añadir una nueva actividad por lo que pulsa sobre el botón que agrega una actividad. A continuación, se le presenta un formulario que debe de rellenar según la actividad a realizar.

Una vez rellenado y guardado, el sistema la almacena automáticamente y la asocia a la parcela, quedando registrada para una consulta posterior tanto por María como por un agricultor. Este proceso se podrá repetir tantas veces como sea necesario con distintos agricultores, asegurando una gestión centralizada y organizada.

- **Valor añadido:** gestión remota de varias fincas autorizadas, acceso rápido a las actividades y mejora de la coordinación y trazabilidad entre cooperativa y agricultores.

5.5. Historias de Usuario

Una **Historia de Usuario** [38] es una descripción general e informal de una función de software escrita desde la perspectiva del usuario final o cliente. Su propósito es describir cómo un elemento de trabajo aporta un valor particular al cliente.

Son redactadas con un lenguaje sencillo y no técnico y consiguen con pocas palabras describir el resultado deseado. A partir de ellas se pueden establecer los requisitos funcionales del sistema y son añadidas a las iteraciones o *sprints* dentro de la metodología ágil [Scrum](#).

Las historias de usuario son importantes ya que ayudan a los equipos de trabajo a centrarse en solucionar problemas para usuarios reales, a que piensen de forma crítica y creativa sobre cómo lograr una meta de forma eficaz y ayudan a motivar al equipo ya que su resolución implica pequeños retos y victorias.

Para la realización de las Historias de Usuario del presente proyecto, la estructura utilizada ha sido la siguiente:

Como [tipo de usuario que realiza la acción], *quiero* [la acción que se quiere realizar] *para* [resultado obtenido].

Además de la historia de usuario, se ha añadido el identificador de esa historia de usuario y un título identificativo.

Según los análisis realizados y la información obtenida, se han establecido las siguientes historias de usuario:

ID	HU-1	Nombre	Inicio de sesión
Descripción	Como usuario, quiero iniciar sesión en la aplicación para acceder de forma segura a mis datos.		

Tabla 5.1: Historia de usuario HU-1

ID	HU-2	Nombre	Sesión segura
Descripción	Como usuario, quiero que mi sesión sea segura para evitar accesos no autorizados.		

Tabla 5.2: Historia de usuario HU-2

ID	HU-3	Nombre	Asignación de roles
Descripción		Como administrador, quiero asignar roles a los usuarios para controlar qué puede hacer cada uno.	

Tabla 5.3: Historia de usuario HU-3

ID	HU-4	Nombre	Control de funcionalidad según rol
Descripción		Como administrador, quiero que el sistema controle el acceso a cada funcionalidad según el rol para evitar manipulación indeseada de datos.	

Tabla 5.4: Historia de usuario HU-4

ID	HU-5	Nombre	Registro de entidades habilitadas
Descripción		Como usuario quiero registrar una entidad en el sistema para que se puedan gestionar los datos de los usuarios que pertenezcan a ella.	

Tabla 5.5: Historia de usuario HU-5

ID	HU-6	Nombre	Control de actividades
Descripción		Como entidad habilitada, quiero gestionar las fincas y sus actividades para que el agricultor no tenga que realizar estas tareas.	

Tabla 5.6: Historia de usuario HU-6

ID	HU-7	Nombre	Gestión de parcelas
Descripción		Como agricultor, quiero registrar una parcela con sus datos para poder gestionarla correctamente.	

Tabla 5.7: Historia de usuario HU-7

ID	HU-8	Nombre	Registro de actividades
Descripción		Como agricultor, quiero registrar una actividad diaria para llevar un control centralizado.	

Tabla 5.8: Historia de usuario HU-8

ID	HU-9	Nombre	Registro de actividad fitosanitaria
Descripción		Como agricultor, quiero registrar un tratamiento fitosanitario para cumplir la normativa.	

Tabla 5.9: Historia de usuario HU-9

ID	HU-10	Nombre	Registro de actividad futura o planificada
Descripción		Como agricultor, quiero planificar actividades futuras para organizar el trabajo de la campaña.	

Tabla 5.10: Historia de usuario HU-10

ID	HU-11	Nombre	Modificación de actividad
Descripción		Como agricultor, quiero modificar una actividad almacenada para corregir datos o marcarla como realizada.	

Tabla 5.11: Historia de usuario HU-11

ID	HU-12	Nombre	Eliminación de actividad
Descripción		Como agricultor, quiero eliminar una actividad para no tenerla guardada en el sistema.	

Tabla 5.12: Historia de usuario HU-12

ID	HU-13	Nombre	Control de usuarios
Descripción		Como administrador, quiero obtener los usuarios registrados en el sistema para llevar un control de quien está registrado.	

Tabla 5.13: Historia de usuario HU-13

ID	HU-14	Nombre	Consulta de parcela
Descripción		Como agricultor, quiero consultar mis parcelas, para obtener información sobre ellas.	

Tabla 5.14: Historia de usuario HU-14

ID	HU-15	Nombre	Consulta de actividades
Descripción		Como agricultor, quiero consultar el historial de actividades de un usuario para saber todo lo que ha realizado.	

Tabla 5.15: Historia de usuario HU-15

ID	HU-16	Nombre	Visualización de parcelas en un mapa
Descripción		Como agricultor, quiero visualizar mis parcelas en un mapa para conocer rápidamente el estado de las tareas.	

Tabla 5.16: Historia de usuario HU-16

ID	HU-17	Nombre	Cierre de sesión
Descripción		Como usuario, quiero cerrar sesión para proteger mis datos cuando dejo de usar la aplicación.	

Tabla 5.17: Historia de usuario HU-17

ID	HU-18	Nombre	Registro de actividades
Descripción		Como agricultor, quiero registrar una actividad directamente desde el mapa para agilizar el trabajo en campo.	

Tabla 5.18: Historia de usuario HU-18

5.6. Criterios de aceptación

Las historias de usuario definidas tienen unos criterios de aceptación asociados que establecen cuando una historia de usuario ha sido completada. Los criterios de aceptación para cada historia de usuario son los siguientes:

■ HU-01

- Dado que introduzco usuario/email y contraseña correctos, cuando inicio sesión, entonces accedo a la aplicación.
- Dado que no he confirmado mi registro, cuando inicio sesión, el sistema no me lo permite.
- Dado que introduzco credenciales incorrectas, cuando intento iniciar sesión, entonces el sistema muestra un mensaje de error.
- Dado que no estoy autenticado, cuando intento acceder a una funcionalidad protegida, el sistema no me lo permite.

■ HU-02

- Dado que inicio sesión, cuando pasa el tiempo de inactividad configurado, entonces la sesión expira.
- Dado que la sesión expira, cuando intento continuar, entonces debo volver a autenticarme.

■ HU-03

- Dado que un usuario tiene un rol asignado, cuando accede al sistema, entonces solo puede realizar lo permitido.

■ HU-04

- Dado que intento realizar una acción no permitida en base a mi rol, entonces el sistema lo bloquea.

■ HU-05

- Un usuario con rol de administrador puede crear una entidad habilitada en el sistema.

- El sistema impedirá que un usuario con otro rol quiera registrar una entidad.
 - Los campos de nombre, NIF y correo electrónico son obligatorios; sin ellos no se podrá crear la entidad.
- **HU-06**
- El sistema no permite que un usuario con rol administrador dentro de una entidad pueda manipular datos de usuarios que no le han dado permiso.
 - El sistema no permite que un usuario pueda gestionar datos o usuarios de otras entidades a las que no pertenece.
 - Un usuario que pertenece a una entidad con rol administrador podrá gestionar las actividades de otro usuario que haya otorgado permisos dentro de la misma entidad.
- **HU-07**
- Dado que introduzco los datos válidos de una parcela, cuando la guardo, entonces la parcela queda registrada en el sistema.
 - Dado que la parcela tiene recintos, cuando se guarda la parcela, entonces los recintos quedan almacenados y vinculados a la parcela.
 - Dado que faltan datos obligatorios de la parcela, cuando intento guardarla, entonces el sistema muestra un error indicando los campos inválidos.
 - Dado que existen recintos con datos inválidos o incompletos, cuando intento guardar la parcela, entonces el sistema muestra un error.
- **HU-08**
- Dado que introduzco datos válidos de una actividad, cuando la guardo, entonces la actividad queda registrada en el sistema.
 - Dado que la actividad tiene estado “Completada”, cuando introduzco una fecha futura, entonces el sistema muestra un error.
 - Dado que falta alguno de los campos obligatorios (fecha, campaña, parcela/recinto o tipo de actividad), cuando intento guardar, entonces el sistema muestra un error indicando los campos inválidos.
 - Dado que la parcela o recinto no existe, cuando intento registrar la actividad, entonces el sistema muestra un error.
 - Dado que el usuario es admin dentro de una entidad, cuando registra una actividad para otro usuario de su entidad con permisos, entonces la actividad se guarda correctamente.
 - Dado que el usuario es “agricultor” o “administrador” y no pertenecen a ninguna entidad con rol “agricultor”, cuando registra una actividad, entonces solo puede hacerlo para sí mismo.

■ HU-09

- Dado que selecciono el tipo de actividad “fitosanitaria”, cuando accedo al formulario de registro, entonces el sistema solicita los campos necesarios como dosis, NIF del aplicador, fecha y tipo de cultivo.
- Dado que introduzco todos los datos válidos de un tratamiento fitosanitario, cuando guardo la actividad, entonces el tratamiento queda registrado en el sistema y asociado a la actividad.
- Dado que falta alguno de los campos obligatorios cuando intento guardar, entonces el sistema muestra un error indicando los campos inválidos.
- Dado que registro un tratamiento fitosanitario válido, cuando se guarda la actividad, entonces se almacena en la tabla de actividades y en la tabla específica de tratamientos.

■ HU-10

- Dado que introduzco los datos válidos de una actividad con fecha futura, cuando la guardo, entonces la actividad queda registrada en el sistema como “Planificada”.
- Dado que intento registrar una actividad planificada, cuando introduzco una fecha pasada, entonces el sistema muestra un error.
- Dado que no se ha insertado una campaña, cuando intento guardar la actividad, entonces el sistema muestra un error.

■ HU-11

- Dado que existe una actividad registrada, cuando modifico sus datos con valores válidos y guardo, entonces la actividad se actualiza correctamente en el sistema.
- Dado que falta algún campo obligatorio tras la modificación, cuando intento guardar los cambios, entonces el sistema muestra un error.
- Dado que se modifica el estado de una actividad “completada” a “planificada”, el sistema muestra un error.

■ HU-12

- Dado que elimino una actividad, si esta existe, entonces desaparece.
- Dado que la actividad contenía datos en tablas específicas, entonces los datos asociados se eliminan o recalculan.

■ HU-13

- Dado que el usuario tiene rol “administrador”, cuando accede al listado de usuarios, entonces el sistema muestra todos los usuarios registrados.
- Dado que el usuario no tiene rol “administrador”, cuando intenta acceder al listado de usuarios, entonces el sistema deniega el acceso.

- Dado que accedo al listado de usuarios, cuando se muestran los resultados, entonces veo los datos básicos de cada usuario (nombre, email, rol y estado).
- **HU-14**
 - Dado que el usuario existe en el sistema, puede acceder a la información de sus parcelas.
 - Dado que el usuario tiene la parcela asignada, entonces se muestra la información de ella.
- **HU-15**
 - Dado que el historial de actividades es de un usuario distinto del que las quiere obtener, en caso de no tener rol de administrador dentro de la misma entidad el sistema lo impedirá.
- **HU-16**
 - Dado que el usuario existe, el sistema obtiene las parcelas asignadas a el y las dibuja sobre un mapa.
- **HU-17**
 - Dado que he iniciado sesión en el sistema, cuando cierro sesión, los tokens que me permiten el acceso son eliminados y no puedo acceder al sistema.
 - Dado que la sesión ha sido cerrada, hay que iniciar sesión de nuevo para poder acceder al sistema.
- **HU-18**
 - Dado que la parcela y el recinto aparece dibujado en el mapa, cuando el usuario clicka en ella, aparece un enlace al formulario de registro.
 - Dado que se registra correctamente la actividad, queda almacenada en las tablas del sistema.

5.7. Requisitos funcionales

Los requisitos funcionales sirven para indicar qué funciones tiene el sistema y qué debe hacer. A partir de las historias de usuario se han extraído los siguientes requisitos funcionales agrupados por áreas. La estructura es del tipo *RF-[ID de HU].XX*.

5.7.1. Autenticación y autorización

- **RF-01.1)** El sistema debe permitir autenticación mediante usuario/email y contraseña.
- **RF-01.2)** El sistema debe validar las credenciales contra la base de datos.

- **RF-01.3)** El sistema debe iniciar una sesión de usuario tras autenticación correcta.
- **RF-01.4)** El sistema debe impedir el acceso a usuarios no autenticados.
- **RF-02.1)** El sistema debe gestionar expiración de sesiones o tokens.
- **RF-02.2)** El sistema debe invalidar sesiones antiguas automáticamente.
- **RF-02.3)** El sistema debe proteger endpoints con autenticación obligatoria.
- **RF-03.1)** El sistema debe soportar al menos los roles: administrador, entidad habilitada (cooperativa), agricultor.
- **RF-03.2)** El sistema debe evaluar permisos en función del rol del usuario.
- **RF-03.3)** El sistema debe ocultar o deshabilitar funcionalidades no autorizadas.
- **RF-04.1)** El sistema debe validar permisos antes de ejecutar cualquier acción.
- **RF-04.2)** El sistema debe impedir operaciones no autorizadas a nivel de backend.
- **RF-17.1)** El sistema debe eliminar los tokens guardados del usuario para evitar futuras peticiones.
- **RF-17.2)** El sistema debe mostrar la página de inicio de sesión en caso de haber cerrado la sesión.

5.7.2. Entidades habilitadas

- **RF-05.1)** El sistema debe permitir registrar una entidad habilitada en el sistema si los campos son correctos.
- **RF-05.2)** El sistema debe impedir el registro de una entidad a un usuario con rol distinto de administrador.
- **RF-06.1)** Se crea una relación entre el usuario y la entidad habilitada con un rol y los permisos que cede a la entidad.
- **RF-06.2)** Los roles posibles dentro de una entidad son: agricultor (farmer) y admin (administrador).
- **RF-06.3)** Las actividades pueden ser manipuladas por el usuario que tenga el rol admin dentro de la entidad y sobre los usuarios que pertenezcan a esa entidad.
- **RF-06.4)** Si usuario agricultor es vinculado con una entidad, cede directamente todos sus permisos a la entidad.

5.7.3. Parcelas y recintos

- **RF-07.1)** El sistema debe permitir al usuario registrar parcelas.
- **RF-07.2)** El sistema debe validar que los campos obligatorios de la parcela sean correctos antes de su creación.
- **RF-07.3)** El sistema debe mostrar un error si la validación de la parcela falla.
- **RF-07.4)** El sistema debe consumir la API de SIGPAC para obtener los datos de parcelas y recintos (códigos y geometrías/polígonos).
- **RF-07.5)** El sistema debe almacenar la parcela en la base de datos.
- **RF-07.6)** El sistema debe almacenar los recintos asociados a una parcela, en caso de que existan.
- **RF-07.7)** El sistema debe vincular los recintos con su parcela correspondiente.
- **RF-14.1)** El sistema debe devolver la información de una parcela vinculada a un usuario específico.
- **RF-16.1)** El sistema debe devolver una lista con la información de una parcela junto con la geometría para el propio usuario.
- **RF-16.2)** El sistema debe validar que el usuario existe.
- **RF-18.1)** El sistema debe mostrar un enlace que lleve al formulario de crear una actividad al clickar sobre un recinto del mapa.

5.7.4. Actividades

- **RF-08.1)** El sistema debe permitir registrar actividades.
- **RF-08.2)** El sistema debe permitir registrar actividades con estado “Planificada” o “Completada”.
- **RF-08.3)** El sistema debe impedir registrar actividades con fecha futura si el estado es “Realizada”.
- **RF-08.4)** El sistema debe validar que la parcela o recinto asociado exista.
- **RF-08.5)** El sistema debe validar que la actividad esté asociada a una campaña.
- **RF-08.6)** El sistema debe validar que el tipo de actividad pertenezca al catálogo permitido.
- **RF-08.7)** El sistema debe validar que los campos obligatorios (fecha, parcela/recinto, campaña, tipo de actividad) estén presentes.
- **RF-08.8)** El sistema debe impedir guardar la actividad si falla alguna validación.
- **RF-08.9)** El sistema debe almacenar la actividad en el sistema actualizando automáticamente las tablas específicas según el tipo de actividad.

- **RF-08.10)** El sistema debe gestionar permisos según el rol del usuario:
 - **RF-08.10.1)** Un usuario que pertenece a una entidad con rol “admin de entidad” puede registrar actividades para sí mismo o para otros usuarios de su entidad con permisos.
 - **RF-08.10.2)** Un usuario que pertenece a una entidad con rol “agricultor” solo puede registrar actividades para sí mismo, salvo que pertenezca a una entidad con permisos delegados, en cuyo caso esta función la hará un admin de la entidad.
 - **RF-08.10.3)** Un usuario con rol de “administrador” podrá registrar actividades para sí mismo.
- **RF-09.1)** El sistema debe permitir registrar actividades de tipo “Fitosanitaria”.
- **RF-09.2)** El sistema debe solicitar los campos específicos: producto, cantidad, nif del usuario que lo realiza y tipo cultivo.
- **RF-09.3)** El sistema debe impedir guardar la actividad si falta alguno de los campos obligatorios.
- **RF-09.4)** El sistema debe mostrar un mensaje de error cuando la validación falle.
- **RF-09.5)** El sistema debe registrar la actividad en la tabla general de actividades.
- **RF-09.6)** El sistema debe registrar el tratamiento en la tabla específica de tratamientos.
- **RF-09.7)** El sistema debe vincular el tratamiento con la actividad correspondiente.
- **RF-10.1)** El sistema debe permitir registrar actividades en estado “Planificada”.
- **RF-10.2)** El sistema debe validar que la fecha de una actividad planificada es futura.
- **RF-10.3)** El sistema debe validar que la actividad esté asociada a una campaña.
- **RF-10.4)** El sistema debe validar que los campos obligatorios (fecha, campaña, parcela/recinto, tipo de actividad) estén presentes.
- **RF-10.5)** El sistema debe almacenar la actividad planificada en el sistema.
- **RF-11.1)** El sistema debe permitir modificar actividades existentes.
- **RF-11.2)** El sistema debe validar que la actividad exista antes de permitir su modificación.
- **RF-11.3)** El sistema debe impedir guardar cambios si alguna validación falla.
- **RF-11.4)** El sistema debe actualizar los datos de la actividad al guardar los cambios.

- **RF-11.5)** El sistema debe impedir establecer el estado “Completada” con una fecha futura.
- **RF-11.6)** El sistema debe actualizar las tablas específicas cuando se modifiquen datos relevantes del tipo de actividad.
- **RF-11.7)** El sistema debe mantener la consistencia entre la tabla de actividades y las tablas específicas.
- **RF-12.1)** El sistema tiene que validar que la actividad existe.
- **RF-12.2)** El sistema debe permitir eliminar una actividad dado su id.
- **RF-12.3)** El sistema debe eliminar los registros de la tabla de actividades y actividades fitosanitarias relacionadas.

5.7.5. Usuarios

- **RF-13.1)** El sistema debe impedir el acceso a los usuarios que no tenga rol administrador.
- **RF-13.2)** El sistema debe mostrar una lista de los usuarios registrados con la información más relevante.
- **RF-15.1)** El sistema debe mostrar las actividades realizadas por un usuario dado su id.
- **RF-15.2)** El sistema debe validar que el usuario exista.
- **RF-15.3)** El sistema mostrará las actividades propias de un usuario o las de otros usuarios en caso de que estén asignadas a una entidad y el usuario tenga rol de admin dentro de la entidad.

5.8. Requisitos no funcionales

Este tipo de requisitos se basan en cómo debe comportarse el sistema en cuanto a rendimiento, seguridad, usabilidad, escalabilidad, etc. Los requisitos no funcionales planteados son los siguientes:

5.8.1. Usabilidad

- **RNF1.1)** Interfaz de usuario fácil de manejar, con accesos rápidos a las funciones más importantes y con una navegación justa.
- **RNF1.2)** El sistema tiene que ser compatible con cualquier dispositivo móvil, independientemente del sistema operativo utilizado (Android o iOS).

5.8.2. Seguridad

- **RNF2.1)** El sistema tiene un control de autenticación a través de JWT. Gracias a un token que obtiene el usuario al iniciar sesión, puede realizar peticiones al servidor.
- **RNF2.2)** El sistema controla la autorización y permisos que tiene el usuario sobre las distintas funcionalidades de la aplicación. Para ello se ha recurrido un sistema basado en roles tanto a nivel de aplicación global como a nivel de rol dentro de una entidad.
- **RNF2.3)** El sistema permite cifrar información y datos sensibles de los usuarios como la contraseña por motivos de seguridad.

5.8.3. Mantenibilidad

- **RNF3.1)** El sistema debe estar diseñado de forma que pueda facilitar la escalabilidad añadiendo nuevas funcionalidades, su mantenimiento y actualización de los datos.
- **RNF3.2)** El sistema debe estar documentado de forma adecuada y suficiente, de manera que pueda permitir a otros usuarios entender el código.
- **RNF3.3)** El sistema está sometido a pruebas y test (unitarios, integración, etc), que permiten validar su correcto funcionamiento.

5.8.4. Disponibilidad y Rendimiento

- **RNF4.1)** El sistema debe estar disponible de forma continua para permitir a los usuarios consultar y registrar información en cualquier momento.
- **RNF4.2)** Si se produce un fallo de conexión o una incidencia en el servidor, el sistema debe informar al usuario mediante mensajes claros y comprensibles.
- **RNF4.3)** Las operaciones más frecuentes, como el inicio de sesión, la carga de parcelas o el guardado de actividades, deben ejecutarse en un tiempo reducido.
- **RNF4.4)** El sistema debe poder crecer en volumen de usuarios y datos sin que su rendimiento se vea afectado de forma notable.

6. Diseño y arquitectura

Este capítulo va a abordar cómo se ha diseñado el sistema desde una perspectiva más abstracta, sin entrar en el código y las implementaciones. Para ello, se van a tratar distintas partes como la **arquitectura** desarrollada, los **modelos de datos** utilizados, diseño de la **interfaz** (cliente) y diseño de la **API** y **servicios** (servidor).

6.1. Arquitectura de alto nivel

En esta sección se va a explicar cómo es la arquitectura del sistema, de qué componentes consta y cómo se comunican entre ellos.

6.1.1. Arquitectura monolítica

Para el diseño del sistema se ha elegido una **arquitectura monolítica** por la siguientes razones:

- **Alto acoplamiento** entre las distintas entidades (actividades, parcelas, recintos, usuarios), causando que la lógica de negocio y validaciones sean más sencillas de gestionar estando centralizadas en un único punto, ayudando a reducir posibles errores de coherencia de datos.
- **Simplicidad de desarrollo** ya que todos los componentes están en el mismo servidor, permitiendo un único despliegue más directo y evitando la necesidad de gestionar comunicación entre servicios mediante eventos.
- Las **transacciones** sobre la base de datos son más rápidas y sencillas de gestionar, al igual que la **evolución** del sistema, ya que permite modificar funcionalidades de forma más ágil sin romper contratos entre los servicios.

Elegir otro tipo de arquitectura como una basada en microservicios, implicaría dividir servicios que están altamente relacionados, introduciendo una “frontera” entre ellos. Esto provocaría posibles fallos distribuidos, llamadas entre servicios a través de APIs, autenticación entre servicios, despliegues independientes y una mayor complejidad al realizar operaciones entre entidades.

Por ejemplo, al crear una actividad es necesario tener en cuenta su relación con un recinto, la parcela a la pertenece dicho recinto, el usuario que la realiza y la entidad a la que pertenece (en caso de existir), además de aplicar validaciones según los roles del usuario. Esta situación, en una arquitectura basada en microservicios, sería mucho más costosa de gestionar, mientras que en un monolito resulta más sencilla al disponer de acceso directo a todos los componentes del sistema.

Además de esto, se ha elegido un **modelo cliente - servidor**. Gracias a este modelo se consigue garantizar las siguientes propiedades:

- **Desacoplamiento**, permitiendo que la parte del cliente esté centrada exclusivamente en la interfaz y la experiencia de usuario, mientras que el servidor gestiona la lógica de negocio y la persistencia de datos.
- **Soporte multiplataforma**, permitiendo que distintos tipos de clientes (móvil o web) puedan servir de forma simultánea, facilitando la expansión del ecosistema Olivaris.
- **Eficiencia de red**, permitiendo intercambiar la información de forma organizada a través de peticiones puntuales, optimizando el uso de recursos.

6.1.2. Componentes principales

Los componentes principales de la arquitectura son:

■ Servidor

Es el encargado de procesar los datos y donde reside la lógica de negocio. Para la organización del código se ha seguido el patrón de diseño **MVC (Modelo-Vista-Controlador)**, con el objetivo de separar responsabilidades y mejorar la mantenibilidad. Se ha definido una estructura jerárquica basada en las siguientes capas:

- **Controlador**: capa encargada de exponer los puntos de acceso (URLs) que permiten a los sistemas externos comunicarse con el servidor. Define la interfaz de la API y gestiona las peticiones entrantes.
- **Lógica de negocio**: capa donde se procesan los datos antes de ser almacenados o enviados a sistemas externos. Aquí se aplican las reglas y validaciones del sistema.
- **Persistencia**: capa responsable de la comunicación directa con la base de datos, encargándose de ejecutar las operaciones SQL necesarias.

El servidor es de tipo RESTful, lo que significa que expone recursos manipulables a través de métodos HTTP. Por ello, la “Vista” del patrón MVC queda delegada casi por completo en el lado del cliente. La única excepción es el envío de correos electrónicos de registro, donde el servidor sí genera el archivo HTML que recibe el usuario.

■ Cliente

Es el componente que define y contiene toda la **interfaz de usuario**. Su código también está organizado en capas para separar las distintas responsabilidades: la definición del cliente de la API, los componentes visuales, la lógica de datos de cada vista y la navegación entre pantallas.

■ Base de datos

Es el sistema encargado de almacenar toda la información de la aplicación.

Su función es suministrar datos al cliente cuando se solicitan y persistir los resultados de las operaciones procesadas por el servidor.

6.1.3. Comunicación entre los componentes

La comunicación entre el servidor y el cliente sigue un modelo **Cliente-Servidor**. Este enfoque permite distribuir e intercambiar información a través de la red de forma eficiente: el cliente inicia la interacción mediante una petición a la API del servidor; este, a su vez, recibe los datos, los procesa y se comunica con la base de datos para consultarlos o almacenarlos. Gracias a esta estructura, se consigue una mayor escalabilidad y una mejor organización de la información.

Por otro lado, la comunicación entre el servidor y la base de datos se realiza mediante un **ORM (Object-Relational Mapping)**, una herramienta estándar en el desarrollo actual. En este caso, se han utilizado JPA e Hibernate integrados con Spring Boot. Estas herramientas permiten gestionar las entidades, sus relaciones y las operaciones SQL de forma automática y transparente, lo que agiliza el desarrollo, reduce errores y optimiza el trabajo con la base de datos.

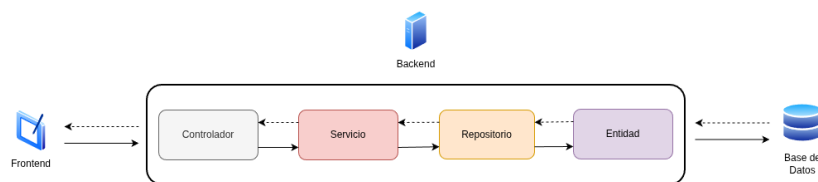


Figura 6.1: Diagrama de comunicación entre componentes

Esta separación clara entre componentes garantiza la total independencia entre ellos. El servidor no necesita conocer los detalles de implementación del frontend; simplemente expone la manipulación de recursos a cualquier cliente, sin importar su lenguaje de programación o plataforma utilizada (web, móvil o aplicaciones de terceros).

A través de este enfoque se consiguen ventajas significativas en el sistema como los siguientes:

- **Reusabilidad y flexibilidad:** permite que diferentes equipos trabajen en el cliente y el servidor de forma simultánea, agilizando el ciclo de desarrollo.
- **Arquitectura sin estado (stateless):** el servidor no almacena información de la sesión del usuario entre peticiones. Cada consulta contiene toda la información necesaria para ser procesada, lo que facilita el escalado y mejora la fiabilidad.
- **Intercambio de datos:** la comunicación se realiza de forma ligera y estandarizada mediante objetos **JSON**, que es el formato de intercambio de datos más extendido.

Autenticación y autorización

Para garantizar un intercambio seguro de información entre los componentes a través de la API, se ha implementado un mecanismo basado en **JSON Web Token (JWT)**. Esta tecnología se ha seleccionado por ser la más adecuada para arquitecturas desacopladas y aplicaciones móviles; al tratarse de un mecanismo *stateless*, el token transporta en su interior toda la información necesaria y está firmado digitalmente, lo que elimina la necesidad de almacenar sesiones activas en el servidor para validar al usuario. Al decodificar el token y verificar su firma, el sistema puede identificar de forma correctamente al usuario en cada petición, protegiendo los recursos de la API contra accesos no autorizados.

Más allá de la identidad, el token es la pieza fundamental para la autorización y gestión de permisos mediante un control de acceso basado en roles. Gracias a la información contenida en el JWT, el backend puede determinar el perfil del usuario (administrador, agricultor o entidad habilitada) y restringir el acceso a funcionalidades específicas. Este control permite modelar con precisión la lógica de negocio de Olivaris: un agricultor puede gestionar exclusivamente sus propias explotaciones, mientras que un administrador de una entidad habilitada, tras recibir la delegación de permisos correspondiente, puede actuar en nombre de terceros y supervisar múltiples explotaciones, garantizando así la trazabilidad y el cumplimiento normativo del Cuaderno Digital de Explotación.

6.2. Modelos de datos

El modelo de datos se representará principalmente mediante dos esquemas: el **esquema de base de datos** y el **diagrama de clases**. El primero es una representación gráfica de las entidades y sus relaciones dentro de la base de datos; el segundo ofrece una visión de las clases del sistema, detallando sus atributos, métodos y las conexiones entre ellas.

6.2.1. Esquema de Base de Datos

El esquema de bases de datos del sistema es el siguiente:

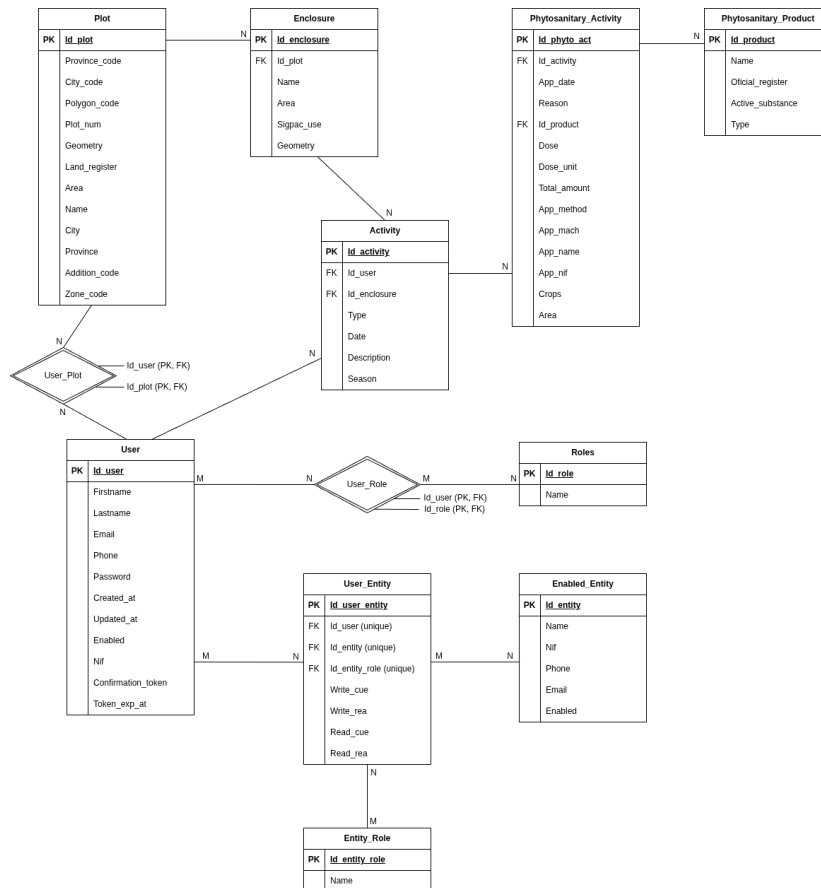


Figura 6.2: Diagrama E/R de la base de datos

- **Usuario:** es la entidad que representa un usuario en el sistema. Cuenta con los siguientes campos:

- Id_user (bigint)
- Firstname (varchar)
- Lastname (varchar)
- Email (varchar)
- Phone (varchar)
- Password (varchar)
- Created_at (timestamp)
- Updated_at (timestamp)
- Enabled (boolean)
- Nif (varchar)
- Confirmation_token (varchar)
- Token_exp_at (timestamp)

- **Roles del sistema:** entidad que representa los roles que un usuario tiene en el sistema. Se pueden distinguir dos tipos: *rol admin*, que es el usuario que tiene un control total sobre la aplicación, pudiendo gestionar todos los usuarios que pertenecen al sistema; y *rol básico*, que es el rol que adquiere cualquier usuario que se registra en el sistema. La tabla tiene los siguientes campos:
 - Id_role (bigint)
 - Name (varchar)
 - **Usuario - Rol:** relación entre un usuario y los roles que tiene en el sistema. En cuanto al diseño, se ha considerado que un usuario puede tener varios roles, formando una jerarquía: el usuario admin tiene rol admin y rol básico. Los campos son los siguientes:
 - Id_user (bigint)
 - Id_role (bigint)
 - **Entidad habilitada:** entidad que representa a una entidad habilitada (por ejemplo: una cooperativa), siendo la que tiene una relación directa con el agricultor y que podrá utilizar el CUE comercial (sistema desarrollado). Se ha fijado este nombre de acuerdo al *Anexo X para el Registro de Autorizaciones para el Uso de Aplicaciones Comerciales*. Los campos que tiene la tabla son los siguientes:
 - Id_entity (bigint)
 - Name (varchar)
 - Nif (varchar)
 - Phone (varchar)
 - Email (varchar)
 - Enabled (boolean)
 - **Rol de la entidad:** entidad que representa el rol que tiene un usuario vinculado a una entidad habilitada. El rol puede ser: *admin*, que corresponde con el usuario administrador de una entidad, siendo el encargado de poder gestionar los datos de los agricultores que pertenecen a la misma entidad, y *farmer* (agricultor), siendo el usuario vinculado con una entidad que delega permisos a los administradores para poder gestionar sus datos (leer y crear actividades). Los campos que tiene son los siguientes:
 - Id_entity_role (bigint)
 - Name (varchar)
 - **Usuario - Entidad - Rol entidad:** relación entre la tabla usuario, entidad y rol de la entidad. Cuando un usuario es vinculado con una entidad, se le añade un rol dentro de la entidad, siendo esta relación la que almacena la información.
- Los permisos que un usuario da a la entidad para manipular los datos es representado a través de los campos *write* y *read* (cue y rea). Si un usuario es

vinculado a la entidad con el rol admin, todos los permisos están a “null”, ya que este usuario es el encargado de gestionar datos a los agricultores; mientras que si el usuario es vinculado con rol farmer, todos los permisos serán puestos a “true”, de manera que los admin dentro de la entidad ya saben si el usuario ha dado permisos o no. Aunque ahora mismos todos los campos tengan el valor “true” si el usuario es farmer, este diseño permite dar solo ciertos tipos de permisos a la entidad si el sistema crece en un futuro o si hay que modificar el tipo de permisos.

Los tipos de **permisos** se han creado siguiendo el *Anexo X* para la elaboración de un CUE comercial. Este anexo explica que un usuario titular de una explotación podrá grabar sus datos en el CUE comercial, lo que equivale a gestionar sus propios datos relativos a una actividad dentro del sistema, o delegar esta función a una entidad habilitada, siendo esta la encargada de gestionar sus actividades, utilizando el CUE comercial (sistema actual) para almacenar los datos.

La tabla tiene los siguientes campos:

- Id_user_entity (bigint)
 - Id_user (bigint)
 - Id_entity (bigint)
 - Id_entity_role (bigint)
 - Write_cue (boolean)
 - Write_rea (boolean)
 - Read_cue (boolean)
 - Read_rea (boolean)
- **Parcela:** entidad que representa una parcela del sistema. Está compuesta por todos los datos necesarios que la representan así como la información proporcionada por el **Visor SIGPAC** [39], como son: código de provincia y municipio (donde se ubica la parcela), código de polígono, número de parcela, geometría que representa la parcela en el mapa, entre otros.

El Visor SIGPAC de Andalucía es una aplicación web que permite obtener información gráfica y alfanumérica de las parcelas agrícolas de Andalucía. El Visor ofrece información , a nivel de recinto, del año en curso y del anterior. Entre la datos que ofrece destacan: uso del recinto, superficie del recinto, pendiente media, coeficiente de regadío y región (entre otras).

Los campos de la tabla son los siguientes:

- Id_plot (bigint)
- Province_code (varchar)
- City_code (varchar)
- Polygon_code (varchar)

- Plot_num (varchar)
 - Geometry (geometry(polygon))
 - Land_register (varchar)
 - Area (double)
 - Name (varchar)
 - City (varchar)
 - Province (varchar)
 - Addition_code (varchar)
 - Zone_code (varchar)
- **Recinto:** entidad que representa un recinto dentro de la parcela. Este recinto está relacionado con una parcela a través de una clave foránea y además, almacena la geometría del polígono que representa el recinto así como su uso.

Para almacenar los datos del recinto y de la parcela que no dependen del usuario (no dados por el), se ha recurrido a utilizar la **API** [40] que ofrece el servicio de **SIGPAC**. Los endpoints utilizados y su uso será explicado en el último apartado de esta sección.

La tabla tiene con los siguientes campos:

- Id_enclosure (bigint)
 - Id_plot (bigint)
 - Name (varchar)
 - Area (double)
 - Sigpac_use (varchar)
 - Geometry (geometry(polygon))
- **Usuario - Parcela:** entidad que relaciona un usuario con una parcela. A través de esta relación se puede saber qué usuario se ha asignado una parcela para poder posteriormente mostrarla en un mapa. El usuario puede tener múltiples parcelas y una parcela está relacionada con múltiples usuarios, ya que la API es pública y todas las parcelas, pertenezcan en la realidad a un usuario o no, pueden ser accedidas por cualquier usuario. En un futuro, se tendría que utilizar otro sistema que permita validar si la parcela que el usuario se ha asignado es realmente suya.

Esta relación es una entidad como tal ya que, además de tener las claves foráneas, cuenta con un atributo propio de la relación que es el nombre que el usuario le pone a la parcela.

Los campos que tiene la tabla son los siguientes:

- Id_user (bigint)

- Id_plot (bigint)
- **Actividad:** entidad que representa una actividad realizada por un usuario. Esta tabla relaciona un usuario con un recinto a través de sus ids como claves foráneas, consiguiendo de esta manera saber qué usuario ha realizado una actividad en un recinto dentro de una parcela. Además de estos datos, también se almacena el tipo de actividad realizada con un dato de tipo *enum*. Este diseño permite que la base de datos pueda escalar, de manera que en esta tabla se almacenen los datos que identifican a una actividad en general y en otra tabla, se almacenarán los datos específicos del tipo de actividad que se realiza (fitosanitaria, riego, poda, etc.), pudiendo tener la información sin repetir y mejor gestionada.

La tabla cuenta con los siguientes campos:

- Id_activity (bigint)
- Id_user (bigint)
- Id_enclosure (bigint)
- Type (varchar)
- Date (date)
- Description (varchar)
- Season (varchar)
- **Actividad fitosanitaria:** entidad que representa una actividad del tipo fitosanitaria. La tabla almacena datos específicos para este tipo de actividad como son: fecha en la que se aplica, producto utilizado, dosis y cantidad, tipo de cultivo y persona que aplica el producto (entre otros). Esta tabla está totalmente relacionada con la tabla actividad, ya que en la realidad es vista la información de ambas tablas como un “todo”, pero en el diseño la información está separada.

La tabla también recoge información de un producto fitosanitario a través de una clave foránea. Esta información se almacena en una tabla a parte, de manera que se evita duplicar información y el sistema escala mejor.

Entre los campos que tiene la tabla se encuentran:

- Id_phyto_act (bigint)
- Id_activity (bigint)
- Id_product (bigint)
- App_date (date)
- Reason (varchar)
- Dose (double)
- Dose_unit (varchar)

- Total_amount (double)
 - App_method (varchar)
 - App_match (varchar)
 - App_name (varchar)
 - App_nif (varchar)
 - Crops (varchar)
 - Area (double)
- **Producto fitosanitario:** entidad que representa un producto fitosanitario en el sistema. Almacena información como el registro oficial, siendo un identificador del producto, además del nombre o tipo de producto.

Los campos que tiene la tabla son los siguientes:

- Id_product (bigint)
- Name (varchar)
- Oficial_register (varchar)
- Active_substance (varchar)
- Type (varchar)

6.2.2. Diagrama de clases

El diagrama de clases del sistema es el siguiente:

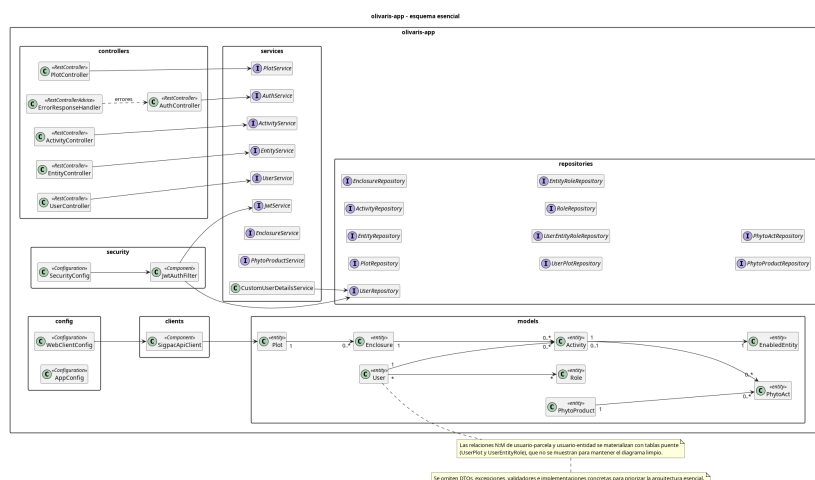


Figura 6.3: Diagrama de clases

6.3. Diseño de la interfaz

El diseño de la interfaz (frontend) ha sido realizado con **React Native** y el framework de desarrollo **Expo**. Esta elección se ha basado en la gran capacidad que tiene React Native para desarrollar aplicaciones multiplataforma de forma eficiente (tanto para Android como iOS), reduciendo tiempos de desarrollo y complejidad técnica. Además, utiliza JavaScript/TypeScript, que son lenguajes de programación altamente utilizados en el desarrollo web actual.

La razón por la que se ha elegido Expo es porque permite prototipar y realizar pruebas con simulaciones muy rápidas a través de su aplicación móvil *Expo Go*: con tan solo una lectura de un código QR se puede acceder a la aplicación, probarla y mostrar los cambios de manera instantánea. Además, Expo permite configurar de manera muy rápida el entorno e incluye una cantidad de APIs considerable con las que se pueden acceder a funcionalidades del móvil (como el GPS).

6.3.1. Decisiones de diseño

El diseño del frontend se ha centrado en ofrecer una experiencia intuitiva y minimalista. Se ha priorizado la claridad visual para que la navegación sea fluida, permitiendo que cualquier perfil de usuario pueda acceder a las distintas funcionalidades sin necesidad de una curva de aprendizaje previa. Al eliminar elementos innecesarios y reducir la carga de información en cada pantalla, se garantiza que el usuario mantenga el foco en las tareas principales, evitando confusiones o fatiga durante el uso de la aplicación.

En cuanto al logo que representa la aplicación es el siguiente:



Figura 6.4: Logo de la aplicación Olivaris

El logo ha sido diseñado teniendo en cuenta que el sector sobre el que se trabaja es el olivar y que el funcionamiento principal de la aplicación es el registro de actividades, de ahí el icono del cuaderno con distintos “checks” y el ramo con la oliva.

A través del logo, se ha obtenido una paleta de colores con tonalidades relacionadas con el verde oliva.

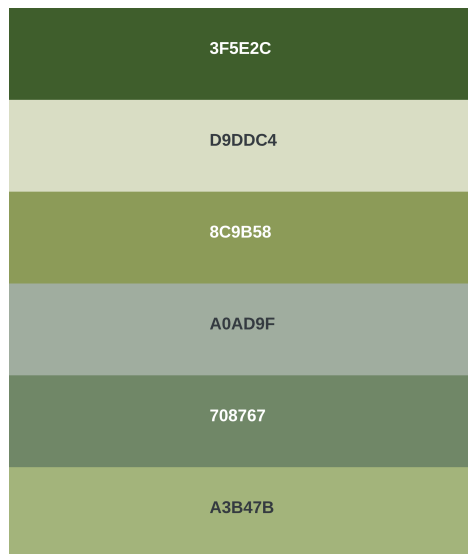


Figura 6.5: Paleta de colores

A continuación, se va a explicar la estructura de la aplicación y qué patrones se han seguido para el diseño y la experiencia de usuario.

Estructura de navegación y flujos de usuario

La navegación se realiza a partir de un componente creado que representa una barra de navegación, estando disponible en todas las pantallas de la aplicación. Se utiliza **expo-router** para gestionar la navegación mediante un *stack navigator*, proporcionando una estructura jerárquica del flujo de la aplicación, de manera que la nueva pestaña a la que se avanza se guarda en la “pila” y si se vuelve a la anterior, se obtendrá al sacar de la estructura la última pantalla almacenada.

La navegación está dividida en dos contextos bien diferenciados:

- **Pantallas públicas:** antes de la autenticación, el usuario solo tiene acceso a dos rutas: **pantalla de inicio/login (/)** y **registro de nuevo usuario (/registerUser)**. Estas pantallas no muestran la barra de navegación y están diseñadas para no permitir acceso más allá sin credenciales válidas.
- **Pantallas protegidas:** una vez autenticado, el usuario accede a un conjunto de rutas mediante validación de tokens. La aplicación implementa una función “guardián” de rutas que verifica la validez del token de acceso antes de permitir la navegación a cualquier pantalla protegida. Si el token ha expirado o no es válido, el usuario es automáticamente redirigido a la pantalla de login, garantizando de esta manera que nadie pueda acceder a funcionalidades sin estar autenticado.

Las pantallas principales son accesibles a través de una barra de navegación horizontal ubicada en la parte inferior de la interfaz, mostrando los iconos de: parcelas, actividades, entidades y usuarios del sistema (este último solo para administradores). Cada sección lleva a su pantalla correspondiente desde cualquier

punto de la aplicación, permitiendo la navegación rápida entre las funcionalidades más usadas.

Además de estas pantallas principales, existen pantallas de registro y modificación que aparecen de forma modal en el flujo: registro de parcelas, actividades, y asignación de usuarios a entidades. Estas pantallas se muestran sin la barra de navegación para evitar distracciones durante el proceso de entrada de datos.

En cuanto a los roles del sistema y autorización (acceso a funcionalidades), en la aplicación se distinguen cuatro perfiles principales de usuario:

- **Usuario básico (agricultor)**

Es el rol común que tienen la gran mayoría de usuarios. Con este rol el usuario puede realizar: consultar sus propias parcelas y crear nuevas, registrar actividades sobre sus parcelas, ver el historial de actividades y gestionar su información personal.

El flujo de un agricultor es simple: tras autenticarse, entra directamente al área de parcelas desde donde puede navegar a actividades y ver su perfil, sin exposición a funcionalidades administrativas.

- **Administrador del sistema:**

Este rol tiene acceso a todas las funcionalidades anteriores y además, puede gestionar y crear usuarios en el sistema con cualquier rol y controlar cualquier entidad habilitada.

- **Administrador de la entidad:**

Este rol lo puede tener un usuario vinculado a una entidad. Permite gestionar a los usuarios dentro de una entidad (añadiendo nuevos o eliminándolos), además de poder gestionar todas las actividades de cada uno de estos usuarios que pertenecen a la entidad.

- **Agricultor de la entidad:**

Este rol lo puede tener un usuario vinculado a una actividad. Permite al usuario visualizar todas las actividades realizadas pero no permite crear nuevas ni modificarlas; de esto se encargarán los administradores de la entidad.

Estos roles se verifican dinámicamente cuando el usuario accede a la aplicación, utilizando la información almacenada en el token y refrescada cuando es necesario. La navegación se adapta en tiempo real no exponiendo directamente aquellas funcionalidades que el usuario no puede usar. Por ejemplo, si un usuario es administrador, ve la opción adicional de “Usuarios del sistema” en la barra de navegación; si no lo es, esa opción no aparece.

Gestión del estado de la aplicación

En la aplicación no se ha planteado un estado global complejo. Los datos que se visualizan en cada pantalla así como los temporales, se gestionan de forma local dentro de cada pantalla mediante estados internos (con el *hook* `useState`) y efectos

de carga (con el *hook* `useEffect`), mientras que la información que se debe mantener entre pantallas o sesiones se guarda de forma persistente en el dispositivo (**SecureStore**). Este enfoque encaja con una aplicación móvil de alcance funcional claro, porque evita una capa de complejidad innecesaria y hace que cada pantalla se encargue de cargar solo la información que necesita.

La parte más importante del estado persistente es la sesión del usuario. Los tokens de acceso y *refresh* se almacenan de forma segura en el dispositivo, junto con información del usuario actual. Eso permite que la aplicación recuerde quién inició sesión, mantenga la sesión activa al volver a abrirla y pueda reconstruir el estado inicial sin obligar al usuario a autenticarse en cada acceso. Además, al inicio de la aplicación se comprueba si el token sigue siendo válido y, si no lo es, se intenta renovar la sesión antes de enviar al usuario a la pantalla de inicio. De esta forma, la experiencia resulta más fluida y se reduce la complejidad en el acceso.

A nivel de sincronización, la aplicación sincroniza con el backend los elementos que forman parte del funcionamiento real del sistema: autenticación, información del usuario, entidades, parcelas, actividades y listado de usuarios del sistema. En estas pantallas se hacen peticiones al servidor cuando son abiertas o cuando se realiza alguna operación de alta, edición o borrado, para asegurar que la información mostrada esté actualizada.

La elección de este enfoque responde sobre todo a dos motivos. El primero es el rendimiento: al no depender de una capa global pesada, la aplicación carga más rápido y solo pide al servidor los datos necesarios en cada momento. El segundo es la experiencia de uso: el usuario percibe una interfaz más estable, con menos esperas y con una sesión que se conserva entre aperturas. También ayuda a mantener el código más claro, porque cada pantalla gestiona su propio ciclo de carga sin acoplarse al resto de la aplicación. En conjunto, este modelo resulta suficiente para el tipo de aplicación desarrollada y encaja bien con una interfaz móvil en la que se prioriza la sencillez, la rapidez y la continuidad de sesión.

Patrones de interacción y experiencia de usuario

Para abordar este subapartado, se va a dividir en las distintas interacciones que el usuario puede tener con la aplicación:

- **Interacción con mapas:** la aplicación integra mapas interactivos mediante **Leaflet**, incrustado en un componente del tipo **WebView** nativo que permite visualizar parcelas y recintos como polígonos geográficos. El mapa se carga con la ubicación actual del usuario como punto de referencia central y muestra todos los polígonos asociados a sus parcelas en color azul. Los polígonos son clickables: al tocar uno de ellos, la aplicación envía un mensaje desde el **WebView** hacia **React Native** para identificar qué parcela o recinto fue seleccionado, permitiendo acciones posteriores como ver detalles o editar información a través de un *pop up*.

El sistema también implementa un caché simple de ubicación: la primera vez que se accede a la pantalla de parcelas, se solicita la posición al dispositivo;

en caso de que haya una ubicación guardada recientemente, se reutiliza para acelerar la carga. Si la pantalla se abre nuevamente, ya no hay que esperar a que el GPS se posicione.

- **Interacción con formularios:** los formularios de registro están diseñados para validar en tiempo real mientras el usuario escribe. Cada campo tiene un estado de error asociado que se actualiza en el momento en que el usuario deja de escribir o toca otro campo. Por ejemplo, en el registro de parcelas, el campo de nombre muestra un error si está vacío, o en el registro de actividades, donde el campo de fecha solo se considera válido si cumple el formato DD-MM-YYYY.

Antes de enviar el formulario al servidor, la aplicación verifica a través de funciones de validación que todos los campos tengan el valor adecuado. Estas funciones se aplican en el momento en el que se pulsa el botón de “Guardar”. Si algún campo sigue siendo inválido, muestra una alerta pidiendo que se revisen los campos sin enviar nada al backend. Con este enfoque se reducen el número de peticiones al servidor y proporciona una retroalimentación inmediata sin saturar la interfaz con mensajes de error. Además de esto, muchos formularios incluyen modales para seleccionar múltiples opciones predefinidas en lugar de introducir texto, como por ejemplo, al seleccionar el estado de una actividad.

- **Interacción con listados:** los listados son mostrados a través de tarjetas individuales con la información estructurada: título destacado, secciones con etiqueta gris y el valor asociado a cada sección. Cada tarjeta puede incluir acciones secundarias como botones de editar o eliminar tanto en la esquina como en la parte inferior. Los listados están ordenados siguiendo un criterio coherente: las actividades aparecen de más reciente a más antigua, reflejando de mejor forma el flujo de trabajo del agricultor.

Cuando se navega hacia una pantalla que tiene un listado, se hace una petición al servidor y se obtienen todos los datos que van a ser mostrados en la tarjeta. En caso de que el usuario navega a otra pantalla y luego vuelva a la misma, la información se recarga automáticamente gracias al uso del *hook* `useEffect`, que realiza la petición de nuevo al backend para obtener la información antes de renderizar el componente. Esto asegura que siempre se van a ver los datos actualizados sin obligar al usuario a pulsar un botón de “refrescar”.

- **Retroalimentación visual:** la retroalimentación visual toma varias formas. Durante la carga de datos, se muestra el indicador clásico de tipo *spinner* junto con un texto, comunicando al usuario que la aplicación está trabajando. Si ocurre algún error durante la ejecución, se mostrará a través de alertas nativas del sistema con un título y un mensaje descriptivo, proporcionando contexto sobre qué salió mal sin detener la navegación de la aplicación. Este método se utiliza también para mostrar los mensajes de éxito, por ejemplo al eliminar un usuario.

Otros mensajes de error como los mostrados en los formularios, se marcan con texto rojo y se ubican debajo del campo, describiendo específicamente qué está mal (“la fecha es necesario”, “formato inválido”, etc).

Principios de diseño visual

El diseño visual de la aplicación ha sido planteado con un enfoque hacia dispositivos móviles y adaptable a distintas resoluciones. Debido a esta decisión, se han desarrollado estructuras verticales, contenedores de ancho completo, márgenes laterales moderados y bloques de contenido que se adaptan a la pantalla según el tamaño disponible.

En cuanto a la accesibilidad, se han priorizado decisiones como: tamaños de textos legibles, uso claro del color para diferenciar acciones y estados, y contraste suficiente entre fondo, superficie y contenido. Para los textos principales se han utilizado tonos oscuros sobre fondos claros, mientras que los mensajes secundarios aparecen en gris para mantener la jerarquía visual sin perder legibilidad. Los botones tienen un color distintivo y un tamaño suficientemente amplio para facilitar la pulsación, algo importante en la interacción táctil. También se ha respetado el espacio seguro de la pantalla a través del componente **SafeAreaView**, evitando que el contenido quede pegado a los bordes o se solape con zonas externas a la aplicación.

La coherencia visual entre pantallas se mantiene a través de un sistema común de colores, radios, sombras y estilos de botones. La aplicación reutiliza una paleta con colores simples: verde como color principal, fondos con colores suaves/claros, y tarjetas blancas para el contenido, ayudando de esta forma a que el usuario identifique rápidamente qué elementos forman parte de la misma interfaz. Las tarjetas, formularios y listados comparten una misma lógica visual: bordes redondeados, separación clara entre bloques y una jerarquía constante entre título, etiqueta y valor. Esto reduce el esfuerzo de aprendizaje, porque el usuario no tiene que reinterpretar cada pantalla desde cero.

También se buscó que la apariencia fuera consistente en componentes de distinta función. Un botón primario se comporta y se ve igual tanto en una pantalla de registro como en un listado; las tarjetas de información mantienen la misma estructura básica; y los formularios repiten el mismo criterio de alineación y espaciado. Esa continuidad visual da sensación de sistema unificado y ayuda a que la aplicación resulte más predecible y fácil de usar.

Comunicación con la API

Para la comunicación con la API desde el frontend, se ha realizado una abstracción del cliente HTTP, es decir, la aplicación encapsula las llamadas en una capa de cliente central llamada **apiClient**, delegando los endpoints concretos a distintos módulos de la API (por ejemplo: **plotApi** o **activityApi**). Cada módulo exporta funciones pequeñas y tipadas, conteniendo cada una el **apiClient**, el tipo de petición que se realiza (post, put, get o delete) y la ruta con sus parámetros requeridos. Esta separación mantiene la UI independiente de la URL de cada endpoint: los componentes consumen funciones directamente en lugar de construir peticiones HTTP manualmente. Gracias a ello es sencillo probar o sustituir la implementación del cliente sin tocar los componentes.

El manejo de errores en red o devueltos por la API se hace a través de estructuras *try/catch*: las llamadas se envuelven por capas de servicio y los componentes capturan los errores a través de esta estructura. En los *catch* se usa la función “getApiErrorMessage” para normalizar los errores en el frontend y devolver retroalimentación al usuario.

En cuanto a la autenticación e integración de tokens, se centraliza en el cliente HTTP: existe un interceptor en el *apiClient* que inyecta el token de acceso en el encabezado *Authorization* para todas las peticiones que son enviadas al backend, evitando duplicar la lógica en cada llamada. La renovación de sesión (*refresh*) se hace mediante una llamada específica disponible en los servicios de autenticación, y el bootstrap de la aplicación coordina la validación/renovación de credenciales antes de llevar al usuario a las pantallas privadas.

6.4. Diseño de la API

Para el desarrollo de este sistema se ha implementado una API que permite la comunicación entre un cliente (en este caso será una aplicación móvil) y el backend, proporcionando acceso seguro a la gestión de usuarios, actividades, parcelas y otras operaciones de negocio.

La API diseñada es de tipo **REST** [41] (explicado en el apartado de *Comunicación y Servicios Web* del estado del arte).

La decisión de implementar una API REST ha sido tomada debido a su simplicidad y facilidad de integrar con cualquier tipo de cliente, ya sea una página web con contenido .html o una aplicación móvil. La estructura que se ha seguido es del tipo **Cliente - Servidor**, ya que permite desacoplar y separar responsabilidades, teniendo en una parte el cliente, que muestra la interfaz de la aplicación al usuario y no sabe cómo trabaja el servidor con los datos; y por otra parte el servidor, encargándose de la lógica de negocio y de cómo se interactúa con la base de datos, sin necesidad de saber cómo es mostrada la información al usuario. Esto permite que ambos trabajen de manera independiente, cada uno con sus funciones, y que se puedan comunicar en cualquier momento aunque la lógica del cliente sea modificada.

Un aspecto muy importante de una API REST es que no guarda ningún estado en el servidor, siendo la comunicación *stateless*. Esto permite que cada petición que es realizada, contiene toda la información necesaria para ser procesada, permitiendo una mejora en la escalabilidad e independencia entre cada petición.

En cuanto al formato de las respuestas, pueden ser de varios tipos como se ha comentado anteriormente, pero en el desarrollo de este sistema se ha optado por devolver archivos en formato JSON, siendo este tipo de archivos muy común, fácil de leer y de procesar. Para poder devolver este archivo generado desde el servidor, primero hay que recibir una petición desde el cliente y esto se hace a través de los métodos HTTP (explicados en el apartado *Comunicación y Servicios Web* del estado

del arte). Gracias a estos métodos, el cliente y servidor se pueden comunicar de forma segura.

6.4.1. Tecnología utilizada y diseño de endpoints

La API se ha implementado en **Java** utilizando **Spring Boot**, en coherencia con la tecnología seleccionada para el backend del sistema. Esta elección facilita el mantenimiento del proyecto, ya que permite reutilizar el mismo ecosistema de herramientas, librerías y patrones de desarrollo en toda la aplicación.

Spring Boot, mediante anotaciones y su contenedor de [inversión de control](#), permite registrar y gestionar automáticamente los componentes de la aplicación (controladores, servicios y repositorios). De este modo, se reduce el código repetitivo, se simplifica la inyección de dependencias y se mejora la organización de la lógica de negocio.

En cuanto al diseño de endpoints, se ha seguido un enfoque REST orientado a recursos, con rutas claras y consistentes, uso de métodos HTTP según la operación (GET, POST, PUT / PATCH y DELETE) y códigos de estado adecuados para cada respuesta. Este criterio favorece una API fácil de documentar e integrar desde clientes móviles o web.

Se han implementado varios controladores de acuerdo al recurso que manipulan. Estos controladores han sido los siguientes:

- **ActivityController**

Este controlador manipula los recursos relacionados con las actividades. Los endpoints desarrollados son los siguientes:

- **POST - /api/activity/user/userId/enclosure/enclosureId**

Descripción: crea una actividad asociada a un recinto para un usuario específico. El ID del usuario y del recinto es pasado como variable de la URL. Además, se podrá pasar el ID de la entidad como parámetro de la petición si la actividad es realizada perteneciendo el usuario a una entidad.

Datos enviados a través del JSON:

```
1      {
2          "date" (LocalDate): "",
3          "season" (string): "",
4          "description" (string | null): "",
5          "type" (ActivityType): ,
6          "status" (ActivityStatus): ,
7          "phytoAct" (CreatePhytoActReq): [ ... ],
8      }
9
```

Extracto de código 6.1: Creación de actividad realizada por usuario

El tipo de dato “ActivityType” es un *enum*, pudiendo definir el tipo de actividad que se guarda (ahora mismo solo tiene el valor “fitosanitaria”), “ActivityStatus” es un *enum* que representa el estado que tiene la actividad, pudiendo ser “planificada” y “completada” principalmente.

Este JSON contiene un campo “phytoAct” que representa la información de la actividad fitosanitaria. En esta clave se almacena una lista que contiene por cada valor el siguiente JSON:

```
1      {
2          "phytoProductId" (long): ,
3          "applicationDate" (LocalDate): ,
4          "reason" (string): "",
5          "dose" (double): ,
6          "doseUnit" (string): ,
7          "totalAmount" (double): [ ... ],
8          "applicationMethod" (string): "",
9          "applicationMachinery" (string): "",
10         "applicatorName" (string): "",
11         "applicatorNif" (string): "",
12         "crops" (string): "",
13         "area" (double):
14     }
```

Extracto de código 6.2: Creación de actividad realizada por usuario

Respuesta: se devuelve un JSON que contiene el id de la actividad y de la actividad fitosanitaria guardada en la base de datos junto con el instante en el que ha sido creado.

- **POST - /api/activity/activityId**

Descripción: añade una nueva actividad fitosanitaria a una actividad ya creada anteriormente. El ID de la actividad es pasado como variable de la URL.

Datos enviados a través de un JSON:

```
1      {
2          "phytoProductId" (long): ,
3          "applicationDate" (LocalDate): ,
4          "reason" (string): "",
5          "dose" (double): ,
6          "doseUnit" (string): ,
7          "totalAmount" (double): [ ... ],
8          "applicationMethod" (string): "",
9          "applicationMachinery" (string): "",
10         "applicatorName" (string): "",
11         "applicatorNif" (string): "",
12         "crops" (string): "",
13         "area" (double):
```



```
14     }
15
```

Extracto de código 6.3: Nueva actividad fitosanitaria añadida a una actividad

Respuesta: devuelve un JSON que contiene el id de la actividad y de la actividad fitosanitaria guardada en la base de datos junto con el instante en el que ha sido creado.

- **DELETE - /api/activity/id**

Descripción: elimina una actividad almacenada en el sistema. El ID de la actividad es pasado como variable de la URL.

Respuesta: devuelve una respuesta sin contenido con código 204.

- **PUT - /api/activity/activityId**

Descripción: actualiza una actividad almacenada en el sistema. El ID de la actividad es pasado como variable de la URL.

Datos enviados a través de un JSON:

```
1     {
2         "description" (string): "",
3         "status" (ActivityStatus): ,
4         "phytoAct" (UpdatePhytoActReq): ,
5     }
6
```

Extracto de código 6.4: Actualización de actividad

El tipo de dato "UpdatePhytoActReq" contiene la información sobre la actividad fitosanitaria que se va a actualizar. Se trata de un JSON y contiene la siguiente información:

```
1     {
2         "phytoActId" (long): ,
3         "reason" (string): "",
4         "dose" (double): ,
5         "doseUnit" (string): "",
6         "totalAmount" (double): ,
7         "applicationMethod" (string): "",
8         "applicationMachinery" (string): "",
9         "applicatorNif" (string): "",
10        "area" (double):
11    }
12
```

Extracto de código 6.5: Actualización de actividad fitosanitaria

Respuesta: devuelve un JSON con toda la información de la actividad.

- **GET - /api/activity/user/userId/enclosure/enclosureId**

Descripción: obtiene todas las actividades de un usuario específico en un

recinto concreto. El ID del usuario y del recinto es pasado como variable de la URL. Además, se puede pasar como parámetros de la URL el *id* de la entidad, en caso de que se quieran consultar las actividades perteneciendo a una entidad, y la *season*, para obtener las actividades de una campaña concreta.

Respuesta: devuelve un JSON que contiene una lista con la información de cada una de las actividades encontradas en la base de datos.

- **GET - /api/activity/user/userId**

Descripción: obtiene todas las actividades de un usuario específico. El ID del usuario es pasado como variable de la URL.

Respuesta: devuelve un JSON que contiene una lista con la información de cada una de las actividades encontradas en la base de datos.

- **AdminController**

Este controlador establece aquellos endpoints que solo van a ser ejecutados por un usuario con rol administrador dentro del sistema. Los endpoints desarrollados son los siguientes:

- **POST - /api/admin/user**

Descripción: crea un usuario del sistema sin necesidad de enviar email y esperar confirmación.

Datos enviados a través de un JSON:

```
1      {
2          "firstname" (string): "",
3          "lastname" (string): "",
4          "email" (string): "",
5          "password" (string): "",
6          "phone" (string | null): "",
7          "nif" (string): "",
8          "entityNif" (string | null): ""
9          "roles" (RoleTypes | null): [ ... ]
10     }
11
```

Extracto de código 6.6: Creación de usuario

“RoleTypes” es un dato de tipo ENUM que define los tipos de roles que pueden haber en el sistema: ROLE_ADMIN y ROLE_BASIC. Si no se pasa ningún rol durante la creación del usuario, este será creado con ROLE_BASIC por defecto. El NIF de la entidad solo es necesario si el usuario que se va a crear se quiere vincular directamente con una entidad específica.

Respuesta: se devuelve un JSON con la información del usuario creado.

- **DELETE - /api/admin/user/id**

Descripción: elimina un usuario del sistema a partir del ID con el que está almacenado en la base de datos. El ID es pasado como variable de la URL.

Respuesta: se devuelve una respuesta sin contenido con código 204.

■ AuthController

Este controlador expone las rutas que van a permitir acceder a la aplicación a través de un registro inicial o iniciando sesión. Va a contener las URLs que generan los tokens para que los usuarios puedan ejecutar el resto de endpoints expuestos por la API una vez hayan iniciado sesión. Los endpoints desarrollados son los siguientes:

• POST - /api/auth/register

Descripción: registra un usuario en el sistema. En este caso, el usuario deberá de confirmar su registro a través del correo que se le enviará por correo electrónico (a diferencia de la creación de usuario que hace el admin).

Datos enviados a través de un JSON:

```
1      {
2          "firstname" (string): "",
3          "lastname" (string): "",
4          "email" (string): "",
5          "password" (string): "",
6          "phone" (string | null): "",
7          "nif" (string): "",
8          "entityNif" (string | null): ""
9          "roles" (RoleTypes | null): [ ... ]
10     }
11
```

Extracto de código 6.7: Registro de usuario

Es el mismo JSON que se utiliza para crear un usuario. Por defecto, el usuario que se crea a través del registro no se vincula de manera inicial a ninguna entidad, por lo que el nif de la entidad no se pasará, al igual que ocurre con el rol (por defecto tendrá rol básico del sistema).

Respuesta: se devuelve un JSON con la información del usuario creado.

• POST - /api/auth/register/entityAdmin

Descripción: registro de un usuario en el sistema y vinculación con una entidad específica con rol de administrador.

Datos enviados a través de un JSON:

```
1      {
2          "firstname" (string): "",
3          "lastname" (string): "",
4          "email" (string): "",
5          "password" (string): "",
6          "phone" (string | null): "",
7          "nif" (string): "",
8          "entityNif" (string | null): ""

```

```

9         "roles" (RoleTypes | null): [ ... ]
10     }
11

```

Extracto de código 6.8: Registro de usuario administrador de entidad

En este caso, el nif de la entidad será enviado para que se pueda vincular el usuario registrado con la entidad específica. El rol será básico en el sistema en caso de no ser pasado en el cuerpo del JSON.

Respuesta: se devuelve un JSON con la información del usuario creado.

- **GET - /api/auth/confirm**

Descripción: confirmación del usuario registrado. Cuando un usuario pulse el botón del email que se le ha enviado, se ejecutará esta funcionalidad que permite confirmar su registro.

Respuesta: se devuelve un JSON con la información del usuario confirmado.

- **POST - /api/auth/login**

Descripción: inicio de sesión de un usuario en el sistema.

Datos enviados a través de un JSON:

```

1     {
2         "email" (string): "",
3         "password" (string): ""
4     }
5

```

Extracto de código 6.9: Inicio de sesión

Respuesta: se devuelve un JSON con el token de acceso y el token de refresco. El token de acceso contiene información cifrada del usuario que permite identificarlo, sabiendo de esta forma si está autorizado a acceder a la información que solicita.

- **POST - /api/auth/refresh**

Descripción: creación de nuevo token de acceso y token de refresco.

Datos enviados: token en la cabecera (headers) de la petición.

Resuesta: se devuelve un JSSON con un nuevo token de acceso y token de refresco.

- **EntityController**

Este controlador manipula los recursos relacionados con una entidad habilitada. Los endpoints desarrollados son los siguientes:

- **POST - /api/entity/**

Descripción: creación de una entidad habilitada en el sistema.

Datos enviados a través del JSON:

```

1      {
2          "name" (string): "",
3          "nif" (string): "",
4          "email" (string): "",
5          "phone" (string | null): ""
6      }
7

```

Extracto de código 6.10: Creación de entidad habilitada

Respuesta: devuelve un JSON con la información del entidad creada.

- **POST - /api/entity/entityId**

Descripción: creación de un vínculo de un usuario con una entidad. La entidad es identificada a través de su ID que es pasado como variable de la URL.

Datos enviados a través del JSON:

```

1      {
2          "email" (string): "",
3          "entityRole" (EntityRoleTypes): "",
4          "writeCue" (Boolean | null): true | false,
5          "writeRea" (Boolean | null): true | false,
6          "readCue" (Boolean | null): true | false,
7          "readRea" (Boolean | null): true | false,
8      }
9

```

Extracto de código 6.11: Creación de usuario vinculado a entidad

El "email" corresponde al del usuario, el valor "EntityRoleTypes" identifica los tipos de roles que tiene un usuario dentro de una entidad, pudiendo ser: administrador o agricultor (*farmer*).

- **PUT - /api/entity/entityId/user/userId**

Descripción: actualización de los permisos y/o roles de un usuario asignado a una entidad. Tanto el ID de la entidad como el ID del usuario son pasados como variables de la URL.

Datos enviados a través de un JSON:

```

1      {
2          "entityRole" (EntityRoleTypes | null): "",
3          "writeCue" (Boolean | null): true | false,
4          "writeRea" (Boolean | null): true | false,
5          "readCue" (Boolean | null): true | false,
6          "readRea" (Boolean | null): true | false,
7      }
8

```

Extracto de código 6.12: Actualización de datos de un usuario vinculado a una entidad

Respuesta: se devuelve un JSON con la información del usuario vinculado a la entidad (roles y permisos dados).

- **DELETE - /api/entity/entityId/user/userId**

Descripción: eliminación del vínculo de un usuario con una entidad específica. Tanto el ID del usuario como el ID de la entidad son pasados como variables de la URL.

Respuesta: se envía una respuesta sin contenido con código 204.

- **GET - /api/entity/user/userId**

Descripción: obtiene todas las entidades de un usuario específico. El ID del usuario es pasado como variable de la URL.

Respuesta: se devuelve un JSON que contiene una lista con la información de la asignación del usuario con cada entidad (roles y permisos).

- **PhytoProductController**

Este controlador manipula los recursos relacionados con los productos fitosanitarios. Los endpoints desarrollados son los siguientes:

- **POST - /api/phytoProduct/**

Descripción: creación de un producto fitosanitario.

Datos enviados a través del JSON:

```
1      {
2          "name" (string): "",
3          "officialRegister" (string): "",
4          "activeSubstance" (string): "",
5          "type" (string): ""
6      }
7
```

Extracto de código 6.13: Creación de producto fitosanitario

El campo "officialRegister" se refiere al número de registro que identifica al producto, "activeSubstance" es la materia activa del producto (componentes químicos o biológicos) y "type" se refiere al tipo de producto (insecticida, fertilizante, etc).

Respuesta: se devuelve un JSON con la información del producto fitosanitario creado.

- **GET - /api/phytoProduct/all**

Descripción: consulta de todos los productos fitosanitarios.

Respuesta: se devuelve un JSON que contiene una lista con la información de cada producto fitosanitario.

■ PlotController

Este controlador manipula los recursos relacionados con una parcela y sus recintos. Los endpoints desarrollados son los siguientes:

- **POST - /api/plot/user/userId**

Descripción: crea una asignación entre un usuario y una parcela. El ID del usuario sobre el que se asigna la parcela es pasado como variable de la URL.

Datos enviados a través del JSON:

```
1      {
2          "name" (string): "",
3          "province" (string): "",
4          "city" (string): "",
5          "polygonCode" (string): "",
6          "plotNum" (string): "",
7          "landRegister" (string): ""
8      }
9
```

Extracto de código 6.14: Creación de asignación entre usuario y parcela

Los campos pasados se refieren al nombre le va a asignar el usuario a la parcela, el nombre de la provincia y municipio en la que se encuentra, el código del polígono y número de la parcela (se pueden consultar en la aplicación de *Visor SIGPAC*), y la referencia catastral.

Respuesta: se devuelve un JSON que contiene la información de la parcela creada junto con sus recintos.

- **DELETE - /api/plot/id**

Descripción: elimina una parcela del sistema. El ID de la parcela es enviado como variable de la URL.

Respuesta: se devuelve una respuesta sin contenido con código 204.

- **DELETE - /api/plot/plotId/user/userId**

Descripción: elimina la asignación entre un usuario y una parcela. El ID del usuario y de la parcela son pasados como variables de la URL.

Respuesta: devuelve una respuesta sin contenido con código 204.

- **GET - /api/plot/plotId**

Descripción: obtiene la información de una parcela específica. El ID es pasado como variable de la URL.

Respuesta: devuelve un JSON con la información de la parcela solicitada.

- **GET - /api/plot/plotId/enclosures**

Descripción: obtiene todos los recintos de una parcela específica. El ID de la parcela es pasado como variable de la URL.

Respuesta: devuelve un JSON con la información de la parcela y de todos los recintos que la componen.

- **GET - /api/plot/user/userId**

Descripción: obtiene todas las parcelas que pertenecen a un usuario. El ID del usuario es pasado como variable de la URL.

Respuesta: devuelve un JSON con una lista que contiene todas las parcelas (junto con sus recintos), que pertenecen al usuario.

- **GET - /api/plot**

Descripción: obtiene el id de un recinto específico dentro de una parcela asignada a un usuario. Para ejecutar este endpoint también se deben de pasar de manera obligatoria los parámetros: *plotName*, *enclosureName*, *userId*.

Respuesta: se devuelve un JSON con el id del recinto almacenado en la base de datos.

- **UserController**

Este controlador manipula los recursos relacionados con un usuario del sistema. Los endpoints desarrollados son los siguientes:

- **GET - /api/user/me**

Descripción: obtiene la información del usuario que lo ejecuta.

Respuesta: devuelve un JSON con la información del usuario que lo ha ejecutado.

- **GET - /api/user/id**

Descripción: obtiene la información de un usuario específico. El ID del usuario es pasado como variable de la URL.

Respuesta: devuelve un JSON con la información del usuario específico.

- **GET - /api/user**

Descripción: es necesario ejecutar el endpoint pasando el "email" como parámetro de la URL. Obtiene un usuario a partir de su correo electrónico.

Respuesta: devuelve un JSON con la información del usuario específico.

- **GET - /api/user/all/entity/entityId**

Descripción: obtiene todos los usuarios que pertenecen a una entidad. El ID de la entidad es pasado como variable de la URL.

Respuesta: devuelve un JSON con una lista que contiene la información de cada usuario que pertenece a la entidad.

- **GET - /api/user/all**

Descripción: obtiene todos los usuarios del sistema.

Respuesta: devuelve un JSON que contiene una lista con la información de cada usuario guardado en el sistema.

Hay un controlador más llamado **ErrorResponseHandler** que es el encargado de manejar las excepciones que se lanzan durante la ejecución del programa, bien por un error en la validación de los datos o por un error generado durante el procesamiento de la lógica de negocio.

6.4.2. Autenticación y autorización

El objetivo del proceso de autenticación y autorización es proteger los recursos de la API identificando a los usuarios y controlando qué acciones pueden realizar según su rol y su relación con las entidades del sistema. En el presente proyecto se ha considerado el registro y verificación por correo, autenticación a través de inicio de sesión con correo electrónico y contraseña, emisión de tokens (acceso y *refresh*) y autorización tanto por roles (*admin* y *basic*) como por reglas de negocio específicas (permisos asociados a entidades según rol).

El flujo que se ha seguido para realizar la autenticación es el siguiente:

- **Registro:** el usuario comienza registrándose mediante la ejecución del endpoint de **registro**. Al registrarse se crea el objeto usuario y se almacena en la base de datos junto con un token de confirmación que se envía por correo. Hasta que el usuario no confirme su registro no podrá acceder al sistema.
- **Confirmación:** el usuario confirmará su cuenta al abrir el enlace en dicho correo, comprobando en el backend si el token es válido y su expiración. De ser todo correcto, el usuario se marca como habilitado y puede acceder al sistema a través del inicio de sesión.
- **Inicio de sesión (*login*):** el usuario enviará el email junto con la contraseña al backend. El backend se encargará de comprobar si el usuario existe buscándolo en la base de datos y obteniéndolo. Con la información del usuario obtenida y si está habilitado en el sistema, se creará un token de acceso y de refresco que utilizará el usuario para las futuras peticiones a la API.
- **Refresh:** por último, en caso de que el usuario necesite nuevos tokens para acceder a los recursos del sistema porque el que tiene ha expirado, enviará el *refresh token* en el campo **Authorization** del cabecera de la petición y, si es válido, generará nuevos tokens que serán enviados al usuario.

Para la verificación de la identidad del usuario que accede al sistema, los tokens con los que accede el usuario son generados con un tipo de clave **HMAC** [42]. Este tipo de clave también conocida como *Código de autenticación de mensaje basado en hash*, es una construcción específica para calcular un código de autenticación de mensaje (MAC) que implica una función hash criptográfica en combinación con una llave criptográfica secreta. En ella va cifrada información propia del usuario como el email (*subject*) y otra información auxiliar como el ID del usuario o el nombre (*claims*).

Cuando el usuario hace peticiones con un token, hay un filtro que va a interceptar la llamada antes de ejecutar el contenido del endpoint. Este filtro extrae el token guardado en el campo *Authorization* de la cabecera, extrae la información

que almacena y verifica tanto si existe en el sistema como si no ha expirado. Si todo es correcto, almacena en un contexto el usuario “logueado”.

En cuanto al control de acceso, como se ha comentado anteriormente, cada usuario tiene una serie de roles asociados a nivel de sistema y a nivel de entidad (en caso de pertenecer a una). Los roles del sistema son dos: **ADMIN** y **BASIC**.

El rol “basic” permite al usuario poder crear parcelas, actividades asociadas a esas parcelas así como la eliminación y actualización de cada una de ellas. También podrá ver a qué entidades pertenece y sus datos personales. En cuanto al usuario con rol “admin”, va a poder realizar todas las acciones que tiene un usuario “basic” pero además, podrá ver, eliminar y actualizar todos los usuarios registrados en el sistema.

Se han definido reglas de acceso a los recursos del sistema así como funciones de validación para determinar si el usuario puede ejecutar una funcionalidad o tiene autorización para acceder a un método/recurso. En próximos apartados se entrará en más detalle sobre esta parte.

6.4.3. API del Visor SIGPAC

SIGPAC ofrece un amplio catálogo de servicios entre los cuales, se puede acceder a información como:

- Los códigos utilizados por SIGPAC para identificar las provincias, municipios y parcelas.
- Geometrías y propiedades de las capas de los recintos, cultivos y elementos del paisaje.
- Información relacionada con la infraestructura del Visor SIGPAC en formato JSON o [GeoJson](#).

Dentro de estos servicios, los que se han utilizado para recoger los datos y almacenarlos en el sistema han sido: **Listas de Códigos SIGPAC** y **Servicio de Consultas de SIGPAC**.

En *Listas de códigos SIGPAC* se encuentran distintos endpoints o URL a las que se pueden acceder para obtener distintos listados que ofrecen los códigos de las provincias, códigos de municipios y código de uso sigpac (cítricos, corrientes y superficies de agua, etc), entre otros.

Para el desarrollo del sistema, se han utilizado los siguientes endpoints dentro de este grupo:

- **Listado de códigos de provincias**

A través de la URL que ofrecen, se puede obtener un listado con todos los códigos asociados a cada provincia. La URL que ofrecen es la siguiente:

<https://sigpac-hubcloud.es/codigossigpac/provincia.json>

A través de esta URL, se obtiene como respuesta un archivo JSON con la siguiente estructura:

```
1  {
2      "nombre": "provincia",
3      "descripcion": "Lista de codigos de Provincia SIGPAC",
4      "codigos": [
5          {
6              "codigo": 1,
7              "descripcion": "ALAVA"
8          },
9          {
10             "codigo": 2,
11             "descripcion": "ALBACETE"
12          },
13          {
14             "codigo": 3,
15             "descripcion": "ALICANTE"
16          },
17          ...
18      ]
19  }
20
```

Extracto de código 6.15: Listado de provincias SIGPAC

Gracias a esta respuesta, se puede realizar una búsqueda por el campo “descripcion”, y obtener el valor del campo “código”, con el que se identificará la provincia.

■ Listado de códigos de municipios

A través de la URL que ofrecen, se puede obtener un listado con todos los códigos de cada municipio asociado a una provincia. La URL que ofrecen es la siguiente:

[https://sigpac-hubcloud.es/codigossigpac/municipio\[cod_prov\].json](https://sigpac-hubcloud.es/codigossigpac/municipio[cod_prov].json).

La variable de la url “cod_prov”, es el código de la provincia (el que se muestra en el listado anterior).

Al realizar una llamada a este endpoint con el código de municipio 4 (por ejemplo), se obtiene como respuesta un archivo JSON con la siguiente estructura:

```
1  {
2      "nombre": "municipio4",
3      "descripcion": "Lista de codigos de la provincia 4",
4      "codigos": [
5          {
6              "codigo": 1,
7              "descripcion": "Abla"
```

```

8         },
9         {
10            "codigo": 2,
11            "descripcion": "Abrucena"
12        },
13        {
14            "codigo": 3,
15            "descripcion": "Adra"
16        },
17        ...
18    ]
19 }
20

```

Extracto de código 6.16: Listado de municipios SIGPAC

Gracias a esta respuesta, se puede realizar una búsqueda por el campo “descripcion”, y obtener el valor del campo “código”, con el que se identificará el municipio.

Por otra parte, está el *Servicio de Consultas SIGPAC*. Con este servicio se puede obtener la información SIGPAC en formato GeoJson de un recinto específico o de todos los recintos asociados a una parcela.

Para el desarrollo del sistema, se han utilizado los siguientes endpoints:

- **Propiedades de recinto por código SIGPAC**

Con esta consulta, se puede obtener las propiedades de todos los recintos de una parcela, dados los códigos SIGPAC de una parcela específica. La URL que ofrecen es la siguiente:

[https://sigpac-hubcloud.es/servicioconsultassigpac/query/recinfoparc/\[pr\]/\[mu\]/\[ag\]/\[zo\]/\[po\]/\[pa\].geojson](https://sigpac-hubcloud.es/servicioconsultassigpac/query/recinfoparc/[pr]/[mu]/[ag]/[zo]/[po]/[pa].geojson).

La variable “pr” representa el código de la provincia, “mu” representa el código del municipio, “ag” representa el código de agregado (por defecto toma el valor 0), “zo” representa el código de la zona (por defecto toma el valor 0), “po” representa el código del polígono y “pa” representa el código de la parcela. Para poder consultar los dos últimos códigos, gracias a la aplicación *Visor SIGPAC* se puede acceder a esta información.

Si es ejecutada una llamada a este endpoint, la respuesta será un archivo en formato .GeoJson con la siguiente estructura:

```

1    [
2      {
3        "provincia": "31",
4        "municipio": 192,
5        "agregado": 0,
6        "zona": 0,
7        "poligono": 5,

```

```

8         "parcela": 83,
9         "recinto": 1,
10        "superficie": 9.4722,
11        "pendiente_media": 11.4,
12        "coef_regadio": 0,
13        "admisibilidad": 0,
14        "incidencias": null,
15        "uso_sigpac": F0,
16        "region": null,
17        "wkt": "POLYGON(...)",
18        "srid": 4258
19    },
20    ...
21 ]
22

```

Extracto de código 6.17: Propiedades de los recintos de una parcela

La respuesta es una lista con elementos que representan cada recinto. Por cada elemento, las propiedades del recinto significan lo siguiente:

- provincia: código de la provincia.
- municipio: código del municipio SIGPAC.
- agregado: código del agregado.
- zona: código de la zona.
- poligono: número del polígono.
- parcela: número de la parcela.
- recinto: código del recinto SIGPAC.
- superficie: superficie del recinto expresado en hectáreas.
- pendiente_media: pendiente media del recinto.
- coef_regadio: coeficiente de regadío del recinto.
- admisibilidad: porcentaje de admisibilidad de aplicación en los recintos de pastos
- incidencias: códigos de las incidencias del recinto separados por comas.
- uso_sigpac: código de uso del recinto.
- region: código de la región asignada al recinto.
- wkt: coordenadas que definen el recinto en formato WKT.
- srid: srid o código EPSG en el que están expresadas las coordenadas del recinto definido en el campo wkt.

La propiedad principal es el valor que hay en el campo “wkt”. En este campo viene una lista de objetos con cada uno de los puntos que forman el polígono del recinto. Estos puntos son representados con valores de latitud y longitud. Teniendo todos los puntos, se puede crear un objeto de tipo “polígono” que posteriormente pueda ser dibujado sobre un mapa.

Si se combinan los códigos obtenidos al ejecutar las llamadas anteriores (provincia y municipio), con esta URL, se pueden obtener todos los recintos de la parcela en cuestión.

7. Implementación

En este capítulo se describe el proceso de implementación del sistema, prestando atención a las principales decisiones técnicas adoptadas, a la evolución del desarrollo y a los problemas encontrados durante el trabajo.

Antes de comenzar la implementación, se realizaron reuniones de seguimiento y análisis que permitieron concretar los requisitos funcionales del sistema y delimitar el alcance de la aplicación. A partir de estas reuniones se identificaron las funcionalidades prioritarias y se organizó el desarrollo en iteraciones sucesivas.

La explicación de la implementación se presenta siguiendo el orden de los sprints realizados, de forma que pueda entenderse la evolución progresiva del sistema desde sus primeras funcionalidades hasta la versión final desarrollada.

7.1. Sprint 1

Para que se pueda acceder a un sistema, es necesario que la “persona” esté registrada en ella. Por ello, en este sprint se fijó la implementación del **registro de un usuario** y del **inicio de sesión**.

Primero se elaboró el esquema de la base de datos. Analizando el uso que iba a tener la aplicación por parte de los usuarios, se fijó un sistema de roles basado en dos tipos: **admin** y **basic** (ya explicados anteriormente).

Teniendo en cuenta esto y que un usuario tiene que estar representado también a través de una entidad, el esquema desarrollado en la primera iteración fue el siguiente:

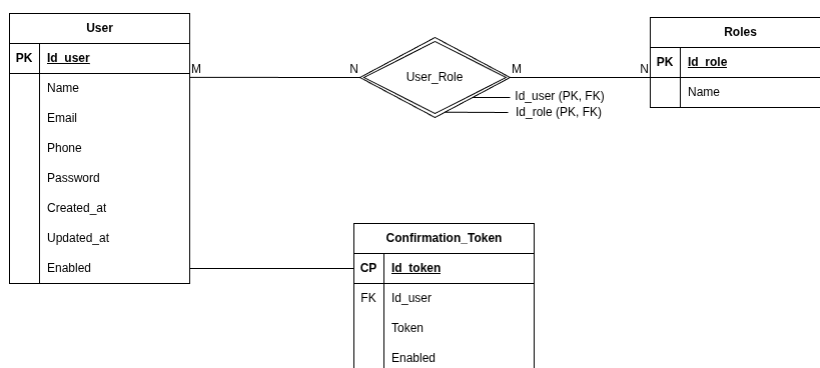


Figura 7.1: Esquema de la base de datos del Sprint 1

La entidad usuario representa al usuario como tal que accede y utiliza la aplicación. Datos como el correo electrónico o el NIF son almacenados en el sistema.

En cuanto a los roles, estos son almacenados en una tabla, en la que solo se guarda el nombre que identifica al rol.

Para poder establecer la relación entre un usuario y el rol que tiene, hay una tabla intermedia que corresponde a una “relación”. Esta tabla almacena el ID del usuario con el ID del rol, estableciendo la relación de esta forma. Esta tabla intermedia permite relacionar a un usuario con varios roles, de manera que un usuario administrador tendrá rol administrador y rol básico, mientras que un usuario normal solo tendrá rol básico.

Para crear las entidades y sus relaciones en la base de datos de PostgreSQL se ha realizado con Spring Boot directamente, ya que el framework tiene un [ORM](#) que facilita esta implementación. Para ello, es necesario previamente establecer la conexión con la base de datos. Esto se hace a través de un archivo conocido como **application.properties**, donde se fija el puerto, nombre de usuario, base de datos, entre otras más propiedades.

Establecidas las propiedades, se procede a crear las entidades. El proyecto ha sido organizado en distintos directorios, cuyo significado y contenido se irá explicando a lo largo de este capítulo. Teniendo en cuenta esto, las entidades han sido guardadas en un directorio llamado **models**, representando los modelos (entidades) del sistema.

Una entidad en Spring Boot es creada como una clase que tiene la anotación **@Entity**. Esto indica a Spring Boot que la clase va a ser un componente del tipo Entidad. Además de esta anotación, se han añadido otras más como: **@Getter** (crea los métodos *gets* para cada atributo), **@Setter** (crea los métodos *setters* para todos los atributos) y **@Table** (permite indicar el nombre que tiene la entidad o tabla en la base de datos).

Siguiendo este patrón, se crearon las dos entidades junto con la relación entre ellas. Además de estas, se creó una entidad más que almacena los tokens de confirmación generados cuando un usuario se registra, sabiendo de esta forma qué token pertenece a cada usuario y saber si su registro está confirmado o no.

A continuación, se crearon los servicios que van a controlar el registro de usuarios y el inicio de sesión, siendo principalmente: **UserService** y **AuthService**.

7.1.1. AuthService

Este servicio es el encargado de manejar la lógica de negocio entre el **registro** de un usuario del sistema o de un administrador de entidad, **confirmación** de un usuario registrado en el sistema, **inicio de sesión** de un usuario y **refresco** o renovación de los *tokens* de acceso.

Entrando en más detalle, cada uno de los métodos que componen la clase tienen la siguiente funcionalidad:

- **registerSystemUser**: registra un usuario en el sistema. A partir de los datos que obtiene, los envía a **UserService** para realizar el registro. Una vez realizado

el registro, crea la URL para confirmar la cuenta y se envía al usuario a través del EmailService.

- **confirm:** a través del token que viaja en la URL del correo electrónico, se obtiene el usuario relacionado con el token. Obtenido el usuario, se habilita en el sistema permitiendo que pueda acceder a la aplicación durante el inicio de sesión.
- **login:** permite la autenticación del usuario en el sistema. Utiliza el *AuthenticationManager* de Spring Boot, que es el encargado de realizar la autenticación buscando al usuario en la base de datos de acuerdo al correo electrónico y la contraseña. Si el usuario es autenticado, se obtiene a través de un objeto del tipo *UserDetails*, que identifica al usuario. Con la información del usuario se crean los tokens de acceso y refresco a través del *JwtService*.
- **refresh:** permite generar nuevos tokens de acceso a la aplicación. Para ello, utiliza el token de refresco. Con este token puede extraer la información codificada y buscar si el usuario sigue existiendo en el sistema y si no ha sido expirado. Si todo es correcto, se crean los nuevos tokens.

Para establecer la seguridad en la aplicación e indicar quién puede acceder a qué endpoints y cuáles requieren autenticación, se utiliza un archivo llamado **SecurityConfig**. En este archivo se fijan propiedades de seguridad como la configuración de **CORS**, si se maneja estado entre las sesiones, cómo son las excepciones que se manejan en caso de que el acceso sea denegado (falta de permisos o autorización) o necesidad de autenticación para ejecutar la funcionalidad. La parte más importante es la siguiente:

```
.authorizeHttpRequests(request -> request
    .requestMatchers(HttpMethod.POST, "/api/auth/register/entityAdmin")
        .authenticated()

    // Allow all requests to auth endpoints
    .requestMatchers("/api/auth/**").permitAll()

    .requestMatchers("/api/admin/**")
        .hasRole("ADMIN")
    .requestMatchers(HttpMethod.POST, "/api/entity/")
        .hasRole("ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/api/plot/{id}")
        .hasRole("ADMIN")
    .requestMatchers("/api/entity/{entityId}/**")
        .hasAnyRole("ADMIN", "BASIC")
    .anyRequest()
        .authenticated()
)
```

En esta parte, como se puede ver, se fija qué endpoints del controlador se pueden ejecutar según el rol, cuáles puede ejecutar cualquier usuario y cuáles requieren autenticación.

Por último y no menos importante, se pueden añadir filtros que Spring Boot necesite aplicar antes de comprobar si el usuario que ejecuta la petición HTTP tiene acceso a un recurso y si está autenticado o no. El filtro implementado es *JwtAuthFilter*. Este filtro se va a aplicar antes de ejecutar el contenido del endpoint del controlador (llamada la servicio), de manera que comprueba si el usuario ha enviado en la cabecera de la petición el token, si es válido o no y si el usuario sigue existiendo en el sistema. En caso de ser correcto, autentica al usuario guardando la información en el contexto de la petición y continuando Spring Boot el flujo del programa; en caso contrario, continua con la cadena de filtros y pasaría al contenido del archivo SecurityConfig, lanzando una excepción por falta de permisos o autenticación.

7.1.2. JwtService

Este servicio es el encargado de crear los tokens de acceso a la aplicación y de refresco (*refresh*). Para crear el token es necesario establecer un **tiempo de expiración**, otro tiempo de expiración para el de refresco y una **clave privada** con la que se firmará el token durante su creación. Esta clave es creada a partir de algoritmos como *HS256* y permite verificar en el servidor que el token no ha sido manipulado durante su transmisión y que proviene de una fuente legítima, evitando falsificaciones y garantizando de esta manera la integridad y autenticidad.

La creación del token tiene la siguiente estructura:

```
Jwts.builder()
    .id(user.getId().toString())
    .claims(Map.of(
        "firstname", user.getFirstname(),
        "userId", user.getId()
    ))
    .subject(user.getEmail())
    .issuedAt(new Date(System.currentTimeMillis()))
    .expiration(new Date(System.currentTimeMillis() + expiration))
    .signWith(getSignKey())
    .compact();
```

Jwts es una clase del paquete *jsonwebtoken* que permite construir tokens en Java. Lo que hacen estas líneas es lo siguiente:

1. Se añade el id del usuario obtenido al realizar un inicio de sesión en la aplicación.
2. Se introduce información adicional (*claims*), como puede ser el nombre del usuario.
3. En el campo *subject* se añade el email del usuario, ya que es la forma de identificar de manera única al usuario sobre el que se crea el token.

4. A continuación se introduce el instante de tiempo en el que se ha creado el token (*issuedAt*) y en qué instante expirará el token (*expiration*).
5. Finalmente se firma el token con la clave privada generada.

Tanto la creación del token de acceso como del token de refresco se hace utilizando el mismo código explicado. Lo único que habría que cambiar es el tiempo de expiración de cada token, teniendo el token de refresco un tiempo de expiración mayor que el de acceso (actualmente el token de acceso tiene un tiempo de expiración de 1 hora y el de refresco de 1 semana).

Además de la creación de tokens, también va a ser el encargado de comprobar si un token es válido o no. Para saber si un token es válido se comprueba si no ha expirado el tiempo (el instante actual es anterior al tiempo de expiración) y que el usuario al que corresponde el token es el mismo que el que está realizando la petición (comprueba que no se esté usando un token de otro usuario).

7.1.3. EmailService

Este servicio es el encargado de realizar el envío de un mensaje de confirmación de registro a través del correo electrónico. El envío se hace a través de una función **asíncrona**, evitando de esta forma un bloqueo y permitiendo continuar la ejecución del código mientras el correo está siendo enviado. La función es la siguiente:

```
try {
    MimeMessage mimeTypeMessage = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(mimeMessage,
                                                    true, "UTF-8");

    helper.setText(emailBody(
        url,
        firstname,
        templatePath,
        emailText
    ), true);
    helper.setFrom(from != null ? from : "");
    helper.setTo(to != null ? to : "");
    helper.setSubject(EMAIL_SUBJECT);
    mailSender.send(mimeMessage);
} catch (MessagingException e) {
    throw new MailSenderException(to);
}
```

La funcionalidad es la siguiente:

1. Se crea un objeto **mimeMessage** a partir de un **JavaMailSender**, siendo el que contendrá la estructura y propiedades del mensaje que se va a enviar.
2. A partir de un **helper** se añadirá el cuerpo del mensaje, que será un archivo HTML junto con algunos parámetros adicionales como la URL que

corresponde al endpoint de *confirm*.

3. Por último, antes de ser enviado el mensaje, se establece el correo del destinatario y el correo del usuario que lo envía, junto con un mensaje de cabecera.

7.1.4. UserService

Este servicio es el encargado de ejecutar todas las funcionalidades relacionadas con un usuario. Algunas de estas funcionalidades son: registro y creación de un usuario del sistema (*createSystemUser*), eliminación de usuario y obtención del usuario según ID o email.

El método de **register** es llamado desde AuthService y es el encargado de crear un usuario con el rol *basic* y almacenarlo en la base de datos. Hay que destacar que en este método es donde se crea el **token de confirmación**, creado a través de la clase UUID que permite generar identificadores únicos universales, muy recomendables para claves primarias:

```
String token = UUID.randomUUID().toString();
LocalDateTime tokenExpiresAt = LocalDateTime.now()
    .plusHours(confirmTokenExpiresHours);
```

Por otro lado, el método de **createSystemUser**, realiza lo mismo que el método de registro con la diferencia de que se pueden establecer los roles que tendrá el usuario (admin, basic o ambos).

7.1.5. Relación entre los componentes

Además de los servicios anteriores, también han sido implementados los siguientes componentes:

- **Controladores:** estos componentes son clases con la anotación **@RestController** de Spring Boot. Esta anotación indica a Spring Boot que el componente creado es un controlador que va a devolver objetos de tipo JSON. Cada controlador contiene distintos endpoints, siendo estos los realizan las llamadas a los servicios mencionados anteriormente.

Tomando como ejemplo el controlador **AuthController**, la estructura es la siguiente:

```
@RestController
@RequestMapping(value = "/api/auth")
@AllArgsConstructor
public class AuthController {

    private AuthService authService;

    @PostMapping("/register")
```

```

    public ResponseEntity<UserDto> registerSystemUser(
        @Valid @RequestBody RegisterRequest request
    ) {
        return authService.registerSystemUser(request);
    }

    @PostMapping("/register/entityAdmin")
    public ResponseEntity<UserDto> registerEntityAdminUser(
        @Valid @RequestBody RegisterRequest request
    ) {
        return authService.registerEntityAdminUser(request);
    }

    @GetMapping("/confirm")
    public ResponseEntity<UserDto> confirm(
        @RequestParam String token
    ) {
        return authService.confirm(token);
    }

    @PostMapping("/login")
    public ResponseEntity<TokenResponse> login(
        @Valid @RequestBody LoginRequest request
    ) {
        return authService.login(request);
    }

    @PostMapping("/refresh")
    public ResponseEntity<TokenResponse> refresh(
        @RequestHeader(
            HttpHeaders.AUTHORIZATION
        ) String authHeader
    ) {
        return authService.refresh(authHeader);
    }
}

```

Como se puede ver, aparece la anotación mencionada además de otras, como por ejemplo **@PostMapping**, que indica que ese endpoint lleva una petición HTTP de tipo "POST"; **@Valid**, que permite validar que los datos del JSON enviados son correctos, **@RequestParam** o **@RequestBody**, que indica que un JSON es enviado en el cuerpo de la petición o que se han pasado parámetros en la URL del endpoint.

Además del resto de controladores (UserController, EntityController, etc), hay un controlador especial que es el encargado de manejar las excepciones que son lanzadas durante el sistema. El controlador ha sido implementado de la

siguiente forma:

```
@RestControllerAdvice
public class ErrorResponseHandler {

    @ExceptionHandler({
        UserAlreadyExistsException.class,
        UserIsEnabledException.class
    })
    public ResponseEntity<ErrorDto> userException(
        Exception ex
    ) {
        ErrorDto error = new ErrorDto(
            "Excepción relacionada con el estado
            del usuario almacenado",
            ex.getMessage(),
            HttpStatus.CONFLICT.value()
        );

        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(error);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorDto> integrityViolation(
        Exception ex
    ) {
        ErrorDto error = new ErrorDto(
            "Error al ejecutar una operación SQL",
            ex.getMessage(),
            HttpStatus.CONFLICT.value()
        );

        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(error);
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ErrorDto> messageNotReadable(
        Exception ex
    ) {
        ErrorDto error = new ErrorDto(
            "Error en la lectura del mensaje",
            ex.getMessage(),
            HttpStatus.CONFLICT.value()
        );

        return ResponseEntity.status(HttpStatus.CONFLICT)
```

```

        .body(error);
    }

    @ExceptionHandler(MailSenderException.class)
    public ResponseEntity<ErrorDto> mailSenderError(
        Exception ex
    ) {
        ErrorDto error = new ErrorDto(
            "Error durante el envío del email de confirmación",
            ex.getMessage(),
            HttpStatus.INTERNAL_SERVER_ERROR.value()
        );

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(error);
    }

    @ExceptionHandler({
        ConfirmTokenNotExistsException.class,
        RoleNotExistsException.class,
        UserNotFoundException.class,
        EntityNotFoundException.class
    })
    public ResponseEntity<ErrorDto> entityNotExists(
        Exception ex
    ) {
        ErrorDto error = new ErrorDto(
            "La entidad no existe en la base de datos",
            ex.getMessage(),
            HttpStatus.NOT_FOUND.value()
        );

        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(error);
    }

    ...
}

```

Este controlador le indica a Spring Boot que es el encargado de manejar las excepciones a través de la anotación **@RestControllerAdvice**. Con la anotación **@ExceptionHandler** se va indicar qué tipo de excepción va a tratar el método y que respuesta va a devolver. Las respuestas tienen la misma estructura a través de un DTO (explicado a continuación) llamado "ErrorDto".

- **Repositorios:** los repositorios son interfaces de Java que tienen la anotación

@Repository. Esta anotación indica a Spring Boot que la interfaz va a ser tratada como un componente dicho tipo, permitiendo reducir la complejidad y el código requerido para implementar el acceso a la capa de manejo de datos y persistencia (base de datos).

Esta interfaz hereda los métodos de otra interfaz llamada **CrudRepository**. Esta interfaz incluye métodos como los siguientes:

- **<S extends T> S save(S entity);** Este método permite guardar una entidad (de cualquier tipo de dato) en la base de datos.
- **Optional<T> findById(ID id);** Este método permite buscar un objeto dentro de la base de datos a partir de un ID. En caso de encontrarlo, se obtiene a través de un método *.get()* del objeto *Optional*.
- **void delete(T entity);** Este método elimina una tupla de la base de datos a partir del propio objeto con el que se está tratando en Spring Boot.
- **long count();** Este método devuelve el número de entidades u objetos que hay en una tabla de la base de datos.

Estos métodos pueden ser usados directamente al instanciar un repositorio y además, pueden ser creados directamente de forma *custom* en el propio componente. Para crear estos métodos, se han utilizados dos formas siguiendo la documentación de Spring [43]:

- Usando palabras clave, como por ejemplo: *and, or, is, equals*, etc. Algunos ejemplos podrían ser: *findByLastnameAndFirstname*, que sería el equivalente a realizar una consulta SQL del tipo ... *WHERE lastname = :lastname AND firstname = :firstname*, o *findByLastnameOrFirstname*, que sería el equivalente en una consulta SQL a ... *WHERE lastname = :lastname OR firstname = :firstname*.
- A través de la anotación **@Query**. Esta anotación permite crear una consulta más compleja, como por ejemplo:

```
@Query("""
SELECT DISTINCT u.email FROM User u
JOIN u.roles sr
JOIN u.userEntity uer
JOIN uer.enabledEntity ee
JOIN uer.entityRole er
WHERE sr.name = :systemRole
      AND ee.nif = :entityNif
      AND er.name = :entityRole
""")
List<String> getEmailEntityAdmins(
    String systemRole,
    String entityRole,
    String entityNif
);
```


En esta consulta se busca el email de un administrador de entidad, usando *DISTINCT* para evitar valores duplicados y realizando *JOINS* con cada una de las tablas necesarias (además de las condiciones con *WHERE* para filtrar el contenido).

Las consultas que se crean con las palabras clave como nombre de los métodos (las primeras de las explicadas), no requieren escribir la consulta SQL como tal. Esto es gracias a que Spring utiliza **Spring Data JPA**. Este módulo de Spring es capaz de analizar un método que está dentro de un repositorio JPA.

JPA e **Hibernate** son los encargados de trabajar en la “sombra” al realizar las operaciones sobre las bases de datos. JPA (*Java Persistence API*) es la especificación que establece las reglas o instrucciones (interfaces) a usar, mientras que Hibernate es el encargado de implementarlas. Gracias a estas dos tecnologías es posible realizar un mapeo de los objetos de Java a los datos que se almacenan en la base de datos y viceversa.

En cuanto a los repositorios implementados durante el sprint 1, fueron principalmente: **UserRepository** y **RoleRepository**.

El primero es el encargado de contener todas las operaciones SQL sobre la tabla de Usuarios, mientras que el segundo es el encargado de realizar todas las operaciones relacionadas con los roles almacenados en la tabla Roles. Algunos ejemplos de consultas dentro del UserRepository pueden ser:

```
Optional<User> findByEmail(String email);

@Query("""
    SELECT u.email FROM User u
    JOIN u.roles r
    WHERE r.name = :roleName
""")
List<String> getEmailByRol(String roleName);

Optional<User> findByConfirmationToken(String confirmationToken);
```

- **DTOs:** los DTOs (*Data Transfer Object*) son objetos planos (*POJO - Plain Old Java Object*) que contienen una serie de atributos que van a ser enviados al servidor, definiendo la estructura de los cuerpos (JSON) que van a ir en las peticiones del cliente al servidor y en el respuestas del servidor al cliente.

Se podría usar directamente el mismo objeto entidad sobre el que se trabaja, por ejemplo enviando un objeto *User*, pero con los DTOs se consigue eliminar información que no quiere ser enviada (como contraseñas) o agregar información extra de otro objeto. Con esta decisión se consigue mayor flexibilidad con la información que se quiere enviar sin manipular entidades.

Algunos de los DTOs utilizados y que han sido implementados durante el sprint son los siguientes:

```

@AllArgsConstructor
@Setter
@Getter
public class LoginRequest {
    @NotBlank(message = "{user.email.notblank}")
    @Email(message = "{user.email.invalid}")
    private String email;

    @NotBlank(message = "{user.password.notblank}")
    @Pattern(
        regexp = "^(?=.*[A-Z])(?=.*[0-9]).{9,}$",
        message = "{user.password.invalid}"
    )
    private String password;
}

```

```

@AllArgsConstructor
@Getter
@Setter
public class RegisterRequest {
    @NotBlank(message = "{user.firstname.notblank}")
    private String firstname;

    @NotBlank(message = "{user.lastname.notblank}")
    private String lastname;

    @NotBlank(message = "{user.email.notblank}")
    @Email(message = "{user.email.invalid}")
    private String email;

    @NotBlank(message = "{user.password.notblank}")
    @Pattern(
        regexp = "^(?=.*[A-Z])(?=.*[0-9]).{9,}$",
        message = "{user.password.invalid}"
    )
    private String password;

    @Pattern(
        regexp = "\\+?[0-9]{7,15}", message = "{user.phone.invalid}"
    )
    private String phone;

    @NotBlank(message = "{user.nif.notblank}")
    @Pattern(regexp = "^[0-9]{8}[A-Z]$")
    private String nif;

    private String entityNif;
}

```

```

        private List<RoleTypes> roles;
    }

```

El primer DTO corresponde al cuerpo de la petición enviada desde el cliente al servidor cuando quiere realizar un inicio de sesión. Se puede observar como se envía tanto el correo como el email. Las anotaciones que tiene sirven para indicar que los campos no pueden ser vacíos ni nulos (*@NotBlank*), que tienen que cumplir una expresión regular (*@Pattern*) y que incluyen métodos de consulta (*@Getter*) como un constructor con todos los argumentos/atributos (*@AllArgsConstructor*).

Los flujos de comunicación o como interactúan entre ellos es de la siguiente forma:

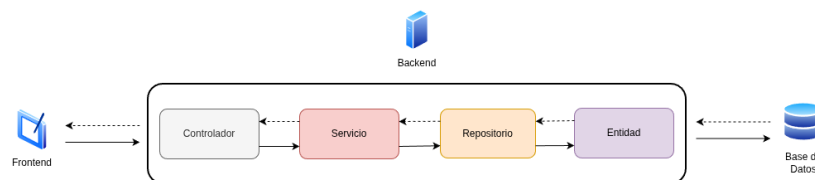


Figura 7.2: Esquema de comunicación entre los componentes

En este esquema se puede observar como los distintos componentes dentro del servidor se comunican entre ellos y como a su vez es comunicado con la base de datos para persistir datos o consultarlos.

7.2. Sprint 2

El desarrollo de este sprint se centró en introducir la entidad nueva “Entidad habilitada”, con su lógica de negocio adecuada e interacción con el resto de componentes del anterior sprint.

Según el *Anexo X - Registro de Autorizaciones para el uso de Aplicaciones Comerciales* de la FEGA (*Fondo Español de Garantía Agraria*), se establece cual es el procedimiento para otorgar las autorizaciones necesarias a los diferentes agentes que van a interactuar con el CUE Comercial, es decir, con la aplicación desarrollada Olivaris.

Entre los agentes que describen, se encuentra el de **Entidad habilitada**. Esta entidad tendrá relación directa con los agricultores y podrá usar todos los CUEs Comerciales que desee. A partri de aquí, se supuso que en el sistema iba a existir esta nueva entidad, por lo que se actualizó el esquema de base de datos al siguiente:

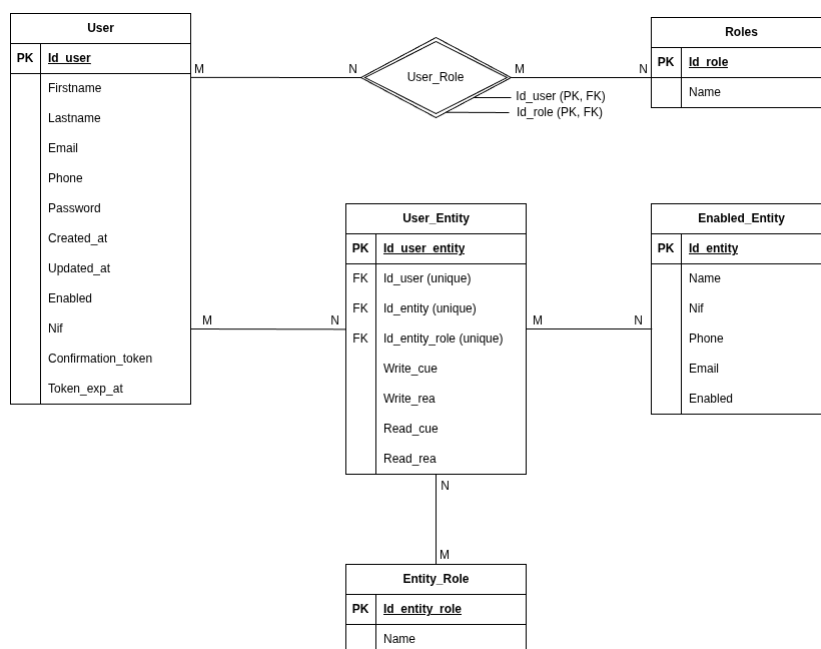


Figura 7.3: Esquema de la base de datos del Sprint 2

En este esquema se puede ver como la “Entidad habilitada” es una nueva entidad o tabla de la base de datos. Esta entidad tiene unos campos mínimos que la identifican y está relacionada directamente con un usuario, de manera, que un usuario que está registrado en el sistema puede pertenecer a una entidad también. Al pertenecer a dicha entidad, va a tener un rol específico, siendo **FARMER** (agricultor) o **ADMIN**. Además de estas características, hay otros campos añadidos a la relación que establece los permisos que el usuario delega sobre ella.

Estos permisos son cuatro: escribir sobre el REA, leer del REA, escribir en el CUE, leer el CUE. Estos permisos se han fijado de esta manera de acuerdo al *ANEXO VIII - Modelo de Autorización de Representación*. En este anexo se establece que un usuario delega sobre una entidad los permisos siguientes:

- Acceso información del REA de la explotación por la que es el/la titular.
- Acceso cumplimentación del REA de la explotación por la que es el/la titular.
- Acceso información del CUE de la explotación por la que es el/la titular.
- Acceso cumplimentación del CUE de la explotación por la que es el/la titular.

Con estos cuatro permisos, un usuario vinculado a la entidad con rol *farmer*, delegará sobre ella todos los permisos de manipulación de datos tanto del REA como del CUE, por lo que estos campos aparecerán con los valores de *true*. En caso de estar vinculado el usuario con la entidad con rol *admin*, estos permisos tendrán valor *null*, ya que un usuario administrador no está delegando ninguna autorización a manipular sus datos sobre la entidad.

El servicio creado que agrupa toda esta lógica es **EntityService** (se decidió acortar el nombre a “entidad” directamente para no tratarla como “entidad habilitada”).

7.2.1. EntityService

Este servicio es el que agrupa la lógica de negocio de asignación de los usuarios a una entidad, así como su gestión y su edición de datos.

El servicio es representado como una interfaz con los siguientes métodos:

```
public interface EntityService {  
    ResponseEntity<EntityDto> create(CreateEntity request);  
  
    ResponseEntity<UserEntityDto> createAssignment(  
        Long entityId,  
        CreateUserEntity body  
    );  
  
    UserEntityDto assignUserToEntity(  
        Long entityId,  
        CreateUserEntity body  
    );  
  
    ResponseEntity<UserEntityDto> updateUserToEntityData(  
        Long entityId,  
        Long userId,  
        UpdateUserEntity body  
    );  
  
    ResponseEntity<Void> delete(Long entityId, Long userId);  
    ResponseEntity<List<EntityDto>> getUserEntities(Long userId);  
}
```

Esta sería la implementación de la interfaz en Spring Boot. Los métodos estarán implementados en una clase llamada **EntityServiceImpl** que será la encargada de implementar los métodos de la interfaz. A continuación se va a proceder a explicar la lógica de negocio que tiene cada uno de ellos.

- **Create:** este método es el encargado de crear una entidad en el sistema con la información mínima de acuerdo a lo establecido en la tabla de la base de datos.
- **CreateAssgiment:** este es el método encargado de asignar una entidad ya creada en el sistema y un usuario registrado. Para realizar la asignación se realiza una llamada al método *AssignUserToEntity*. La implementación no se hizo en este método directamente porque la lógica se reutiliza en *AuthService* para asignar automáticamente el rol admin al registrar un usuario de entidad.
- **AssignUserToEntity:** este método va a ser el encargado de contener toda la lógica de negocio para crear la asignación. El método recibirá el ID de la entidad e información relevante en el cuerpo de la petición como el correo del usuario y qué rol va a tener (*admin* o *farmer*). Además, recibirá el valor que tendrá cada uno de los permisos ya comentados anteriormente: *writeCue*, *writeRea*, *readCue*, *readRea*. En caso de querer que el usuario creado sea un

admin de la entidad, el valor de los permisos será *null*, es decir, no se pasan directamente en el cuerpo de la petición.

Hay que aclarar que aunque se ofrezca esta flexibilidad en la forma de otorgar permisos a la entidad, actualmente todos los permisos dados tienen el valor *true* en caso de que el usuario creado tenga el rol *farmer*. En caso de que el usuario no quiera otorgar permisos no se vincularía directamente con la entidad, ya que de acuerdo al anexo comentado anteriormente, el usuario cede todos los permisos de gestión a la entidad en caso de pertenecer a ella.

A continuación se puede ver el código del método implementado:

```
public UserEntityDto assignUserToEntity(
    Long entityId,
    CreateUserEntity body
) {
    User userDb = userRep.findByEmail(body.getEmail())
        .orElseThrow(() -> new UserNotFoundException(
            "El usuario no existe en el sistema"
        ));

    EnabledEntity entityDb = entityRep.findById(entityId)
        .orElseThrow(() -> new EntityNotFoundException(
            "La entidad habilitada no existe en la base de datos"
        ));

    EntityRole entityRoleDb = entityRoleRep.findByName(
        body.getEntityRole().toString()
    )
        .orElseThrow(() -> new EntityNotFoundException(
            "El rol dentro de la entidad no
            existe en la base de datos"
        ));

    Boolean writeCue = null;
    Boolean writeRea = null;
    Boolean readCue = null;
    Boolean readRea = null;

    // If role is farmer -> check the permissions
    //                               from request body
    // Else if admin role -> permissions will be null
    if(entityRoleDb.getName()
        .equals(EntityRoleTypes.ROLE_FARMER.toString()))
    {
        writeCue = body.getWriteCue() != null ?
            body.getWriteCue() : false;
        writeRea = body.getWriteRea() != null ?
            body.getWriteRea() : false;
    }
```

```

        readCue = body.getReadCue() != null ?
            body.getReadCue() : false;
        readRea = body.getReadRea() != null ?
            body.getReadRea() : false;
    }

    UserEntityRole newUserEntityRole = new UserEntityRole(
        userDb,
        entityDb,
        entityRoleDb,
        writeCue,
        writeRea,
        readCue,
        readRea
    );

    UserEntityRole userEntityRoleDb = userEntityRoleRep
        .save(newUserEntityRole);

    return UserEntityDto.fromEntity(userEntityRoleDb);
}

```

- **UpdateUserToEntityData:** este método permite actualizar los datos de la relación entre el usuario y una entidad. Con datos se refiere a los permisos que el usuario tiene así como su rol. Esta funcionalidad se ha dejado en el backend pero desde el frontend no ha sido creada, es decir, ahora mismo no se contempla la modificación de los permisos o roles del usuario vinculado; el usuario se crea con unos datos fijos y en caso de ser modificados, se eliminaría su asignación y se crearía de la nueva manera. Esta decisión ha sido tomada para priorizar otras funcionalidades que se consideran más imprescindibles en la aplicación.

En cuanto a la lógica de negocio implementada, lo que se hace es obtener el objeto que representa un valor en la relación (tabla que relaciona usuario con entidad) y modificar cada uno de los valores según son pasados en el cuerpo de la petición, por ejemplo: si el rol es cambiado a *admin*, solo se modifica el campo pasado.

- **Delete:** este método va a eliminar una relación entre un usuario y una entidad, de manera que dejan de estar vinculados pero la entidad sigue existiendo en el sistema. Hay otro método implementado (*DeleteEntity*) que va a eliminar directamente la entidad de todo el sistema.

Estos métodos van a recibir los ID de la entidad y del usuario (en caso de eliminar el vínculo).

- **GetUserEntities:** este método permite obtener todas las entidades a las que pertenece un usuario. La implementación del método es la siguiente:

```

public ResponseEntity<List<EntityDto>> getUserEntities(
    Long userId
) {
    // Check if user exists
    User userDb = userRep.findById(userId)
        .orElseThrow(() -> new UserNotFoundException(
            "El usuario no existe en el sistema"
        ));

    // Get the user entities that belong to him
    List<UserEntityRole> entitiesList = userEntityRoleRep
        .findEntityByUserId(userId);

    List<EntityDto> entitiesDto = entitiesList.stream()
        .map(EntityDto::fromUserEntityRole)
        .toList();

    return ResponseEntity.status(HttpStatus.ACCEPTED)
        .body(entitiesDto);
}

```

Se puede ver como primero se busca el usuario en el sistema y, en caso de existir, se busca en todas las entidades en la tabla que almacena las relaciones.

7.2.2. Migraciones de Bases de datos

Durante la realización del sistema ha habido que modificar algún atributo, relación o estructura de la base de datos. Para evitar eliminar toda la información almacenada en ella al aplicar los cambios, se ha recurrido al uso de **Flyway**.

Flyway [44] es una herramienta *open source* que permite realizar migraciones de bases de datos. Una migración de bases de datos consiste en transferir información o datos de un esquema a otro mientras se cambia la estructura del sistema de almacenamiento.

Para utilizar Flyway en Java, se han necesitado las dependencias: *spring-boot-starter-flyway*, que permite el uso de Flyway en Spring Boot, y *flyway-database-postgresql*, que permite la comunicación con PostgreSQL. Además de esto, en el archivo de configuración de propiedades de Spring Boot (*application.properties*) se han añadido las propiedades que permiten a Flyway trabajar con bases de datos existentes sin necesidad de crear una (*baseline-on-migrate=true*), y otra para establecer el inicio de las migraciones desde el valor 1 (*baseline-version=0*). Esto último se ha hecho así puesto que en el primer uso de flyway, se creó una tabla interna en la base de datos que lleva un registro de las migraciones realizadas empezando desde la primera desde 0

Las migraciones se realizan a través de archivos escritos en **SQL**, siendo

nombrados de la forma: *V(nº de versión)_(lo que va a realizar)*. Algunos de los archivos creados han sido:

- **V1__modify_user_entity_role_tables.sql**: esta migración se creó para quitar una tabla de relación extra entre el usuario y la entidad que controlaba los permisos del usuario vinculado a la entidad. Se decidió que se podía prescindir de la relación e introducir los permisos en la relación directa entre ambas entidades (como está actualmente).
- **V4__uppercase_user_plot_plotname.sql**: esta migración realizó la modificación de todos los nombres de las fincas a mayúsculas, con la finalidad de evitar problemas de búsquedas e inserción de nombres con las letras en minúsculas o mayúsculas.

7.2.3. Implementación de Integración Continua

La **Integración Continua (CI)** [45] es un componente muy importante dentro del mundo *Devops*. Es la primera parte del flujo de trabajo automatizado que utilizan los Devops para agilizar la entrega de un producto software. Con esta fase se busca automatizar la integración de los cambios de código de varios contribuidores en un único proyecto de software.

Para este proyecto, la integración continua ha sido desarrollada utilizando la funcionalidad de **GitHub Actions** ofrecida por GitHub. GitHub Actions es una plataforma de integración y despliegue continua (CI/CD) que permite automatizar las pruebas, compilación y despliegue de un proyecto.

Para implementarlo, ha sido necesario crear dentro del proyecto una carpeta llamada *workflows* dentro de la carpeta *.github*. El archivo que contiene la carpeta se llama **test-and-build.yml**. El contenido del archivo es el siguiente:

```
name: Test and Build

on:
  push:
    branches: [ "main", "sprint-1" ]

jobs:
  test_and_build:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ./olivaris-app

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```

- name: Set up JDK 21
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: maven

- name: Run test
  run: mvn clean test

- name: Build with Maven
  run: mvn -B package -DskipTests

- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: olivaris-app/target/surefire-reports/

```

Entre las partes más importantes del archivo se encuentran las siguientes:

- **on:** esta parte va a definir los eventos que van a activar el *workflow*. En este caso, cuando se haga un *push* sobre la rama “main” o “sprint 1” del repositorio, se activará el workflow.
- **jobs:** en esta parte se indican las tareas que se van a ejecutar. En este caso, se indica que el trabajo va a ejecutarse con la imagen *ubuntu-latest* junto con el directorio de trabajo sobre el que se ejecutarán los comandos *run*.
- **steps:** a partir de aquí se define paso a paso lo que se va a hacer. Primero se empieza clonando el repositorio para, a continuación, instalar y configurar el JDK de Java con las propiedades correspondientes. Finalmente se ejecutará una limpieza de los tests para ejecutarlos de nuevo, comprobando de esta manera que el código subido al repositorio es correcto y pasa las pruebas. A continuación, empaqueta el proyecto de java saltando los tests y siempre sube al proyecto de Github un informe con los resultados de los tests pasados.

7.2.4. Tests en Spring Boot

Las **pruebas (tests) de software** [46] son el proceso de evaluar y verificar que un producto o aplicación de software funciona correctamente, de forma segura y eficiente, de acuerdo con sus requisitos específicos.

Los pruebas de software se van a clasificar en distintos tipos:

- **Pruebas de unidad:** son pruebas que validan que cada unidad del software funcione según lo esperado. La unidad es considerada como el componente comprobable más pequeño de la aplicación.

- **Pruebas de integración:** garantiza que los distintos componentes que constituyen el software funcionen juntos de manera efectiva.
- **Pruebas del sistema:** este tipo de pruebas buscan comprobar si el sistema en completo rinde de la forma adecuada. Esta parte incluye todo tipo de pruebas como las funcionales, no funcionales, de interfaz, de estrés y de recuperación.
- **Pruebas de aceptación:** estas pruebas tienen como objetivo verificar que todo el sistema funciona según lo previsto.

Para poder realizar tests en Spring Boot, en este proyecto se ha utilizado el framework **JUnit**, siendo el más popular para realizar tests en Java y el encargado de realizar la ejecución de las pruebas. Además de este framework, se han recurrido a otras herramientas como **Mockito**, que permite crear “objetos simulados” para aislar el código y no depender de dependencias externas.

Los tests se crean a través de clases con métodos, siendo los métodos los propios tests. Dependiendo lo que se quiera probar, se van a utilizar unas anotaciones u otras de JUnit. Un ejemplo podría ser el siguiente:

```
@DataJpaTest
public class UserRepoTest {

    @Autowired
    private UserRepository userRep;

    @Autowired
    private RoleRepository roleRep;

    @Test
    public void createUserTest() {
        Role adminRole = roleRep.findByName("ROLE_ADMIN")
            .orElseThrow(() -> new RuntimeException(
                "ROLE_ADMIN not found"
            ));

        User savedUser = userRep.save(
            UserFixtures.createBasicUser(adminRole)
        );

        assertThat(savedUser).isNotNull();
        assertThat(savedUser.getId()).isGreaterThan(0);
    }
}
```

En este test se pueden encontrar distintas anotaciones:

- **@DataJpaTest:** es una anotación que indica a Spring Boot que en esa clase se van a hacer pruebas sobre repositorios (consultas sobre la base de datos). De esta forma, Spring Boot solo va a cargar los componentes necesarios sin necesidad de cargar todos (servicios, modelos, etc), optimizando de esta

forma la velocidad de ejecución de los tests.

- **@Autowired:** indica a Spring Boot que las variables se van a instanciar a través de la inyección de dependencias correspondientes (en este caso *UserRepository* y *RoleRepository*).
- **@Test:** indica a Spring Boot que ese método es un test.

En este ejemplo dado se está realizando un test de repositorio, ya que se está guardando un usuario en la base de datos para comprobar a continuación que tiene un ID superior a 0 y que no es nulo.

Otras de las anotaciones que se pueden dar a las clases podrían ser:

- **@SpringBootTest:** esta anotación indica a Spring Boot que cargue todo el contexto de la aplicación junto con los componentes. Es muy útil cuando se van a realizar tests que requieren cargar todas las capas (repositorio, controlador y servicio).
- **@WebMvcTest:** esta anotación es muy útil cuando se quieren realizar pruebas de controlador solo, ya que carga componentes relacionados con controladores y manejadores de excepciones (entre otros); no carga servicios ni repositorios.

A parte del test mostrado en el ejemplo, se han realizado otros de **repositorio**, como por ejemplo para comprobar si se ha asignado correctamente un usuario a una entidad; otros **unitarios**, como por ejemplo el siguiente:

```
@Test
public void deleteEntityClearsActivity
    ReferencesBeforeRemovingEntity()
{
    ResponseEntity<Void> response = entityService
        .deleteEntity(1L);

    assertEquals(HttpStatus.NO_CONTENT, response.getStatusCode());
    verify(activityRepository).detachEntityFromActivities(1L);
    verify(entityRepository).delete(entity);
}
```

Este test permite comprobar que al ser ejecutado el método del servicio que se encarga de eliminar una entidad, la entidad es eliminada correctamente.

Además, también se han realizado test de **controlador**, para comprobar que los distintos componentes interactúan correctamente. Un ejemplo puede ser el siguiente:

```
@Test
public void createActReturnsCreatedWhenRequestIsValid()
    throws Exception
{
    ActivityCreatedResponse mockResponse = ActivityCreatedResponse
        .builder()
        .activityId(99L)
```

```

        .phytoActId(List.of(201L))
        .createdAt(LocalDateTime.now())
        .message("Actividad creada")
        .build();

when(actService.create(
    eq(1L),
    eq(10L),
    eq(3L),
    any(CreateActivityRequest.class)
))
.thenReturn(ResponseEntity.status(HttpStatus.CREATED)
    .body(mockResponse));

String json = objectMapper.writeValueAsString(validRequest);

mockMvc.perform(MockMvcRequestBuilders.post(
    "/api/activity/user/1/enclosure/10"
)
    .with(user(authenticatedUser(1L, "ROLE_BASIC")))
    .param("entityId", "3")
    .contentType(MediaType.APPLICATION_JSON)
    .content(json))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.activityId").value(99L))
    .andExpect(jsonPath("$.message").value("Actividad creada"));

verify(actService).create(
    eq(1L),
    eq(10L),
    eq(3L),
    any(CreateActivityRequest.class)
);
}

```

Este test de controlador comprueba si un usuario que no pertenece a una entidad puede crear una actividad. Se puede observar como primero se mockea la respuesta de la actividad devuelta cuando se va a ejecutar el endpoint de creación y cómo se verifica a continuación que la respuesta esperada es correcta.

7.3. Sprint 3

Este sprint tuvo una duración superior a los anteriores ya que se terminó de completar la base de datos, lógica de negocio asociada al backend, nuevos modelos y controladores. Además de esto, se implementó el frontend, librerías externas para representar datos y la conexión entre el cliente y el servidor.

Una vez establecida la estructura de la base de datos y la forma de almacenar la información con respecto a las entidades y usuarios, se pasó a introducir en el esquema nuevas tablas o entidades que representen las actividades realizadas y parcelas de los usuarios. El nuevo esquema de base de datos obtenido fue el siguiente:

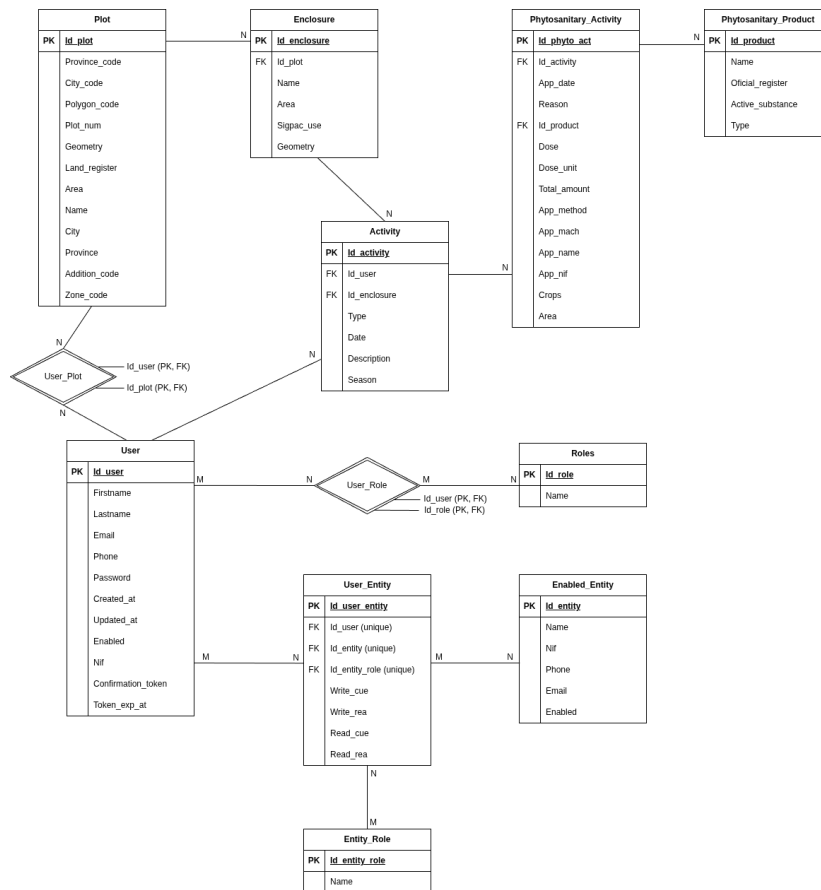


Figura 7.4: Esquema de la base de datos del Sprint 3

Este esquema ha sido el último realizado y muestra cómo ha quedado finalmente la base de datos. En la parte superior se pueden observar las nuevas tablas. Estas tablas son las siguientes:

- **Parcela:** esta tabla identifica a una parcela en el sistema. Los campos se han creado en base al contenido que se guarda en cada parcela en el Visor SIGPAC, además de otros que se han considerado importantes como la geometría.

La geometría se refiere al polígono que forma la parcela. Gracias al uso de la base de datos PostgreSQL se ha podido añadir una librería llamada **PostGIS** [47]. Esta librería permite almacenar y manipular datos geoespaciales. Entre sus características incluye:

- Almacenamiento de datos geoespaciales, como puntos, líneas, polígonos y multi-geométrías.
- Búsqueda rápida de datos espaciales basados en la localización.

- Funciones espaciales que permite realizar cálculos como distancias, áreas o intersecciones.
- Procesamiento de geometrías.

Cuando se realiza la petición a la API de SIGPAC para obtener los datos de una parcela, se va a devolver en un campo una lista de puntos con latitud y longitud. A través del uso de PostGIS, se puede almacenar en la base de datos el polígono creado como resultado de la unión de todos los puntos, identificando de esta manera a la parcela cuando se quiera representar en el frontend.

- **Usuario - Parcela:** esta tabla muestra la relación entre una parcela y un usuario, representando la asignación del usuario con una parcela. Las parcelas tienen la información pública en la API de SIGPAC, de manera que cualquiera persona podría asignarse una parcela que en realidad no le pertenece y guardar todas las actividades vinculadas a ella que quisiera. El sistema actual permite esta funcionalidad puesto que no hay una entidad externa que pueda validar que realmente esa parcela pertenece al usuario que se la ha asignado.
- **Recinto:** esta tabla contiene los datos relevantes que representan un recinto. Una parcela está compuesta por uno o más recintos, por lo que va a tener un campo que representa la geometría, además de otros como el uso. Cuando se hace una petición a la API de SIGPAC para obtener los datos de una parcela, se van a obtener en la respuesta todos los recintos. De esta manera, cada recinto de la parcela se almacena en la tabla y posteriormente, con la unión de las geometrías de todos los recintos se obtiene la geometría de la parcela (engloba a todos los recintos).
- **Actividad:** esta tabla va a representar los datos generales de una actividad, como el usuario que la realiza y sobre qué recinto se hace. Como el recinto está relacionado con la parcela, de esta forma también se puede saber sobre qué parcela se ha realizado.

Aunque se hayan contemplado para este proyecto solo actividades de tipo fitosanitario, el esquema de base de datos puede escalar a actividades de otro tipo gracias a esta tabla. Con esta tabla se consigue separar información relevante de una actividad general y a través de una tabla específica (cómo la que se verá a continuación), se puede entrar en el detalle de una actividad concreta.

- **Actividad fitosanitaria:** esta tabla representa una actividad de tipo fitosanitario. Los campos que tiene han sido dados de acuerdo al *Anexo III - Registros de los tratamientos fitosanitarios y de fertilización*, del Real Decreto 1311/2012, por el que se establece el marco de actuación para conseguir un uso sostenible de los productos fitosanitarios. En este anexo se detallan los datos generales de una explotación que van a ser usados para el cuaderno de explotación. La información es extensa y la cantidad de datos es amplia, por lo que se han escogido los que se han considerado más relevantes ahora mismo. En un futuro, en caso de ser necesario, se podrían añadir nuevos campos a la base de datos.

Algunos de los campos escogidos (de acuerdo al anexo), han sido: fecha de tratamiento, número de parcela (ya está relacionada por lo que a través del recinto se podría obtener), identificación del aplicador, tipo de cultivo, superficie tratada y producto fitosanitario.

- **Producto fitosanitario:** es la tabla que representa al producto fitosanitario utilizado en una actividad fitosanitaria.

7.3.1. Backend

Una vez explicadas las nuevas tablas añadidas a la base de datos, se va a proceder a comentar los nuevos servicios creados durante el sprint y las funcionalidades más relevantes que se han implementado a nivel de backend.

ActivityService

Este servicio es el que contiene toda la lógica de negocio relacionada con las actividades. Dentro de este servicio se encuentran los siguientes métodos:

- **Create:** este método es el encargado de crear una actividad en el sistema. Para poder ejecutarlo y crear la actividad, es necesario pasar el ID del usuario (quien realiza o crea la actividad), el ID del recinto sobre el que se realiza la actividad, el ID de la entidad bajo la que se realiza la actividad (tendrá un valor *null* en caso de no pertenecer a ninguna entidad) y el cuerpo de la petición o *body* que contiene los datos de la actividad que se va a crear (tanto los que rellenan los campos de la tabla actividad como de la tabla actividad fitosanitaria).

El código implementado para crear una actividad es el siguiente:

```
@PreAuthorize(
    "@activityValidator.canCreateActivities(
        #userId, #entityId, #body
    )"
)
public ResponseEntity<ActivityCreatedResponse> create(
    Long userId,
    Long enclosureId,
    Long entityId,
    CreateActivityRequest body
) {
    // If activity exists -> Get and update it
    // else -> create a new activity
    Optional<Activity> optionalAct = actRep
        .findByDateAndEnclosureIdAndUserIdAndTypeAndEntityId(
            body.getDate(), enclosureId, userId,
            body.getType(), entityId
        );
}
```



```

final Activity act;

if(optionalAct.isEmpty()) {
    if(!sameYear(body.getSeason(), body.getDate())) {
        throw new ActivityException(
            "La camapaña tiene que coincidir
            con el año de la actividad"
        );
    }

    act = createNewActivity(
        userId, enclosureId, entityId, body
    );
} else {
    throw new EntityExistsException(
        "La actividad ya existe en el sistema"
    );
}

if(body.getDescription() != null &&
    !body.getDescription().isBlank()) {
    act.setDescription(body.getDescription());
}

if(body.getStatus() != null) {
    act.setStatus(body.getStatus());
} else if(act.getStatus() == null) {
    act.setStatus(body.getDate()
        .isAfter(LocalDate.now()) ?
        ActivityStatus.PLANNED : ActivityStatus.COMPLETED
    );
}

// Create the phytosanitary activities
body.getPhytoAct().stream()
    .forEach(phytoActDto -> {
        PhytoAct phytoAct = phytoActService
            .createPhytoActivity(act, phytoActDto);

        act.getPhytoAct().add(phytoAct);
    });

Activity actDb = actRep.save(act);

// Create the DTO and return the response
ActivityCreatedResponse actDto = new
    ActivityCreatedResponse(

```

```

        actDb.getId(),
        actDb.getPhytoAct().stream()
            .map(pA -> pA.getId())
            .toList(),
        LocalDateTime.now(),
        "Una actividad de tipo fitosanitaria
        ha sido registrada"
    );

    return ResponseEntity.status(HttpStatus.CREATED)
        .body(actDto);
}

```

El funcionamiento del método es el siguiente:

1. Se busca si existe una actividad ya guardada en base a la fecha, recinto, usuario, entidad y tipo. Se hace de esta manera para que si coinciden todos los datos, se utilice directamente la actividad ya almacenada en el sistema, evitando la creación de una nueva.
2. Si la actividad existe en el sistema, se lanzará una excepción comunicándolo al cliente. Sin embargo, si la actividad no ha sido encontrada, se procede a crear una nueva comprobando previamente que el año de la campaña y de la fecha en que se realiza coinciden. De esta manera, se evitan crear actividades que estén fuera de la campaña actual, considerando tanto campañas cerradas como campañas que aún no han comenzado.
3. A continuación, se van comprobando si los datos opcionales se han pasado o no en el cuerpo de la petición. Por ejemplo, en caso de que no se haya pasado el estado de la actividad (planificada o completada), se comprueba si la fecha de la actividad es posterior al día actual; si es así, el estado que tendrá será planificada, de lo contrario será completada.
4. Una vez creada la actividad, se pasa a crear las actividades fitosanitarias. Los datos de estas actividades vienen a través de la lista que hay en el cuerpo de la petición. De esta manera, se recorre la lista y por cada elemento, se crea una actividad fitosanitaria y se asigna a la actividad creada.
5. Finalmente, se guarda en la base de datos la actividad junto con los datos de la actividad fitosanitaria y se crea el DTO que será devuelto en el cuerpo de la respuesta.

Hay que mencionar la anotación que aparece en la primera línea del código: **@PreAuthorize**. Las funciones que acompañan esta anotación van a devolver un valor booleano, indicando de esta forma si el método se puede ejecutar o no.

En este caso la función indica si se tiene autorización para crear actividades o

no. Para poder crear una actividad se tiene que cumplir:

- El usuario que va a crear la actividad puede hacerlo si es para sí mismo y no pertenece a ninguna entidad. También podrá si pertenece a una entidad y tiene rol de administrador en ella, ya que ningún otro miembro de la entidad podría hacerlas en su lugar. El usuario que crea la actividad podrá hacerlo para otro usuario en caso de que ambos pertenezcan a la misma entidad, el que la crea tenga rol de administrador dentro de la entidad y el usuario al que se la crea haya dado los permisos correspondientes a la entidad.
- La fecha y el estado de la actividad son correctos, es decir, una actividad no podrá tener un estado “completada” si la fecha de la actividad es futura, o una actividad no puede tener un estado “planificada” si la fecha de la actividad es pasada.
- La fecha de la campaña es correcta si el año de la campaña no es inferior al año actual, puesto que no se pueden crear actividades para campañas pasadas o cerradas.

Estas funciones que se utilizan con la anotación `@PreAuthorize` han sido usadas en muchos otros métodos para controlar quien puede ejecutar cada lógica de negocio, de acuerdo a roles y relaciones con las entidades.

- **Delete:** este método se encarga de eliminar una actividad almacenada en la base de datos. En este caso por ejemplo hay una función booleana con `@PreAuthorize` que se encarga de comprobar que usuario puede eliminar o actualizar las actividades. La lógica es muy similar al apartado anterior: podrá hacerlo un usuario que no pertenece a ninguna entidad o que pertenece a cualquiera con rol admin, o podrá hacerlo sobre otro usuario que pertenece a la misma entidad y ha dado todos los permisos a ella.
- **UpdateActivity:** este método se encarga de actualizar los datos de una actividad determinada, permitiendo cambiar datos de la actividad general como de las actividades fitosanitarias que están relacionadas con ella.
- **GetEnclosuresActByUser:** este método devuelve todas las actividades realizadas en un recinto específico por un usuario concreto. Se puede buscar todas las actividades almacenadas en el sistema o las realizadas en una campaña concreta (en caso de pasar el valor de la temporada en la petición).
- **GetUserActivites:** método que devuelve todas las actividades realizadas por un usuario del sistema.
- **AddNewPhytoActivity:** método que agrega una nueva actividad fitosanitaria sobre una actividad ya creada. Este método busca la actividad concreta a través del ID y con los datos de la actividad fitosanitaria pasados en el cuerpo de la petición, se crea una nueva para añadirla a la actividad.

EnclosureService

Este es el servicio que contiene toda la lógica de negocio relacionada con los recintos del sistema. Entre los métodos implementados se encuentran:

- **CreatePlotEnclosures:** este método se encarga de crear los recintos de una parcela. Para ello, va a recibir por parámetros una lista con todas las coordenadas de cada recinto y la parcela en la que pertenecen.

El código que se ha implementado es el siguiente:

```
public List<Enclosure> createPlotEnclosures(
    List<Map<String, Object>> enclosuresCoord, Plot plot
) {
    List<Enclosure> plotEnclosuresList = new ArrayList<>();
    int i = 1;

    // Create each enclosure and save on database
    for(Map<String, Object> enclosureMap : enclosuresCoord) {
        Map<String, Object> geometry = (Map<String, Object>)
            enclosureMap.get("geometry");
        List<List<Object>> coordinates = (List<List<Object>>)
            geometry.get("coordinates");

        Map<String, Object> properties = (Map<String, Object>)
            enclosureMap.get("properties");
        Double area = ((Number) properties.get("superficie"))
            .doubleValue();
        String use = properties.get("uso_sigpac")
            .toString();

        Polygon polygon = this.convertToPolygon(coordinates);
        String name = "Recinto " + i;
        i++;

        Enclosure enclosure = new Enclosure(
            plot,
            name.toUpperCase(),
            area,
            use,
            polygon
        );

        plotEnclosuresList.add(enclosure);
        enclosureRep.save(enclosure);
    }

    return plotEnclosuresList;
}
```

La lógica es la siguiente:

1. La función recibe la lista de coordenadas obtenidas en la respuesta de la petición a la API de SIGPAC.
2. Se recorre cada elemento de la lista y de ella se obtienen los valores de los campos *geometry* (que contiene las coordenadas de cada punto que va a formar el polígono) y *properties* (contiene otros datos auxiliares como el área del recinto y qué tipo de uso tiene según SIGPAC).
3. A través de la función **convertToPolygon**, se va a recorrer cada punto de la lista de coordenadas y se crea un objeto **Coordinate** a través de la latitud y longitud del punto que es guardado en una lista. Con cada objeto "coordinate", se va a trazar una línea que los una y finalmente se crea el polígono.
4. Finalmente, se guarda cada polígono en una lista que será devuelta y en la base de datos.

Para poder trabajar con objetos de tipo **Coordinate**, **Polygon** o **LinearRing** (línea al unir cada punto), existe en Java una API llamada **JTS (Java Topology Suite)**. Esta API permite crear objetos espaciales y usar funciones geométricas 2D. De esta manera, se pueden crear los objetos de tipo polígono que posteriormente se podrán insertar en el campo *geometry* de la base de datos, ya que PostGIS y JTS operan bajo el mismo estándar, facilitando la manipulación de los objetos desde el propio lenguaje Java a la base de datos de PostgreSQL y viceversa.

PhytoActService

Este servicio es el encargado de gestionar la lógica de negocio relacionada con las actividades fitosanitarias. Los métodos implementados son los siguientes:

- **CreatePhytoActivity:** este método es el encargado de crear una actividad fitosanitaria. Para ello, va a buscar el producto fitosanitario utilizado en el sistema y va a comprobar si la fecha de la actividad fitosanitaria está entre el día actual como mínimo y como máximo en el día de la fecha fijada por la actividad, ya que en ese rango de tiempo se pueden realizar varias actividades fitosanitarias (aplicar distintos tipos de productos). Si no estuviese así, habría que crear una nueva actividad con datos repetidos para insertar la nueva actividad fitosanitaria.
- **UpdatePhytoAct** actualiza una actividad fitosanitaria concreta según los datos que son enviados en el cuerpo de la petición.
- **UpdatePhytoActivities:** actualiza un conjunto de actividades fitosanitarias, recorriendo una lista y ejecutando el método anterior para cada elemento de la lista.

PlotService

Este servicio va a recoger toda la lógica de negocio relacionada con la gestión y manipulación de la entidad que representa una parcela. Entre los métodos más destacados que han sido implementados se encuentran los siguientes:

- **CreateUserPlot:** este método es el encargado de asignar una parcela registrada en el sistema a un usuario concreto. Para ello, se van a extraer los datos enviados en el cuerpo de la petición que representan una parcela: provincia, ciudad, código del polígono y número de parcela. De esta forma, se buscará si la parcela existe en el sistema, obteniéndola para asignársela al usuario o de lo contrario, creándola y almacenándola en el sistema. El código implementado que se encarga de crear una parcela con sus recintos es el siguiente:

```
public Plot createPlotAndEnclosures
    WithResolvedGeoData(
        CreatePlot request,
        int provinceCode,
        int cityCode,
        String landRegister
    ) {
    String province = request.getProvince()
        .trim().toUpperCase();
    String city = request.getCity()
        .trim().toUpperCase();
    String polygonCode = request.getPolygonCode().trim();
    String plotNum = request.getPlotNum().trim();

    int provinceCode = this.getProvinceCode(province);
    int cityCode = this.getCityCode(
        city, String.valueOf(provinceCode)
    );

    // Create and save the plot
    Plot newPlot = new Plot(
        String.valueOf(provinceCode),
        String.valueOf(cityCode),
        polygonCode,
        plotNum,
        landRegister,
        province,
        city
    );

    Plot plotDb = plotRep.save(newPlot);

    // Get plot enclosures from SIGPAC API
```

```

SigpacGeoJsonResponse plotEnclosures = sigpacApiClient
    .getPlotEnclosures(
        provinceCode,
        cityCode,
        0,
        0,
        Integer.valueOf(polygonCode),
        Integer.valueOf(plotNum)
    );

// Create and save the plot enclosures on the system
List<Map<String, Object>> enclosureCoord = plotEnclosures
    .getFeatures();
List<Enclosure> plotEnclosuresList = enclosureServ
    .createPlotEnclosures(enclosureCoord, plotDb);

// Calculate the area and geometry plot
List<Polygon> geometries = new ArrayList<>();
Double area = 0.0;

for(Enclosure en : plotEnclosuresList) {
    area += en.getArea();
    geometries.add(en.getGeometry());
}

if(!geometries.isEmpty()) {
    // Union() method combine all geometries
    // in one geometry
    Geometry combinedGeometry = CascadedPolygonUnion
        .union(geometries);

    if(combinedGeometry instanceof Polygon polygon) {
        plotDb.setGeometry(polygon);
    }

    plotDb.setEnclosures(plotEnclosuresList);
    plotDb.setArea(area);
    plotDb = plotRep.save(plotDb);
}

return plotDb;
}

```

La lógica seguida es la siguiente:

1. Se obtienen los datos enviados en la petición. En el caso de la **referencia catastral**, se obtiene a través de la petición a un endpoint de la API de

SIGPAC que devuelve la referencia catastral entre los datos de la parcela, evitando de esta manera que se tenga que introducir de forma manual (es una cadena de caracteres grande).

2. Se procede a realizar las peticiones a la API de SIGPAC para obtener los códigos del municipio y de la provincia a través de las funciones **getProvinceCode** y **getCityCode**.
 3. Se crea el objeto parcela (*plot*) con toda esta información y se almacena en la base de datos.
 4. Se obtienen todos los recintos de la parcela especificada a través de la petición a la API de SIGPAC (función **getPlotEnclosures**).
 5. Se extrae el valor del campo *features*, que contiene toda la información de cada recinto que compone la parcela y se ejecuta la función explicada anteriormente que va a crear el objeto polígono de JTS para cada recinto, devolviendo una lista con todos ellos.
 6. Finalmente, se crea la parcela a través de la unión de cada geometría de los recintos (función *union* de *CascadedPolygonUnion*).
- **DeleteUserPlot:** método encargado de eliminar la relación entre un usuario y una parcela a través del ID del usuario y el ID de la parcela.
 - **GetPlot:** método que busca una parcela en el sistema según el ID especificado y devuelve el objeto si es encontrado; de lo contrario lanzará una excepción.
 - **GetPlotEnclosures:** método que busca una parcela en base a su ID y devuelve un objeto que contiene una lista con cada recinto que la compone.
 - **GetUserPlots:** este método va a buscar un usuario en la base de datos en base a su ID. En caso de existir, va a obtener todas las parcelas que tiene asignadas, obtiene los recintos de cada una de ellas, los almacena en una lista y finalmente devuelve la lista con toda la información.
 - **UpdateUserPlot:** método que se encarga de actualizar los datos de una parcela asignada a un usuario. Permite modificar tanto el nombre de la parcela como el registro catastral (en caso de ser necesario).

PhytoProductService

La lógica de negocio implementada en este servicio está relacionada con los productos fitosanitarios. Los métodos implementados son los siguientes:

- **Create:** método que se encarga de crear un producto fitosanitario a partir de la información pasada y lo almacena en el sistema.
- **GetAllPhytoProduct:** método que busca todos los productos fitosanitarios almacenados en el sistema y los devuelve para su consulta.

7.3.2. Frontend

Como ha sido comentado anteriormente, el *stack* utilizado para el desarrollo del frontend ha sido React Native con Expo y TypeScript. Entre los archivos más importantes se encuentran:

- **package.json:** archivo que contiene todas las dependencias utilizadas en el proyecto, como por ejemplo *expo-router* para la navegación entre pantallas.
- **_layout.tsx** archivo que define las rutas a las que se pueda navegar en la aplicación usando expo-router. Este archivo se encuentra en el directorio *app*.
- **app.json** y **eas.json:** definen la configuración de Expo.
- **tsconfig.json:** define la configuración de TypeScript.

En cuanto a la estructura del frontend, algunos de los directorios que muestran como se ha organizado el proyecto se ha organizado son los siguientes:

- **app:** directorio que contiene cada uno de los archivos que representan las páginas o entradas a las que acceden el *router* de expo.
- **components:** directorio que contiene todos los componentes reutilizables, como por ejemplo: formularios, modales, barra de navegación y mapas (entre otros).
- **screens:** directorio que contiene todas las pantallas que se muestran al navegar por la aplicación. Estas pantallas agrupan componentes y son mostradas cuando se accede a alguna de las rutas establecidas en el directorio *app*.
- **services:** directorio que contiene la lógica de negocio y manejo de las respuestas al realizar llamadas a la API, pudiendo filtrar datos y evitando manipular la respuesta directamente desde el archivo que realiza la petición.
- **api:** contiene los clientes HTTP orientado a las entidades del backend, de manera que cada archivo contiene solo las peticiones hacia los endpoints que son de su competencia, por ejemplo: el archivo *plotApi.tsx* realiza las peticiones solo hacia el controlador de *plot* del backend.
- **types:** define los DTOs y modelos en TypeScript para que haya un tipado fuerte.
- **assets:** almacena recursos estáticos como las imágenes o iconos utilizados.
- **styles:** aunque los estilos de los componentes se definen en el propio archivo donde se crea el componente, en este directorio se almacenan los temas y estilos compartidos en distintos puntos de la aplicación.

Flujo entre capas

A través del uso de expo-router, se van a definir todas las rutas en el directorio *app*. Cuando se navega entre pantallas, lo que está ocurriendo “por detrás”, es un

acceso a la ruta concreta que corresponde a un archivo de este directorio. El archivo en cuestión va a devolver una pantalla (*screen*), y esta *screen* devuelve los componentes almacenados en *components* que sean necesarios para que se vea la pantalla de la aplicación correctamente.

Componentes

Algunos de los componentes más interesantes que se han implementado son los siguientes:

- **Mapa:** el componente *Map* proporciona la visualización del espacio geográfico de cada parcela almacenada en la aplicación. Está implementado en *map.tsx* y utiliza principalmente los servicios:
 - **location.tsx:** este servicio se encarga de gestionar el acceso a la ubicación del dispositivo. Comprueba si el usuario ha dado permisos a usar la ubicación y, si es así, obtiene la ubicación almacenada en caché si ha accedido recientemente a ella o de lo contrario, extrae la ubicación actual y la almacena en una caché.
 - **mapService.tsx:** este servicio se encarga de transformar el formato de las ubicaciones pasadas desde el backend al formato que necesita Leaflet, ya que el componente *MapView* muestra este componente.

El diseño del componente se basa en la usabilidad táctil y la eficiencia en la actualización de datos.

Para su desarrollo, se han utilizado dos librerías principalmente: **Leaflet** y **OpenStreetMap**. Leaflet es una librería de JavaScript de código abierto que permite generar mapas interactivos. Está caracterizada por su simplicidad y usabilidad, además de tener muchas funcionalidades extras a través de *plugins* y una amplia documentación de su API. Con esta librería se ha podido dar funcionalidades al mapa dibujado en la aplicación, como el poder hacer *zoom* o mover el mapa.

Por el otro lado, está OpenStreetMap. Esta herramienta [48] consiste en una base de datos cartográfica mantenida por una comunidad, convirtiéndola en *open source*. Es distribuida gratuitamente y es muy utilizada para crear mapas digitales.

Para que Leaflet pueda funcionar, se ha integrado en el HTML a través del componente *WebView*, permitiendo de esta manera mostrar las parcelas y construir un mapa interactivo sin depender de una vista nativa compleja. Una de las partes más importantes del código es la siguiente:

```
const map = L.map('map', {
  zoomControl: true,
  attributionControl: true,
});

L.tileLayer('https://tile.openstreetmap.org
```

```

        /{z}/{x}/{y}.png', {
      maxZoom: 20,
      attribution: '&copy; OpenStreetMap contributors',
    }).addTo(map);

const bounds = L.latLngBounds([]);

if(location && Number.isFinite(location.latitude) &&
  Number.isFinite(location.longitude)) {
  const currentLocation = [
    location.latitude,
    location.longitude
  ];
  L.marker(currentLocation).addTo(map)
    .bindPopup('Ubicación actual');
  bounds.extend(currentLocation);
}

payload.polygons.forEach((polygon) => {
  const coords = polygon.coords.map(
    (coord) => [coord.latitude, coord.longitude]
  );

  if(coords.length < 3) {
    return;
  }

  const layer = L.polygon(coords, {
    color: polygon.type === 'plot' ?
      '#17751e' : '#2e5bd8',
    weight: 2,
    fillColor: polygon.type === 'plot' ?
      '#3f5e2c' : '#2e5bd8',
    fillOpacity: 0.28,
  });

  layer.on('click', () => {
    window.ReactNativeWebView.postMessage(JSON.stringify({
      type: 'polygon',
      polygonId: polygon.id,
    }));
  });

  layer.addTo(map);
  const layerBounds = layer.getBounds();

  if(layerBounds.isValid()) {

```

```

        bounds.extend(layerBounds);
    }
});

if(bounds.isValid()) {
    map.fitBounds(bounds.pad(0.2));
} else if(location && Number.isFinite(location.latitude) &&
    Number.isFinite(location.longitude)) {
    map.setView([location.latitude, location.longitude], 16);
} else {
    map.setView([40.4168, -3.7038], 6);
}

```

La explicación del código es la siguiente:

1. Se crea el mapa, inicializando el componente de Leaflet con la instrucción *L.map* y añadiéndole las propiedades que permiten hacerle zoom y que el usuario se pueda mover por él, además de mostrar la información del proveedor cartográfico (*attributionControl*).
2. Se añade la capa de OpenStreetMap que va a mostrar el mapa, añadiendo un límite de zoom.
3. Se crea un objeto *bounds*, que será el que contenga todas las coordenadas que se pintarán en el mapa.
4. Se comprueba si existe una ubicación válida del usuario (a través de la ubicación del dispositivo). Si es así, se crea un marcador que muestra la ubicación actual del usuario junto con una etiqueta.
5. A continuación, se recorren todos los polígonos que se han obtenido como resultado de la consulta al backend y se transforman en el formato que necesita Leaflet para poder representarlos.
6. Se comprueba si el polígono tiene al menos 3 puntos y si es así, se crea una capa que representa el polígono junto con el color que lo representa.
7. Se añade sobre el polígono un evento de click que será enviado a React Native en el caso de ser activado, indicando la acción a realizar y el ID del polígono.
8. Se añade el polígono al mapa y se obtienen sus límites (*bounds*), pudiendo de esta forma el mapa saber cuanto es el área total ocupada por todos los elementos, utilizándolo en el siguiente paso.
9. Finalmente, se decide como cuadrar la vista: si hay límites válidos ajusta el zoom para que se vean todos los polígonos, si no hay límites pero sí ubicación, se centra el mapa según la ubicación, y si o hay ninguna de las dos, se centra en Madrid (centro de España).

El componente ha quedado finalmente de la siguiente manera:

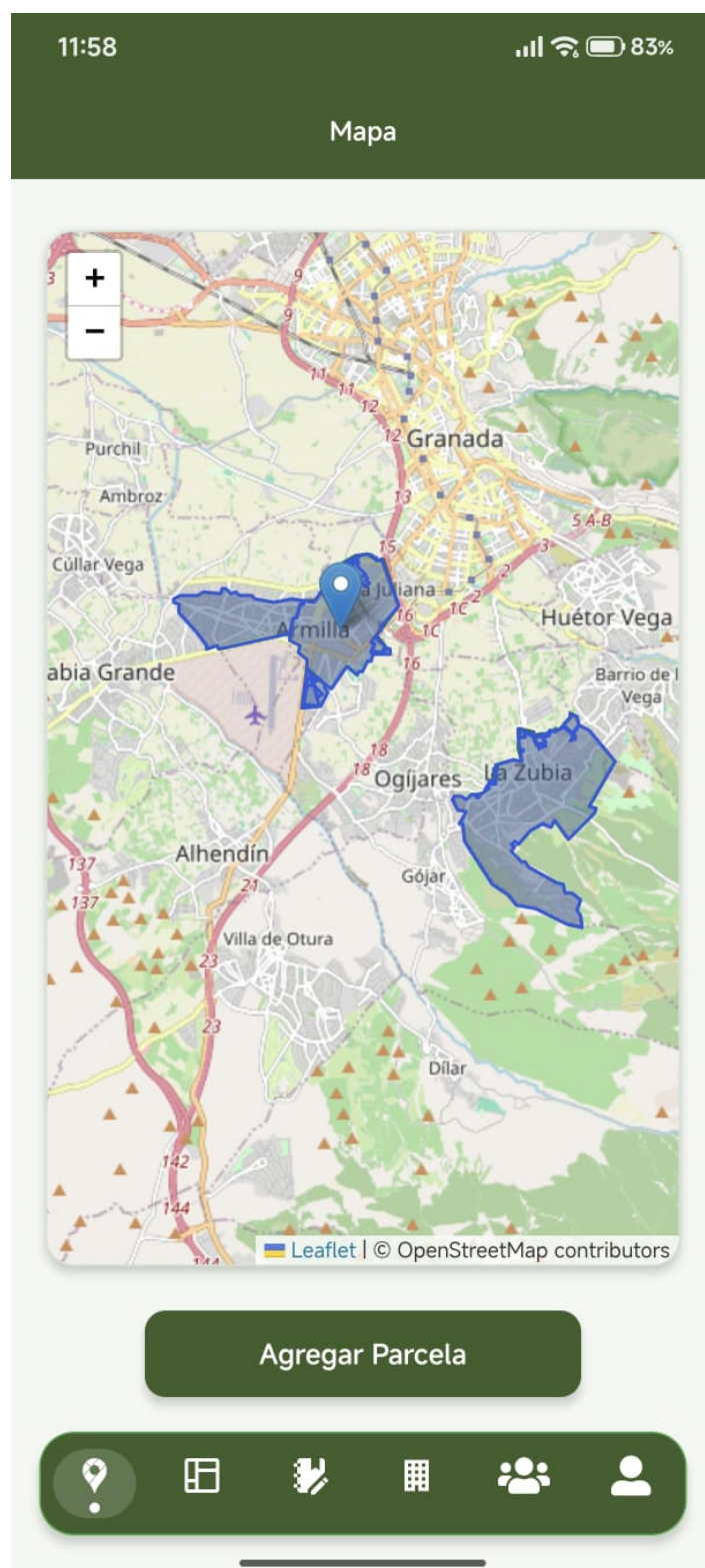


Figura 7.5: Componente del mapa visto desde la aplicación

La imagen anterior corresponde a la pantalla de inicio cuando se accede a la aplicación. El resto de pantallas (siendo un usuario administrador del sistema) son las siguientes:

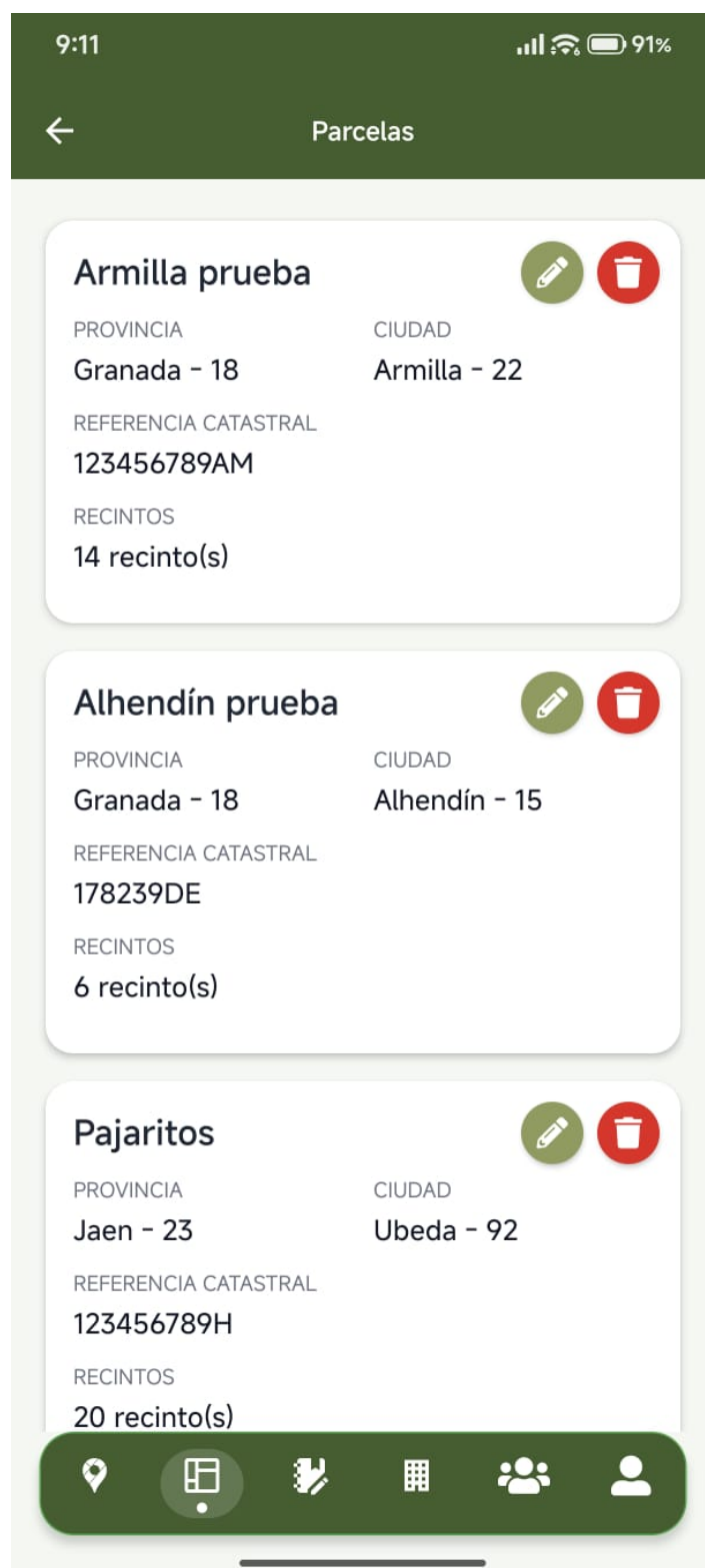


Figura 7.6: Pantalla de parcelas



Figura 7.7: Pantalla de actividades

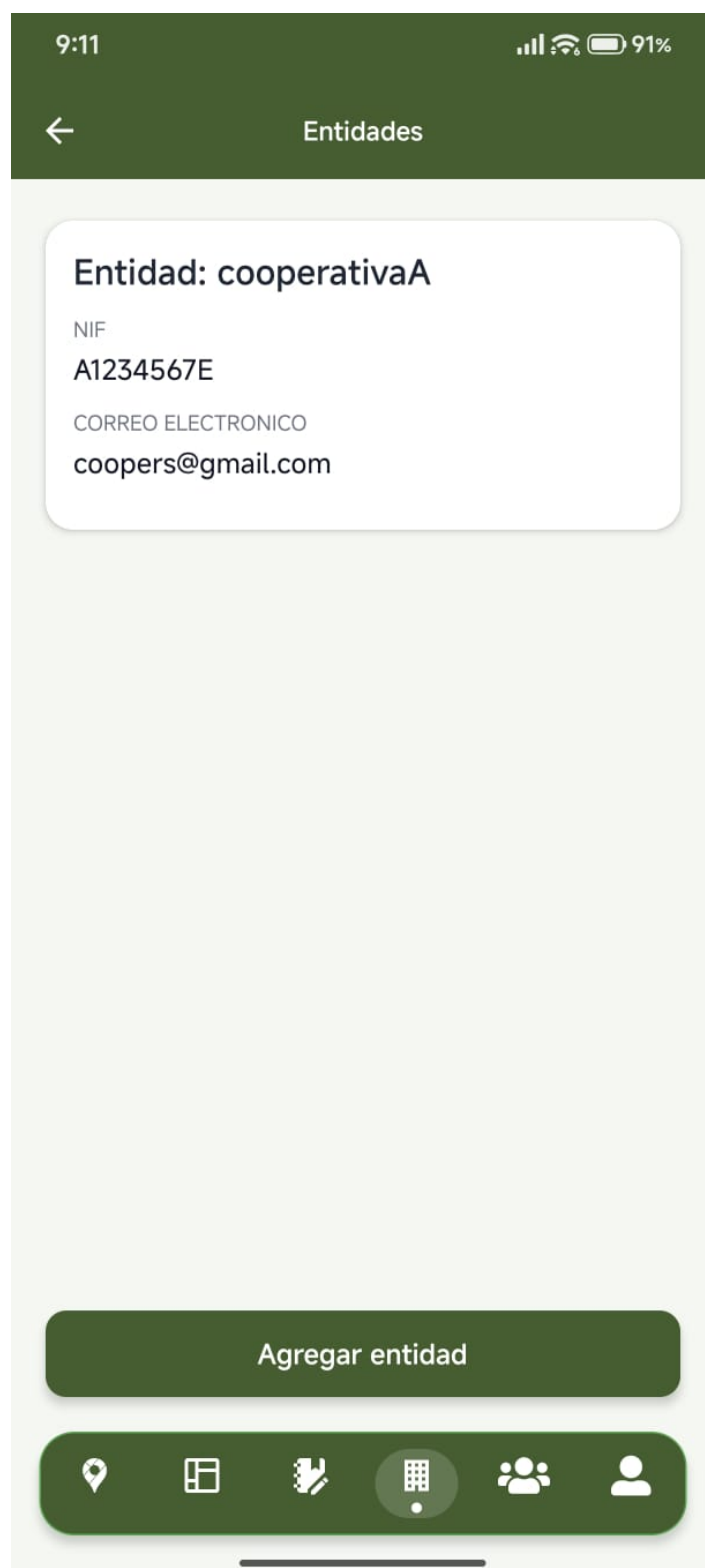


Figura 7.8: Pantalla de entidades

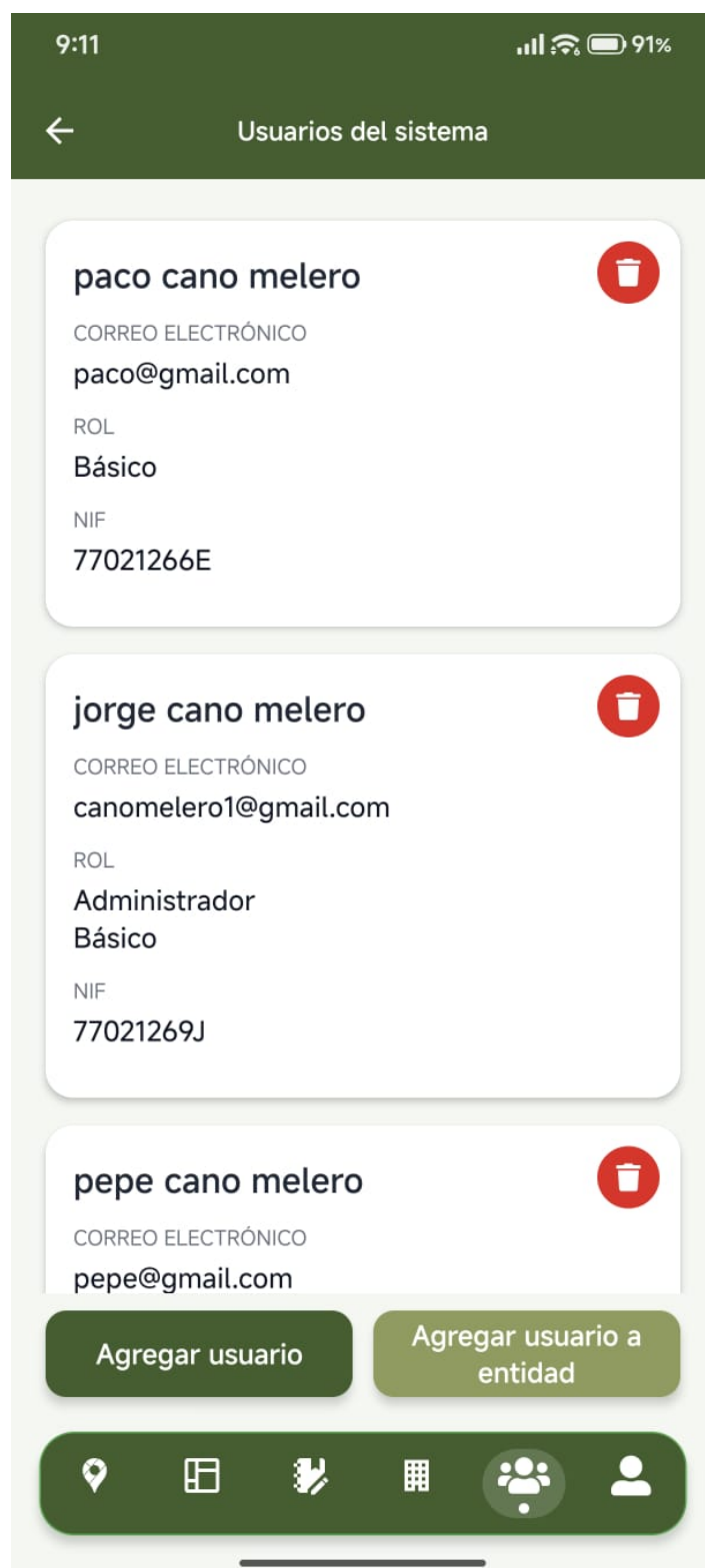


Figura 7.9: Pantalla de usuarios del sistema

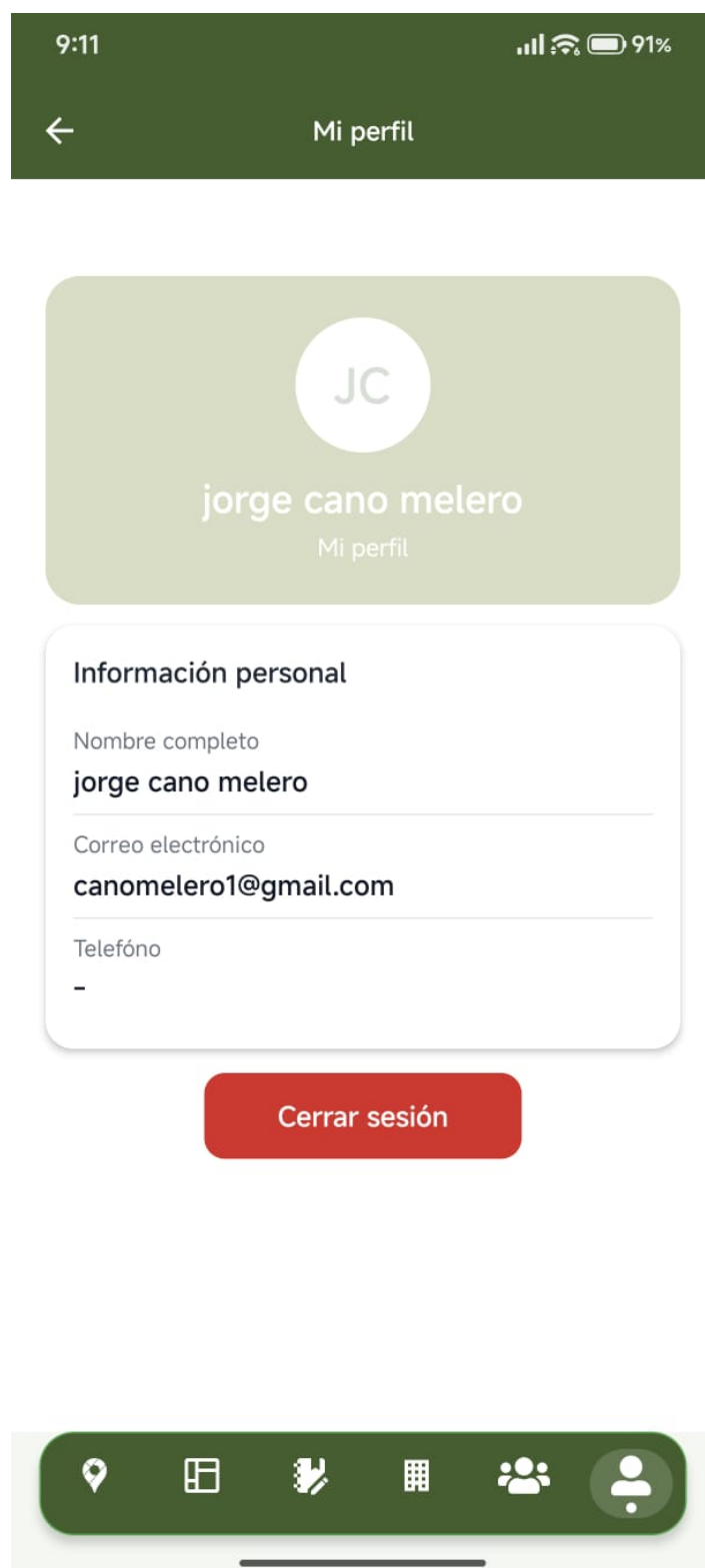


Figura 7.10: Pantalla del perfil de usuario

7.4. Sprint 4

Este sprint ha sido el último y se ha basado en investigar cómo desplegar el backend en un servidor en la nube y cómo generar la APK del frontend para que cualquier dispositivo móvil pueda descargarla y utilizarla. Además, ha servido para mejorar detalles de usabilidad en el frontend y añadir otras funcionalidades a la aplicación que se han considerado interesantes.

7.4.1. Despliegue con Docker

El despliegue del backend se ha realizado con **Docker**. Docker [27] es una plataforma que permite crear y desplegar rápidamente aplicaciones en distintos sistemas y entornos. Para ello, empaqueta toda la aplicación (software) en unas unidades llamadas **contenedores**. Estas unidades incluyen todo lo necesario para que el software se ejecute, además de bibliotecas, herramientas, código y la versión ejecutable.

El backend está desarrollado como una aplicación de Spring Boot con Maven definida a través del archivo **pom.xml**. Este archivo es de gran importancia ya que es el que contiene todas las dependencias utilizadas en el sistema junto con sus versiones, por lo que es necesario conocerlas para Docker pueda empaquetarlas y ejecutar el sistema en cualquier entorno.

Dockerfile

Este archivo define cómo Docker va a construir la aplicación en una **imagen**. Una imagen es una plantilla que contiene todo lo necesario para poder ejecutar una aplicación. El *dockerfile* implementado hace lo siguiente:

1. Usa una imagen ya creada en DockerHub llamada **eclipse-temurin:21-jdk-jammy** para compilar un proyecto en Java.
2. Copia todas las dependencias de maven definidas en el *pom.xml* así como la configuración necesaria.
3. A continuación, ejecuta maven para descargar todas las dependencias del proyecto y realiza una compilación para tener el archivo *.jar*.

En este punto del dockerfile se tiene el proyecto compilado a falta de ejecutarse. Este archivo continua de la siguiente manera:

1. Una vez compilado el proyecto, se utiliza la imagen **eclipse-temurin:21-jre-jammy**. Esta imagen va a permitir ejecutar el *.jar* generado de la fase anterior de compilación.
2. A continuación, se crea un usuario sin privilegios para ejecutar la aplicación desde ese usuario y no desde un usuario root que pueda tener todos los permisos (esta decisión se realiza por motivos de seguridad, para que si

alguien consiguiera acceder al contenedor no tenga todos los privilegios del sistema).

3. Finalmente el archivo es ejecutado a través de los comandos establecidos en la sentencia *ENTRYPOINT* cuando el contenedor arranque.

Docker compose

Además del archivo anterior, se ha creado otro llamado **docker-compose.yml**. Este archivo es el encargado de “orquestrar” cada servicio, es decir, creará una instancia de cada imagen y coordinará los contenedores. Para entender la diferencia con el archivo anterior, Dockerfile es quién establece como crear una imagen y cómo se ejecuta, mientras que Docker Compose se encarga de coordinar y gestionar varios contenedores que trabajan en conjunto.

En este archivo se han levantado 3 contenedores distintos:

- **Base de datos de PostgreSQL/PostGIS:** este servicio utiliza la imagen **postgis/postgis:16-3.4-alpine**. Esta elección se debe al uso de una base de datos que tiene instalada la herramienta PostGIS para poder trabajar con objetos geométricos y datos geoespaciales. También se van a definir las variables que permiten el acceso a la base de datos, como el nombre de usuario y contraseña, y se va a montar un volumen persistente para que los datos no se pierdan cuando el contenedor sea detenido.

Este archivo va a especificar como Docker tiene que crear la base de datos y qué contenido almacenar en ella. Para ello, se le indicará a Docker qué fichero tiene que copiar en el directorio para que PostgreSQL pueda ejecutarlo la primera vez que se crea la base de datos.

- **Aplicación de Spring Boot:** este servicio va a esperar a que la base de datos esté creada e inicializada. Una vez hecho esto, se va a levantar la imagen creada en el Dockerfile que contiene la aplicación de Spring Boot compilada y ejecutada.

En este servicio se van a exponer variables de entorno que son extraídas desde el archivo *.env* para que pueda funcionar de esta manera tanto en local como en un servidor remoto sin necesidad de manipular el código fuente de la aplicación.

Finalmente, la aplicación se expone en el puerto 8080 para que otros contenedores dentro de la misma red puedan acceder a ella y no esté accesible directamente desde el exterior; de esto se encargará Nginx.

- **Nginx como proxy inverso:** Nginx es un servidor web que actúa como proxy inverso, quitando la exposición directamente de las aplicaciones al exterior y actuando como un intermediario que se encarga de distribuir el tráfico y sirviendo como puerta de entrada al servidor. Su configuración se va a definir en un archivo llamado **default.conf**.

En este archivo se le va a indicar a Nginx que esté escuchando en el puerto 443 con certificado SSL, de manera que si se le envía una petición, Nginx la reenvía al contenedor de la aplicación que está expuesto en el puerto 8080.

Se ha tomado la decisión de usar Nginx como proxy inverso para no exponer la aplicación directamente al exterior, mejorando de esta forma la seguridad y centralizando el tráfico web. Además, si se quiere encriptar la información a través de HTTPS, Nginx permite manejar certificados TLS y la redirección de HTTP a HTTPS directamente.

Con este archivo se consigue que todo el entorno funcione de forma conjunta y coordinada a través de un solo comando: **docker compose up**. Con esta práctica se consigue como resultado un sistema desplegado de forma modular, reproducible y más fácil de mover entre entornos.

8. Evaluación y experimentación

En este capítulo se presentan las pruebas de carga realizadas para evaluar el rendimiento y la estabilidad del sistema desarrollado. Estas pruebas permiten verificar cómo se comporta la aplicación bajo diferentes condiciones de uso, simulando múltiples usuarios concurrentes y diferentes niveles de carga.

8.1. Métricas de evaluación

Las pruebas de carga se han diseñado para medir y analizar las siguientes métricas clave:

- **Tiempo de respuesta:** es el tiempo que tarda el servidor en responder a las peticiones de los usuarios. Se mide en milisegundos y es fundamental para garantizar una experiencia de usuario satisfactoria. Un tiempo de respuesta bajo indica que la aplicación responde de manera rápida y eficiente.
- **Número de errores:** registra la cantidad de peticiones que fallan durante las pruebas. Los errores pueden deberse a problemas en el servidor, excepciones no controladas o timeouts. Monitorizar esta métrica permite identificar problemas de estabilidad y fiabilidad del sistema.
- **Peticiones por segundo (RPS):** mide la capacidad del sistema para procesar múltiples peticiones simultáneamente. Indica cuántas peticiones puede manejar el servidor por segundo antes de saturarse o presentar degradación en el rendimiento.
- **Comportamiento bajo carga:** evalúa cómo se degrada el rendimiento del sistema conforme aumenta el número de usuarios o peticiones simultáneas. Permite identificar el punto de ruptura del sistema y entender sus limitaciones de escalabilidad.

8.2. Locust

Locust es una herramienta de código abierto que permite realizar tests sobre una aplicación. El comportamiento es definido a través de un script en Python que permite evaluar cómo rinde el sistema ante muchas peticiones de usuarios simultáneos.

Para poder usarlo, se ha tenido que instalar la dependencia de locust en un entorno virtual en Python y, además, se han creado dos archivos. Estos archivos son los siguientes:

- **Locust_login.py:** simula usuarios concurrentes realizando autenticación mediante el endpoint `/api/auth/login`. El script verifica que las respuestas contengan el código de estado correcto y el token de acceso. Este test permite evaluar el rendimiento del sistema en una operación básica y fundamental.
- **Locust_plots.py:** evalúa un endpoint más complejo que requiere autenticación previa y retorna una mayor cantidad de datos (listado de parcelas de un usuario). Este script simula un escenario más cercano a un uso real de la aplicación con múltiples consultas encadenadas. Permite observar cómo se comporta el sistema bajo operaciones que requieren mayor procesamiento y transferencia de datos.

8.2.1. Resumen de pruebas realizadas

La siguiente tabla presenta un resumen de todas las pruebas de carga ejecutadas:

Entorno	Endpoint	Usuarios	Ramp-up	Mediana	Errores	Conclusión
CSIRC	/auth/login	100	10 s	5.2 s	0	Tiempos muy altos; sugiere recursos limitados
Localhost	/auth/login	100	10 s	<1 s	0	Rendimiento normal en local
On-premise	/auth/login	100	10 s	<1 s	0	Descarta problemas de arquitectura
On-premise	/auth/login	1000	10 s	260 ms (P95)	0	Buen comportamiento bajo carga
On-premise	/auth/login	1000	100 s	2 s	0	Degradación; aproximación a saturación
On-premise	/plots	1000	10 s	Elevado	0	Operación compleja; congestión en cola

Tabla 8.1: Resumen de pruebas de carga con Locust

8.3. Ejecución y análisis de resultados

Para ejecutar los archivos, se escribe la sentencia **locust -f nombre_archivo.py** desde la terminal. A continuación, se abre un navegador web y se escribe la URL **http://localhost:8089** (o la IP junto con el puerto que es indicado desde la terminal).

Si se han seguido estos pasos, aparecerá en la pantalla lo siguiente:

LOCUST

Host: Status: READY RPS: 0 Failures: 0%

Start new load test

Number of users (peak concurrency) *

1

Ramp up (users started/second) *

1

Host

Advanced options

One or more User class in your locustfile has no host attribute set. Please provide one in the field above.

START

Figura 8.1: Inicio de Locust

En esta interfaz se pueden ajustar los siguientes parámetros:

- **Número de usuarios:** establece cuántos usuarios concurrentes van a realizar la ejecución de la tarea establecida en el script.
- **Ramp up:** establece el tiempo en segundos en el que se van a ir añadiendo los usuarios de manera progresiva.
- **Host:** indica a sobre qué dominio se van a realizar las peticiones.

Se empezó ejecutando el script de login para 100 usuarios y un *ramp up* de 10, de manera que cada 10 segundos, se iban a ir añadiendo usuarios de forma progresiva hasta llegar a un límite de 100.

Los resultados obtenidos fueron los siguientes:

LOCUST

Host: https://olvaris.ugr.es Status: STOPPED RPS: 13.54 Failures: 0% NEW RESET

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1093	0	5600	6200	11000	5262.82	611	11193	450	14.3	0
	Aggregated	1093	0	5600	6200	11000	5262.82	611	11193	450	14.3	0

Figura 8.2: Estadísticas para 100 usuarios en el login desde el CSIRC

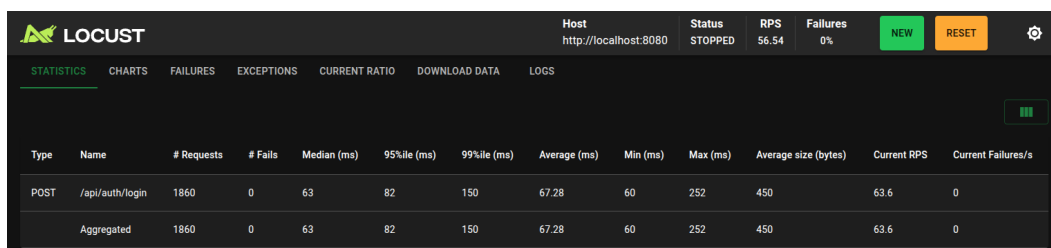
Con estos datos obtenidos se interpreta lo siguiente:

- **Request y failures:** el sistema ha procesado 1093 peticiones de login y no ha habido ningún error. Esto quiere decir que todas las respuestas tenían un código 201 y que todas ellas contenían el *accessToken* entre sus campos.
- **Median:** indica que el tiempo de respuesta de media es de 5.2 segundos, siendo un tiempo muy alto para una operación de inicio de sesión.
- **95 %ile:** esto indica que el 95 % de las peticiones tardaron menos de 6.2 segundos en resolverse. Este dato indica que en general es estable el sistema debido a que no hay mucha variabilidad aunque el tiempo sea alto.

- **99 %ile:** indica que el 99 % de las peticiones tardaron menos de 11 segundos.
- **Min y Max:** estos datos indican el tiempo mínimo o más rápido de una petición (611ms) y el tiempo más alto o más lento en procesar una petición (11s).
- **RPS:** indica el número de peticiones que el backend está procesando o “soportando” por segundo, es decir, procesa 14 peticiones de login cada segundo.

Como se puede observar, tiempos de respuesta de 5 segundos aproximadamente para una operación de login con 100 usuarios es demasiado alto. Estos resultados se obtuvieron ejecutando el sistema desde un servidor proporcionado por el CSIRC, por lo que se pasó a probarlo en local y descartar posibles errores en la arquitectura del sistema, en *queries* a la base de datos o en el código.

Ejecutando el mismo script con los mismo datos de usuarios y ramp up pero en localhost, se obtuvieron los siguientes resultados:

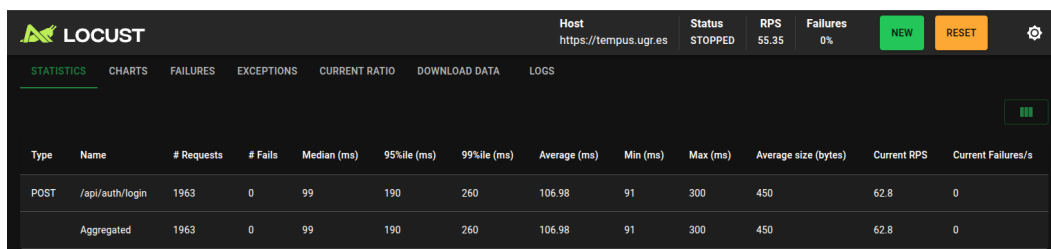


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1860	0	63	82	150	67.28	60	252	450	63.6	0
	Aggregated	1860	0	63	82	150	67.28	60	252	450	63.6	0

Figura 8.3: Estadísticas para 100 usuarios en el login desde localhost

En esta imagen se pueden apreciar resultados más lógicos para un login. Es cierto que es sobre mi PC en local, por lo que no hay una gran latencia pero los resultados se acercan más a la realidad.

Una vez comprobado que los tiempos son más razonables en local, se hizo la misma prueba pero desplegando el sistema en otro servidor *on-premise* proporcionado por el Juan Luis Laredo, tutor del presente Trabajo de Fin de Grado. Al ejecutar en este servidor el script con los mismos parámetros se obtuvieron los siguientes resultados:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1963	0	99	190	260	106.98	91	300	450	62.8	0
	Aggregated	1963	0	99	190	260	106.98	91	300	450	62.8	0

Figura 8.4: Estadísticas para 100 usuarios en el login desde on premise

En este servidor se puede observar como los datos obtenidos son también más

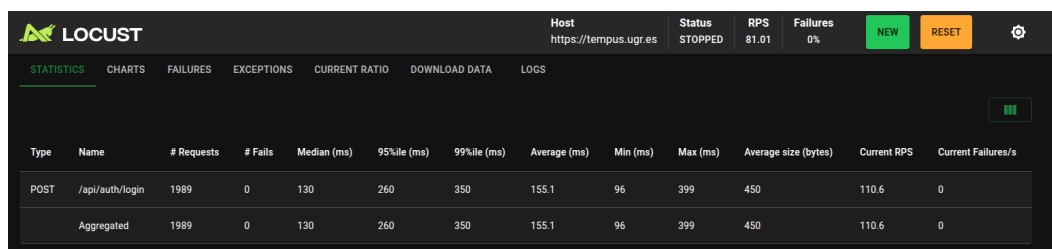
razonables para un login con 100 usuarios, ya que se tienen tiempos de respuesta que no llegan a 1 segundo.

Con estas pruebas realizadas tanto en local como con un servidor *on premise*, se descartó la hipótesis de fallos en arquitectura o en código. Los resultados sugieren que es probable que el servidor del CSIRC tenga capacidades de recursos más limitadas, lo cual podría explicar los tiempos de respuesta considerablemente más altos observados en comparación con otras infraestructuras. Sin embargo, esta conclusión se basa únicamente en el rendimiento medido y no en datos exactos de la configuración de recursos del servidor.

8.3.1. Comparación de tiempos en el servidor on premise

A continuación, se va a proceder a ejecutar el script del login en el servidor *on premise* bajo dos condiciones distintas: en una se tendrá 1000 usuarios con un ramp up de 10s, y en otra se tendrán también 1000 usuarios pero con un ramp up de 100s. Con estos datos se busca “estresar” el servidor para ver hasta qué punto puede atender peticiones bajo tiempos de respuesta razonables.

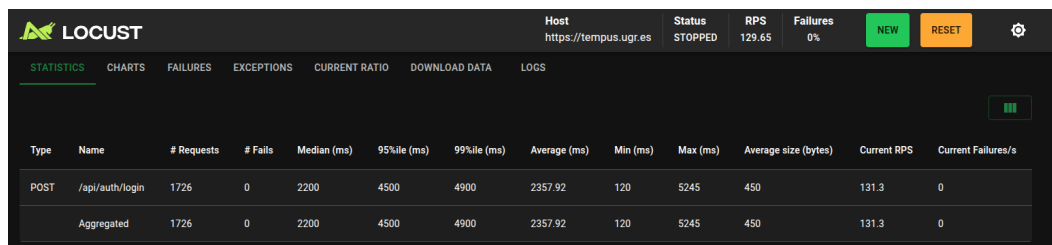
Los resultados con 1000 usuarios y ramp up de 10s son los siguientes:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1989	0	130	260	350	155.1	96	399	450	110.6	0
	Aggregated	1989	0	130	260	350	155.1	96	399	450	110.6	0

Figura 8.5: Estadísticas para 1000 usuarios en el login desde on premise

Sin embargo, los resultados con 1000 usuarios y ramp up de 100s son los siguientes:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1726	0	2200	4500	4900	2357.92	120	5245	450	131.3	0
	Aggregated	1726	0	2200	4500	4900	2357.92	120	5245	450	131.3	0

Figura 8.6: Estadísticas para 1000 usuarios en el login desde on premise

Aquí se puede observar como inyectando usuarios cada 100s, los tiempos de respuesta van aumentando llegando hasta los 2 segundos con 1700 peticiones aproximadamente. En este escenario, es probable que el servidor esté aproximándose a los límites de su capacidad de procesamiento, manteniendo unas

RPS similares a la prueba anterior mientras siguen entrando más usuarios. Esta circunstancia sugeriría que el servidor se acerca al punto de saturación, lo que podría provocar que múltiples peticiones se acumulen en cola y aumente la latencia general del sistema.

Además, otro dato relevador es el percentil 95: en la primera prueba se tiene que el 95 % de las peticiones son ejecutadas en menos de 260ms, mientras que en la segunda prueba, el 95 % de las peticiones son ejecutadas en menos de 4.5s, lo que indica hay muchas peticiones en cola esperando ser atendidas.

A continuación se va a probar el script que obtiene las parcelas de un usuario con 1000 usuarios y ramp up de 10s. Los resultados obtenidos son los siguientes:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	420	0	380	5400	5600	1213.85	222	6316	450	14.1	0
GET	/api/plot/user/userid	2890	0	610	2800	3800	949.22	182	4855	237009	89.1	0
GET	/api/user/me	420	0	75	210	390	94.82	36	836	213	14.1	0
Aggregated		3730	0	490	2900	4500	882.81	36	6316	183708.97	117.3	0

Figura 8.7: Estadísticas para 1000 usuarios en la obtención de parcelas

En este caso, se están realizando peticiones a 3 endpoints diferentes, pudiendo observar como el endpoint que obtiene todas las parcelas del usuario tiene los tiempos de respuesta más altos. Sin embargo, la mayoría de peticiones al endpoint del login presentan latencias mayores (95 % en 5s), lo que sugiere la presencia de congestión en la cola del servidor, posiblemente debido a que el volumen de peticiones se aproxima a los límites de capacidad del sistema.

8.4. Conclusiones y limitaciones de la evaluación

A través de las pruebas de carga realizadas con Locust, se ha podido extraer las siguientes conclusiones:

- **Validación de estabilidad:** el sistema ha demostrado ser funcional y estable bajo cargas moderadas (hasta 100 usuarios simultáneos), sin registrar errores en la ejecución de las operaciones críticas como autenticación y consulta de datos.
- **Identificación de cuellos de botella:** las pruebas han revelado que el sistema experimenta una degradación significativa en el rendimiento conforme aumenta el número de usuarios concurrentes, sugiriendo que existen limitaciones en la capacidad de procesamiento o en la arquitectura actual.
- **Impacto de la infraestructura:** se ha confirmado que las características del servidor de despliegue tienen una influencia considerable en los tiempos de

respuesta, siendo fundamental elegir una infraestructura adecuada para el nivel de carga esperado.

Sin embargo, es importante destacar las limitaciones de esta evaluación:

- **Simulación artificial:** las pruebas de carga simulan comportamiento de usuarios, pero no capturan la complejidad de patrones reales de uso, como distribuciones irregulares de carga, conexiones intermitentes o comportamientos impredecibles.
- **Ausencia de análisis de costos:** las pruebas no han considerado aspectos como el costo operativo de escalar la infraestructura o las optimizaciones necesarias para mejorar el rendimiento.

A pesar de estas limitaciones, las pruebas proporcionan un punto de referencia útil para entender el comportamiento del sistema bajo carga y sirven como base para futuras mejoras en rendimiento y escalabilidad.

9. Conclusiones y trabajo futuro

El objetivo principal de este Trabajo de Fin de Grado ha consistido en diseñar e implementar una plataforma digital para la gestión colaborativa de explotaciones de olivar. El trabajo parte de la necesidad de organizar de forma estructurada la información asociada a parcelas, recintos, usuarios, entidades habilitadas y actividades agrícolas, en un contexto en el que la digitalización y la trazabilidad de la información agraria adquieren cada vez más importancia.

Como respuesta a esta problemática se ha desarrollado Olivaris, un prototipo funcional basado en una arquitectura cliente-servidor. El sistema incorpora un backend con API REST, autenticación mediante JWT, control de acceso por roles, gestión de usuarios y entidades, registro de parcelas y recintos, gestión de actividades agrícolas y soporte para información geoespacial. Además, se ha desarrollado una aplicación móvil que permite interactuar con las principales funcionalidades del sistema.

Los resultados obtenidos permiten concluir que la solución propuesta es técnicamente viable. Olivaris permite centralizar información relevante de una explotación agrícola, organizarla en torno a parcelas y actividades, y gestionar distintos perfiles de usuario con permisos diferenciados. De este modo, el sistema ofrece una base funcional para mejorar la organización de la información y facilitar la colaboración entre agricultores y entidades habilitadas.

Uno de los aspectos más relevantes del proyecto es la integración de distintos componentes tecnológicos en una solución coherente: backend, base de datos, información geoespacial, aplicación móvil, mecanismos de seguridad, despliegue mediante contenedores y pruebas de rendimiento. Esta combinación permite que el sistema no se limite a una funcionalidad aislada, sino que constituya una base extensible sobre la que incorporar nuevas capacidades.

Asimismo, el diseño del sistema se ha planteado teniendo en cuenta el marco SIEX/REA/CUE, de forma que la estructura de la información pueda facilitar una posible adaptación futura a los requisitos de estas plataformas. Aunque no se ha implementado una integración efectiva con servicios oficiales, el trabajo realizado permite avanzar hacia una gestión agrícola más digital, estructurada y preparada para escenarios de interoperabilidad.

Las pruebas realizadas han permitido validar el comportamiento técnico del sistema en distintos escenarios de carga. Aunque esta evaluación no sustituye a una validación completa con usuarios reales, sí aporta una primera evidencia sobre el funcionamiento del backend y sobre la influencia que tiene la infraestructura de despliegue en el rendimiento de la aplicación.

En conjunto, Olivaris cumple los objetivos principales planteados al inicio del proyecto y demuestra que es posible construir una plataforma funcional para la gestión colaborativa del olivar utilizando tecnologías actuales, abiertas y

extensibles.

9.1. Limitaciones del sistema

A pesar de los resultados obtenidos, el sistema presenta algunas limitaciones que deben tenerse en cuenta.

En primer lugar, la plataforma no incorpora todavía una integración efectiva con los servicios oficiales asociados al marco SIEX/REA/CUE. El diseño se ha orientado a facilitar una posible compatibilidad futura, pero la conexión real con dichas plataformas queda fuera del alcance implementado.

En segundo lugar, la aplicación no dispone de funcionamiento offline. Esta limitación es relevante en el contexto agrícola, ya que muchas parcelas pueden encontrarse en zonas con conectividad limitada. En el estado actual, el uso de la aplicación depende de la conexión con el backend.

Otra limitación es que la generación de informes se encuentra en una fase inicial. Aunque permite generarlos en base a las actividades realizadas, en una herramienta de este tipo sería útil poder generar informes en base a los criterios: parcela, campaña, producto aplicado o fecha.

Asimismo, el sistema se ha centrado principalmente en una aplicación móvil. Una versión web permitiría mejorar la consulta, revisión y administración de datos desde ordenador, especialmente para entidades habilitadas o usuarios que gestionen un número elevado de parcelas.

Finalmente, la evaluación se ha centrado en pruebas técnicas y de rendimiento. Sería necesario realizar pruebas con usuarios reales del sector agrícola para valorar la usabilidad de la aplicación, la claridad de los formularios y la adecuación de los flujos de trabajo a situaciones reales.

9.2. Trabajo futuro

A partir de las limitaciones identificadas, se plantean varias líneas de trabajo futuro.

Una de las más relevantes sería avanzar en la integración con el marco SIEX/REA/CUE, estudiando los formatos de intercambio de información, los requisitos técnicos y los mecanismos de autenticación necesarios para una futura conexión con servicios oficiales.

También resultaría importante incorporar funcionamiento offline y sincronización posterior, de forma que el usuario pueda registrar actividades en campo aunque no disponga de conexión en ese momento.

Otra línea de mejora sería ampliar la generación de informes, permitiendo filtrar y exportar información por parcela, campaña, producto aplicado, tipo de actividad

o intervalo temporal. Esta funcionalidad aumentaría notablemente el valor práctico de la plataforma.

Además, sería conveniente desarrollar una interfaz web que complemente a la aplicación móvil. Esto permitiría ofrecer una experiencia más cómoda para tareas de revisión, administración, generación de informes y gestión de grandes volúmenes de información.

Por último, sería recomendable realizar pruebas de usabilidad con agricultores, técnicos agrícolas o personal de cooperativas. Esta validación permitiría detectar problemas reales de uso, simplificar formularios y priorizar mejor las funcionalidades futuras.

9.3. Conclusión final

Olivaris ha permitido demostrar la viabilidad de una plataforma digital para la gestión colaborativa del olivar, integrando en una misma solución la gestión de usuarios, entidades, parcelas, recintos, actividades agrícolas, información geoespacial y control de permisos.

Aunque el sistema presenta limitaciones y todavía existen funcionalidades por desarrollar, el trabajo realizado constituye una base sólida y extensible. En este sentido, el TFG cumple su objetivo principal: construir un prototipo funcional y técnicamente coherente que contribuye a la digitalización de la gestión agrícola y que puede servir como punto de partida para futuras evoluciones hacia una herramienta más completa y cercana a las necesidades del sector.

Bibliografía

- [1] Aceite de las Valdesas. Principales países productores de aceite de oliva, 2026. URL https://www.aceitedelasvaldesas.com/faq/varios/produccion-aceite-de-oliva-por-paises/?srsltid=AfmB0orTk2Hc_z0941GBcuBYAxz1h0AVcWQ8fFockJPVzTscrF9gl97-. Accedido el 09 de enero de 2026.
- [2] Agencia Estatal Boletín Oficial del Estado. Real decreto 1054/2022, 2026. URL <https://www.boe.es/buscar/doc.php?id=BOE-A-2022-23054>. Accedido el 15 de junio de 2026.
- [3] Pesca y Alimentación Ministerio de Agricultura. Documentación técnica agrícola siex, 2026. URL <https://www.fega.gob.es/es/siex/documentacion-tecnica-agricola-siex>. Accedido el 11 de marzo de 2026.
- [4] Pablo Ortiz Sanchidrián y Lourdes Riestra Alba. Descubrimiento de servicios para la evolución dinámica de sistemas software mediante transformación de modelos, 2009. URL https://oa.upm.es/1387/1/PFC_PABLO_ORTIZ_LOURDES_RIESTRA.pdf. Accedido el 16 de enero de 2026.
- [5] Neal Ford Mark Richards. Fundamentals of software architectures, O'Reilly, 2020. ISBN 978-1-492-04345-4.
- [6] MDN Developer Resource. Métodos de petición http, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods>. Accedido el 19 de enero de 2026.
- [7] IBM. ¿qué es graphql?, 2026. URL <https://www.ibm.com/es-es/think/topics/graphql>. Accedido el 19 de enero de 2026.
- [8] Viewnext. ¿qué es el grpc? ventajas e inconvenientes, 2022. URL <https://www.viewnext.com/que-es-el-grpc-ventajas-e-inconvenientes/>. Accedido el 19 de enero de 2026.
- [9] IBM. Soap, 2021. URL <https://www.ibm.com/docs/es/radfs/9.7.0?topic=standards-soap>. Accedido el 19 de enero de 2026.
- [10] MDN Developer Resource. Html, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTML>. Accedido el 20 de enero de 2026.
- [11] MDN Developer Resource. Css, 2026. URL <https://developer.mozilla.org/es/docs/Web/CSS>. Accedido el 20 de enero de 2026.
- [12] MDN Developer Resource. Javascript, 2026. URL <https://developer.mozilla.org/es/docs/Web/JavaScript>. Accedido el 20 de enero de 2026.
- [13] React dev. React, 2026. URL <https://es.react.dev/>. Accedido el 20 de enero de 2026.

- [14] Angular docs. Angular, 2026. URL <https://docs.angular.lat/>. Accedido el 20 de enero de 2026.
- [15] Flutter docs. Flutter, 2026. URL <https://esflutter.dev/>. Accedido el 20 de enero de 2026.
- [16] React Native docs. React native, 2026. URL <https://reactnative.dev/>. Accedido el 20 de enero de 2026.
- [17] Android developer. Kotlin, 2026. URL <https://developer.android.com/kotlin?hl=es-419>. Accedido el 20 de enero de 2026.
- [18] ESIC University. Ionic, 2025. URL <https://www.esic.edu/rethink/tecnologia/ionic-que-es-impacto-en-desarrollo-apps-c>. Accedido el 20 de enero de 2026.
- [19] Microsoft. Spring boot, 2026. URL <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-java-spring-boot#:~:text=Obt%C3%A9n%20una%20introducci%C3%B3n%20a%20Spring%20Boot%20en,que%20facilitan%20el%20desarrollo%20de%20aplicaciones%20Java>. Accedido el 20 de enero de 2026.
- [20] Amazon Web Services. Django, 2026. URL <https://aws.amazon.com/es/what-is/django/>. Accedido el 20 de enero de 2026.
- [21] FastApi docs. Fastapi, 2026. URL <https://fastapi.tiangolo.com/es/features/#fastapi-features>. Accedido el 20 de enero de 2026.
- [22] CTO en OpenWebinars Jesús Lucas Flores. Node.js, 2019. URL <https://openwebinars.net/blog/que-es-nodejs/>. Accedido el 26 de enero de 2026.
- [23] Microsoft. Bases de datos relacionales, 2026. URL <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-a-relational-database>. Accedido el 26 de enero de 2026.
- [24] Amazon Web Services. Bases de datos nosql, 2026. URL <https://aws.amazon.com/es/nosql/>. Accedido el 26 de enero de 2026.
- [25] Google Cloud. Bases de datos en la nube, 2026. URL <https://cloud.google.com/learn/what-is-a-cloud-database?hl=es-419>. Accedido el 26 de enero de 2026.
- [26] Sentries. Estrategias de despliegue: concepto y tipos, 2023. URL https://sentries.io/blog/estrategias-de-despliegue/#Tipos_de_despliegues. Accedido el 27 de enero de 2026.
- [27] Amazon Web Services. Docker, 2026. URL <https://aws.amazon.com/es/docker/>. Accedido el 27 de enero de 2026.
- [28] Documentación de Kubernetes. Kubernetes, 2026. URL <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/>. Accedido el 27 de enero de 2026.

- [29] MDN Developer Resource. Http cookies, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies>. Accedido el 29 de enero de 2026.
- [30] JWT documentation. Introduction to json web tokens, 2026. URL <https://www.jwt.io/introduction#what-is-json-web-token>. Accedido el 29 de enero de 2026.
- [31] auth0. ¿qué es oauth 2.0?, 2026. URL <https://auth0.com/es/intro-to-iam/what-is-oauth-2>. Accedido el 29 de enero de 2026.
- [32] Microsoft. ¿qué es la autenticación multifactor?, 2026. URL <https://support.microsoft.com/es-es/topic/-qu%C3%A9-es-la-autenticaci%C3%B3n-multifactor-e5e39437-121c-be60-d123-eda06bddf661>. Accedido el 29 de enero de 2026.
- [33] Atlassian. Metodología ágil scrum, 2026. URL <https://www.atlassian.com/es/agile/scrum>. Accedido el 9 de marzo de 2026.
- [34] Documentación de GitHub. Acerca de projects, 2026. URL <https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>. Accedido el 10 de marzo de 2026.
- [35] Glassdoor. Sueldos para el puesto de desarrollador full stack junior en españa, 2026. URL https://www.glassdoor.es/Sueldos/desarrollador-full-stack-junior-sueldo-SRCH_K00,31.htm. Accedido el 26 de mayo de 2026.
- [36] oSIGris. ¿qué es el cuaderno de campo digital y para qué sirve?, 2026. URL <https://osigris.com/siex/news04.php>. Accedido el 25 de febrero de 2026.
- [37] Junta de Andalucía. Cuaderno de explotación digital, 2026. URL <https://www.juntadeandalucia.es/organismos/agriculturapescaaguaydesarrollorural/areas/agricultura/cuaderno-explotacion.html>. Accedido el 25 de febrero de 2026.
- [38] Atlassian Max Rehkopf. Historias de usuario, 2026. URL <https://www.atlassian.com/es/agile/project-management/user-stories>. Accedido el 26 de febrero de 2026.
- [39] Agua y Desarrollo Rural Consejería de Agricultura, Pesca. Visor sigpac andalucía, 2026. URL <https://www.juntadeandalucia.es/organismos/agriculturapescaaguaydesarrollorural/servicios/sigpac/visor.html>. Accedido el 14 de abril de 2026.
- [40] Pesca y Alimentación Ministerio de Agricultura. Api sigpac, 2026. URL <https://sigpac-hubcloud.es/>. Accedido el 14 de abril de 2026.
- [41] Red Hat. Api rest, 2026. URL <https://www.redhat.com/es/topics/api/what-is-a-rest-api>. Accedido el 20 de abril de 2026.

- [42] Wikipedia. Hmac, 2026. URL <https://es.wikipedia.org/wiki/HMAC>. Accedido el 28 de abril de 2026.
- [43] Spring. Spring jdbc, 2026. URL <https://docs.spring.io/spring-data/relational/reference/jdbc/query-methods.html>. Accedido el 7 de mayo de 2026.
- [44] Wikipedia. Flyway, 2026. URL [https://en.wikipedia.org/wiki/Flyway_\(software\)](https://en.wikipedia.org/wiki/Flyway_(software)). Accedido el 19 de mayo de 2026.
- [45] Max Rehkopf. ¿en qué consiste la integración continua?, 2026. URL <https://www.atlassian.com/es/continuous-delivery/continuous-integration>. Accedido el 19 de mayo de 2026.
- [46] Stephanie Susnjara y Ian Smalley. ¿qué son las pruebas de software?, 2026. URL <https://www.ibm.com/es-es/think/topics/software-testing>. Accedido el 20 de mayo de 2026.
- [47] PostGIS. About postgis, 2026. URL <https://postgis.net/>. Accedido el 20 de mayo de 2026.
- [48] Wikipedia. Open street map, 2026. URL <https://en.wikipedia.org/wiki/OpenStreetMap>. Accedido el 27 de mayo de 2026.

Anexo 1: Declaración de uso responsable de herramientas de IA

Durante la elaboración de este Trabajo de Fin de Grado se han utilizado herramientas de inteligencia artificial generativa, como modelos de lenguaje de gran escala y asistentes de programación, de forma auxiliar y bajo criterios de uso ético, crítico y responsable.

El uso de estas herramientas ha estado orientado a apoyar determinadas tareas del proceso de desarrollo del proyecto, sin sustituir la auditoría personal, el criterio técnico en la toma de decisiones y la responsabilidad académica del estudiante.

En particular, las herramientas de IA se han utilizado para las siguientes finalidades:

- Revisión lingüística, mejora de estilo y reformulación de fragmentos redactados previamente por el estudiante.
- Generación de ejemplos, explicaciones alternativas o aclaraciones conceptuales.
- Apoyo en la depuración, optimización y refactorización del código desarrollado.
- Generación de fragmentos de código auxiliares posteriormente revisados, adaptados y validados de acuerdo al criterio del estudiante.

En todos los casos expuestos anteriormente, los resultados proporcionados han sido revisados críticamente. El estudiante ha comprobado la corrección técnica, la coherencia con los objetivos del trabajo, la adecuación al contexto académico y, cuando ha sido necesario, la validez de los algoritmos y fragmentos de código generados.

El uso de IA generativa no ha sustituido el trabajo intelectual propio ni la toma de decisiones fundamentales del proyecto. El diseño, desarrollo, análisis, evaluación, redacción final y conclusiones del TFG son responsabilidad del autor de este trabajo.

El estudiante declara que ha utilizado estas herramientas de forma honesta y transparente, evitando presentar como propio trabajo no comprendido, no revisado o generado automáticamente sin supervisión crítica.

Teniendo esto en cuenta, las herramientas utilizadas han sido:

- **ChatGPT y Gemini:** utilizados para mejorar la redacción de distintas partes de la memoria, buscando una mayor coherencia y estilo lingüístico. También han sido utilizados para realizar consultas a nivel técnico sobre cómo utilizar ciertas herramientas (como Locust o Docker), o cómo realizar la conexión entre los componentes.

- **Github Copilot:** ha sido utilizado para la generación de fragmentos de código del frontend principalmente, buscando dar una mejora de la usabilidad de la aplicación así como del apartado visual. También, ha sido utilizado para optimizar partes del código del despliegue con Docker.

Anexo 2: Glosario

En esta sección se van a recoger las definiciones de términos que hayan sido utilizados en la memoria:

Application Programming Interface (API) : conjunto de reglas, protocolos y herramientas que permite que dos aplicaciones de software se comuniquen e intercambien datos entre sí de forma segura.

Single Page Application : es un tipo de aplicación web que ejecuta todo su contenido en una sola página, cargando el contenido HTML, CSS y JavaScript por completo al abrir la web. Al ir pasando de una sección a otra, solo necesita cargar el contenido nuevo de forma dinámica si este lo requiere, pero no hace falta cargar la página por completo.

Progressive Web Apps : es una aplicación web que, gracias a tecnologías modernas (HTML, JavaScript, CSS), ofrece una experiencia similar a una aplicación nativa, permitiendo su instalación en la pantalla de inicio, funcionar sin conexión y enviar notificaciones push, todo desde una única base de código y sin necesidad de tiendas de aplicaciones, combinando lo mejor de la web y las apps nativas.

Client-Side Rendering : técnica que permite al navegador del usuario descargar un HTML mínimo y luego usar JavaScript para construir y renderizar el contenido de la página web dinámicamente en el cliente.

Marcos de trabajo (frameworks) : conjunto estandarizado de herramientas, librerías, reglas y componentes reutilizables que actúa como esqueleto para desarrollar software, aplicaciones web o móviles de forma rápida y eficiente.

JSON Web Token : es un estándar abierto para la transmisión segura de la información entre dos partes (típicamente un cliente y un servidor). Cada JWT es firmado digitalmente para prevenir su manipulación y además, contiene *claims* (piezas de información) sobre el usuario o la sesión.

Single Sign-on : procedimiento de autenticación que habilita a un usuario determinado para acceder a varios sistemas con una sola instancia de identificación.

JSONB : formato binario para guardar JSON que permite indexar los datos.

Certificado TLS : un certificado SSL/TLS es un objeto digital que permite a los sistemas verificar la identidad y, posteriormente, establecer una conexión de red cifrada con otro sistema mediante el protocolo Secure Sockets Layer/Transport Layer Security (SSL/TLS). Los certificados se emiten con un sistema criptográfico conocido como infraestructura de claves públicas (PKI). PKI permite que una parte establezca la identidad de otra parte mediante el uso de certificados si ambos confían en un tercero, conocido como autoridad de certificación.

Scrum : Scrum es un marco de administración que los equipos utilizan para organizarse por cuenta propia y trabajar de cara a alcanzar un objetivo común. Describe un conjunto de reuniones, herramientas y funciones para entregar proyectos de forma eficiente y permiten a los equipos de trabajo adaptarse rápidamente a los cambios.

GitHub Actions : herramienta de GitHub que permite automatizar flujos de trabajo directamente en un repositorio, permitiendo construir, testear y desplegar el código. Los **flujos de trabajo** son archivos YAML que ejecutará uno o más trabajos cuando se active un evento; por ejemplo: si un cambio es subido a un repositorio, puede haber un flujo de trabajo que realice los tests del código para comprobar que lo subido es correcto.

ORM (Object-Relational Mapping) : técnica de programación para convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y la utilización de una base de datos relacional como motor de persistencia. En la práctica esto crea una base de datos orientada a objetos virtual, sobre la base de datos relacional.

GeoJson : GeoJSON es un formato estándar abierto diseñado para representar elementos geográficos sencillos, junto con sus atributos no espaciales, basado en JavaScript Object Notation.

Inversión de Control : es un principio de Ingeniería de Software que permite transferir el control de objetos a un contenedor o framework. Para implementarlo se utiliza un patrón de diseño conocido con el nombre de Inyección de Dependencias, permitiendo la comunicación entre componentes.

Leaflet : biblioteca de JavaScript de código abierto diseñada para crear mapas interactivos en la web. Es una alternativa gratuita, rápida y muy utilizada con respecto a otras como Google Maps.

Componente WebView : componente de software utilizado en React Native que actúa como un navegador integrado dentro de una aplicación móvil o de escritorio. Permite cargar y mostrar páginas web directamente en la aplicación sin necesidad de redirigir al usuario a un navegador de su sistema.

Hook useState : tipo de *Hook* utilizado tanto en React como en React Native, que permite guardar el estado de una variable que se muestra en un componente, permitiendo mostrar los cambios en él cada vez que es cambiado su valor.

Hook useEffect : tipo de *Hook* utilizado tanto en React como en React Native, que permite sincronizar un componente con un sistema externo. Esto permite, por ejemplo, hacer peticiones a una API para obtener la información que se va a mostrar en un componente antes de ser renderizado.

CORS : también conocido como Intercambio de Recursos de Origen Cruzado o *Cross-Origin Resource Sharing*, es un mecanismo de seguridad que implementan los navegadores web, limitando las peticiones HTTP que un cliente puede hacer a un dominio distinto de aquel en el que se aloja.